

muito freqüentemente. Mesmo assim, a junção envolve diversos acessos a disco. Assim, quando for possível, é preferível ter os dados necessários a uma consulta em uma única linha, ao invés de tê-los distribuídos em diversas linhas, exigindo a sua junção.

- Diminuir o número de chaves – Para a implementação eficiente do controle da unicidade da chave primária ou chave alternativa, o SGBD usa normalmente uma estrutura de acesso auxiliar, um índice. O índice permite que o SGBD teste rapidamente a existência do valor de uma chave primária sendo incluída, sem que seja necessário fazer uma leitura exaustiva de toda tabela.

Também para o controle de chaves estrangeiras são utilizados índices. O índice é usado pelo SGBD quando uma chave primária estiver sendo excluída ou alterada. Neste caso, o índice permite o acesso rápido à tabela que contém a chave estrangeira, para verificação da existência do antigo valor da chave primária. Exemplificando, no caso do banco de dados da Figura 4.3, o índice para a chave estrangeira seria um índice na tabela *Emp* pela coluna *CodigoDepto*. Este índice seria usado, por exemplo, quando uma linha de *Dept* fosse excluída, para garantir a ausência na tabela *Emp* de linhas referenciando a linha de *Dept* excluída.

A regra geral de projeto físico de banco de dados é que, para cada chave primária, alternativa ou estrangeira, é necessário definir um índice.

Pela forma em que são implementados, índices tendem a ocupar espaço considerável em disco. Além disso, a inserção ou remoção de entradas em um índice podem exigir múltiplos acessos a disco. Assim sendo, quando for necessário escolher entre duas alternativas de implementação, deve ser preferida aquela que exige o menor número de chaves e, conseqüentemente, o menor número de índices.

Para exemplificar, vamos considerar que se deseja armazenar dados sobre clientes em um banco de dados relacional. Deseja-se armazenar, para cada cliente, seu código, seu nome, o nome da pessoa de contato, o endereço e o telefone. Estes dados poderiam ser implementados através de uma das seguintes alternativas:

```
Cliente (CodCliente, Nome,
        NomeContato, Endereço, Telefone)
```

OU

```
Cliente (CodCliente, Nome, NomeContato)
ClienteEnder (CodCliente, Endereço, Telefone)
CodCliente referencia Cliente
```

Na primeira alternativa, o SGBD cria apenas um índice por código de cliente, a chave primária da tabela. Na segunda alternativa, o SGBD cria, para cada tabela, um índice por código de cliente. Como cada cliente aparece nas duas tabelas, os dois índices possuem exatamente as mesmas entradas, resultando em armazenamento e processamento dobrados. A primeira alternativa é a preferida se considerarmos este princípio de projeto.

- **Evitar campos opcionais** – Campos opcionais são campos que podem assumir o valor vazio (NULL em SQL). Os SGBDs relacionais usualmente não desperdiçam espaço ao armazenar um campo vazio, pois usam técnicas de compressão de dados e registros de tamanho variável no armazenamento interno de linhas. Assim, em princípio, não há problemas em usar este tipo de campo.

Uma situação que pode gerar problemas é aquela na qual a obrigatoriedade ou não do preenchimento de um campo depende do valor de outros campos. Neste caso, em alguns SGBDs, o controle da obrigatoriedade deve ser feito pelos programas que acessam o banco de dados, o que deve ser evitado.

Nas seções que seguem, é explicado o processo de projeto lógico, ou seja, os passos envolvidos na transformação de um modelo conceitual ER em um modelo lógico relacional. Neste processo, são aplicados os princípios acima descritos.

O processo de projeto lógico consta dos seguintes passos:

- 1 Implementação inicial de entidades e respectivos atributos
- 2 Implementação de relacionamentos e respectivos atributos
- 3 Implementação de generalizações/especializações

5.2.1 implementação inicial de entidades

Este passo é razoavelmente óbvio: cada *entidade* é traduzida para uma *tabela*. Neste processo, cada *atributo* da entidade define uma *coluna* desta tabela. Os atributos identificadores da entidade definem as colunas que compõem a chave primária da tabela.

Trata-se aqui de uma tradução *inicial*. Pelas regras que seguem nas próximas seções, as tabelas definidas nesta etapa ainda poderão ser fundidas, no caso

de algumas alternativas de implementação de relacionamentos **1:1** e hierarquias de generalização/especialização.

A Figura 5.3 apresenta um exemplo da transformação de uma entidade em uma tabela. A figura mostra o DER e o esquema relacional correspondente. A entidade *PESSOA* com seus atributos código, nome, endereço, data de admissão e data de nascimento é transformada na tabela denominada *Pessoa* com colunas denominadas *CódigoPess*, *Nome*, *Endereço*, *DataNasc* e *DataAdm*. Como o atributo código é identificador da entidade, a coluna correspondente a este atributo é a chave primária da tabela.

■ nomes de atributos e nomes de colunas

Não é aconselhável simplesmente transcrever os nomes de atributos para nomes de colunas. Nomes de colunas serão referenciados freqüentemente em programas e outras formas de texto em computador. Assim, para diminuir o trabalho dos programadores é conveniente manter os nomes de colunas curtos. Além disso, em um SGBD relacional, o nome de uma coluna não pode conter brancos, nem hífen. Assim, nomes de atributos compostos de diversas palavras devem ser abreviados. Com base nestas considerações, os nomes de atributos data de nascimento e data de admissão foram traduzidos para os nomes de colunas *DataNasc* e *DataAdm* respectivamente.

Nas linguagens de banco de dados, o nome da tabela é muitas vezes usado como qualificador do nome da coluna. Exemplificando, para referenciar a coluna *Nome* da tabela *Pessoa* muitas vezes é usado um termo na forma *Pessoa.Nome*. Por isso, não é recomendado incluir, no nome de uma coluna, o nome da tabela em que ela aparece. Assim, é preferível usar o nome de coluna *Nome*, a usar os nomes de coluna *NomePess* ou *NomePessoa*. A exceção a esta regra é a coluna chave primária da entidade. Como esta coluna pode



Esquema relacional correspondente:

Pessoa (*CódigoPess*, *Nome*, *Endereço*, *DataNasc*, *DataAdm*)

Figura 5.3 Transformação de entidade em tabela.

aparecer em outras tabelas, na forma de chave estrangeira, é recomendável que os nomes das colunas que compõem a chave primária sejam sufixados ou prefixados com o nome ou sigla da tabela na qual aparecem como chave primária. Por este motivo, a coluna chave primária da tabela do exemplo recebeu a denominação de `CodigoPess`.

Outra recomendação quanto à nomeação de colunas é relativa ao uso de abreviaturas. Muitas vezes usa-se determinadas abreviaturas para tipos de campos que se repetem, como `Cod` para um código e `No` ou `Num` para um número. A recomendação é que se use sempre a mesma abreviatura em todo o banco de dados.

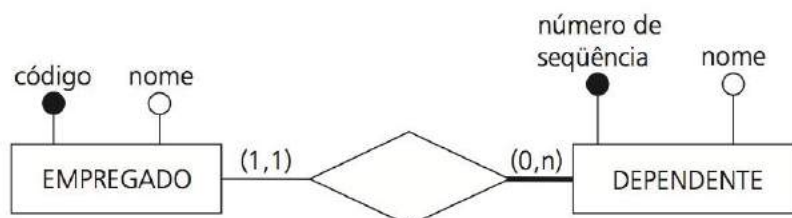
■ relacionamento identificador

A tradução de uma entidade que possui um *relacionamento identificador* tem algumas particularidades que são discutidas nesta seção.

Para iniciar, vamos considerar como exemplo a entidade *DEPENDENTE* mostrada no diagrama da Figura 5.4. Segundo este modelo, um dependente é identificado pelo código do empregado ao qual ele está vinculado e por um número de seqüência que distingue os diversos dependentes de um mesmo empregado.

A regra de implementação de relacionamentos identificadores é a seguinte:

- Para cada relacionamento identificador, é criada uma chave estrangeira na tabela que implementa a entidade identificada pelo relacionamento



Esquema relacional correspondente à entidade *DEPENDENTE*:

`Dependente (CodigoEmp, NoSeq, Nome)`

Figura 5.4 Implementação de entidade com relacionamento identificador.

identificador. Esta chave estrangeira é formada pelas colunas da chave primária da tabela referenciada pela chave estrangeira. Na Figura 5.4, isto significa que uma coluna `CódigoEmp` (chave primária da tabela correspondente à entidade *EMPREGADO*) deve ser adicionada à tabela *DEPENDENTE*.

- A chave primária da tabela que implementa a entidade identificada pelo relacionamento identificador é formada por:
 - colunas correspondentes aos atributos identificadores da entidade, se existirem, e
 - chaves estrangeiras que implementam os relacionamentos identificadores.

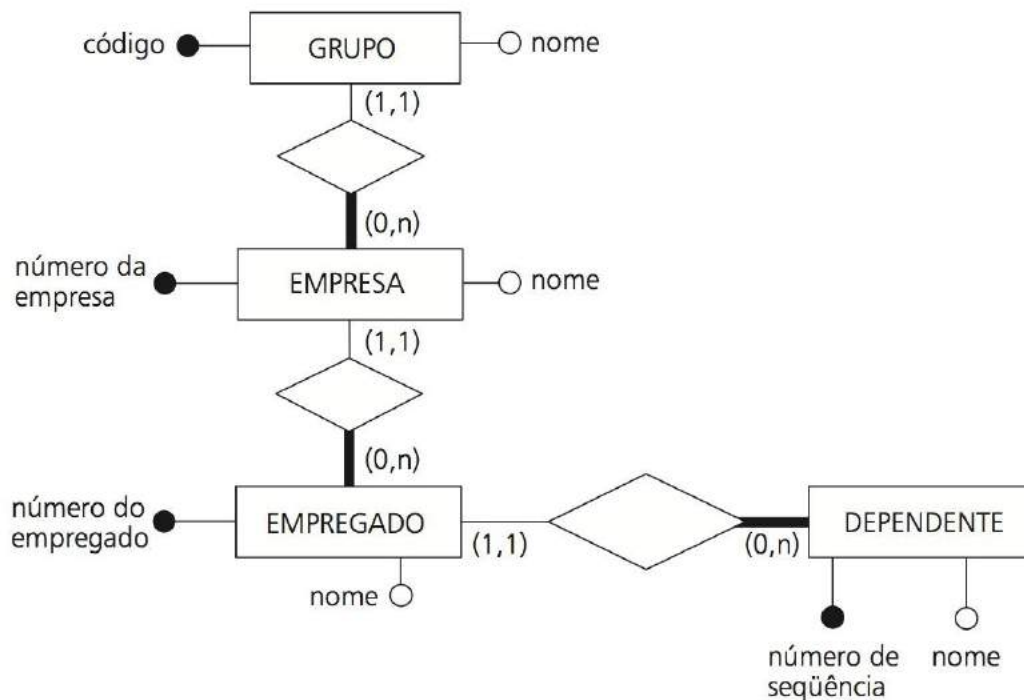
No exemplo da Figura 5.4, isto significa que a chave primária da tabela *Dependente* é composta pelas colunas `CódigoEmp` (chave estrangeira que implementa o relacionamento identificador) e `NoSeq` (coluna que implementa o atributo identificador número de seqüência da entidade *DEPENDENTE*).

Cabe observar que, quando a entidade que possui o relacionamento identificador referencia uma entidade, que, por sua vez, também tem um relacionamento identificador, é necessário propagar por vários níveis a importação de chaves estrangeiras para a chave primária.

Como exemplo, vamos considerar o modelo ER da Figura 5.5. Neste caso, para compor a chave primária da tabela *Dependente*, é necessário usar, além do número de seqüência deste dependente, também o identificador do empregado. Entretanto, um empregado é identificado por seu número e pelo identificador da empresa à qual ele está vinculado. Por sua vez, a empresa é identificada por um número e pelo identificador do grupo ao qual ela pertence. Em outros termos, um dependente é identificado pela combinação das seguintes informações:

- código do grupo da empresa à qual seu empregado está vinculado;
- número da empresa à qual seu empregado está vinculado;
- número de seu empregado e
- seu número de seqüência.

Esta linha de raciocínio leva à chave primária da tabela *Dependente* que é mostrada na Figura 5.5.



Esquema relacional correspondente:

Grupo	(CodGrup, Nome)
Empresa	(CodGrup, NoEmpresa, Nome)
Empregado	(CodGrup, NoEmpresa, NoEmpreg, Nome)
Dependente	(CodGrup, NoEmpresa, NoEmpreg, NoSeq, Nome)

Figura 5.5 Implementação de entidade com relacionamento identificador: entidade referência entidade identificada por relacionamento identificador.

5.2.2 implementação de relacionamentos

O fator determinante para a tradução a ser adotada no caso de relacionamentos é a *cardinalidade* mínima e máxima das entidades que participam do relacionamento. Antes de detalhar a alternativa proposta para cada tipo de relacionamento, apresentamos as três alternativas básicas de tradução de relacionamentos para o modelo relacional.

■ tabela própria

Nesta tradução, o relacionamento é implementado através de uma *tabela própria* que contém as seguintes colunas:

- colunas correspondentes aos identificadores das entidades relacionadas,
- colunas correspondentes aos atributos do relacionamento.

Caso o relacionamento sendo traduzido tiver a cardinalidade **n:n**, a chave primária desta tabela é formada pelas colunas correspondentes aos identificadores das entidades relacionadas e pelas colunas correspondentes aos atributos identificadores do relacionamento, caso estes existam. Cada conjunto de colunas que corresponde ao identificador de uma entidade é chave estrangeira em relação à tabela que implementa a entidade referenciada. Caso for um relacionamento com cardinalidades **1:n** ou **1:1**, a chave primária tem uma composição diferente, conforme discutido nas seções específicas para estes tipos de relacionamentos (páginas 147 e 152).

Um exemplo deste tipo de tradução é apresentado na Figura 5.6. Nesta figura, a parte do esquema do banco de dados referente à regra em questão está apresentada em negrito. Esta convenção será usada no restante da apresentação das regras de tradução de relacionamentos.

A tabela Atuação implementa o relacionamento *ATUAÇÃO*. A chave primária da tabela é formada pelas colunas *CodEng* e *CodProj*, que correspondem aos identificadores das entidades relacionadas (*ENGENHEIRO* e *PROJETO*). Cada uma destas colunas é uma chave estrangeira da tabela que implementa a entidade relacionada. A coluna *Função* corresponde ao atributo do relacionamento.



Esquema relacional correspondente:

```

Engenheiro (CodEng, Nome)
Projeto (CodProj, Título)
Atuação (CodEng, CodProj, Função)
  CodEng referencia Engenheiro
  CodProj referencia Projeto
  
```

Figura 5.6 Tradução de relacionamento por *tabela própria*.

■ adição de colunas

A outra alternativa de implementação de um relacionamento é a adição de colunas em uma das tabelas correspondentes às entidades que participam do relacionamento. Um exemplo deste tipo de tradução é apresentado na Figura 5.7. Este tipo de tradução somente é possível quando uma das entidades que participa do relacionamento tem cardinalidade máxima **1** (no exemplo, trata-se da entidade *EMPREGADO*). A tradução consta em inserir na tabela correspondente à entidade com cardinalidade máxima **1** as seguintes colunas:

- colunas correspondentes ao identificador da entidade relacionada (no exemplo, o identificador da entidade *DEPARTAMENTO*); estas colunas formam uma chave estrangeira em relação à tabela que implementa a entidade relacionada – no caso, a coluna *CodDept*,
- colunas correspondentes aos atributos do relacionamento – no caso, a coluna *DataLota*.

■ fusão de tabelas de entidades

A terceira forma de implementar um relacionamento é através da fusão das tabelas referentes às entidades envolvidas no relacionamento. Um exemplo deste tipo de tradução é apresentado na Figura 5.8. Esta tradução somente pode ser aplicada quando o relacionamento é de tipo **1:1**. A tradução consta em implementar, em uma única entidade, todos os atributos de ambas as entidades, bem como os atributos eventualmente existentes no relacionamento.



Esquema relacional correspondente:

Departamento (*CodDept*, Nome)
 Empregado (*CodEmp*, Nome, *CodDept*, *DataLota*)
CodDept referencia Departamento

Figura 5.7 Tradução de relacionamento por *adição de coluna*.



Esquema relacional correspondente:

Conferência (CodConf, Nome, DataInstComOrg, EnderComOrg)

Figura 5.8 Tradução de relacionamento através de fusão de tabelas.

5.2.3 detalhes da implementação de relacionamentos

A alternativa específica que deve ser usada na tradução de um relacionamento é determinada pelas *cardinalidades* mínima e máxima das entidades envolvidas nos relacionamentos.

A Tabela 5.1 dá uma visão geral das alternativas que podem ser usadas. Para cada combinação de cardinalidades de relacionamento, a tabela mostra qual a alternativa preferida e outras alternativas, se houver. A alternativa preferida é indicada pelo símbolo V. Para alguns tipos de relacionamentos, existem outras alternativas que geram implementação correta, mas que, pelos princípios por trás do projeto lógico (página 138), não constituem a melhor implementação. Elas são indicadas pelos símbolos $\pm$ e $\mp$, em ordem decrescente de preferência de uso. Finalmente, as alternativas que não fazem sentido, porque levam a construções inválidas na abordagem relacional, são indicadas pelo símbolo x.

Nas seções que seguem, a regra de tradução correspondente a cada um dos tipos de relacionamento é justificada e exemplificada.

5.2.4 relacionamentos 1:1

■ ambas as entidades têm participação opcional

A Figura 5.9 apresenta um exemplo de relacionamento **1:1** no qual a participação de ambas as entidades é *opcional* (a cardinalidade mínima de ambas as entidades no relacionamento é zero). De acordo com a Tabela 5.1, a alternativa preferida de tradução de relacionamentos com esta cardinalidade é a

Tabela 5.1 Alternativas para implementação de relacionamentos

Tipo de relacionamento	Regra de implementação		
	Tabela própria	Adição coluna	Fusão tabelas
Relacionamentos 1:1			
(0,1)  (0,1)	±	V	x
(0,1)  (1,1)	∓	±	V
(1,1)  (1,1)	∓	∓	V
Relacionamentos 1:n			
(0,1)  (0,n)	±	V	x
(0,1)  (1,n)	±	V	x
(1,1)  (0,n)	∓	V	x
(1,1)  (1,n)	∓	V	x
Relacionamentos n:n			
(0,n)  (0,n)	V	x	x
(0,n)  (1,n)	V	x	x
(1,n)  (1,n)	V	x	x

V: Alternativa preferida

±: Pode ser usada, primeira opção

∓: Pode ser usada, segunda opção

x: Não cabe como solução

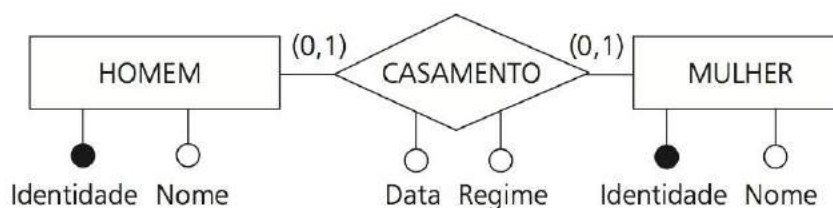


Figura 5.9 Implementação de relacionamento 1:1 com participação opcional de ambas as entidades.

adição de colunas na tabela referente a uma das entidades que participam do relacionamento. Como é um relacionamento 1:1, qualquer das entidades que participam do relacionamento pode ser a escolhida.

Uma solução poderia ser:

Mulher (IdentM, Nome, **IdentH**, **Data**, **Regime**)

IdentH referencia Homem

Homem (IdentH, Nome)

Neste esquema, as colunas referentes ao relacionamento estão marcadas em negrito. Trata-se de colunas referentes aos atributos de casamento, bem como a coluna **IdentH**, chave estrangeira que implementa o relacionamento. Neste caso, optou-se, arbitrariamente, por adicionar colunas à tabela Mulher. Da mesma forma, poderiam ter sido adicionadas colunas (identificador da mulher e atributos de casamento) à tabela Homem.

A outra alternativa seria a de gerar uma *tabela própria* para o relacionamento, conforme o esquema a seguir:

Mulher (IdentM, Nome) [Homem (IdentH, Nome)]

Casamento (IdentM, IdentH, Data, Regime)

IdentM referencia Mulher

IdentH referencia Homem

A tabela que implementa o relacionamento é a tabela Casamento. Nesta tabela, as colunas **IdentH** e **IdentM** são ambas chaves estrangeiras, implementando desta forma a vinculação da linha de casamento às linhas de homem e mulher correspondentes. Como se trata de um relacionamento 1:1, tanto a coluna **IdentH**, quanto a coluna **IdentM** podem ser consideradas para a chave primária. No exemplo, a coluna **IdentM** foi arbitrariamente escolhida como chave primária, sendo **IdentH** uma chave alternativa.

A primeira alternativa (adição de colunas) é a preferida, pois minimiza a necessidade de junções, já que os dados de uma pessoa (na opção escolhida, a mulher) estão na mesma linha que os dados do casamento.

A desvantagem da primeira alternativa é que pode levar à utilização da segunda alternativa (tabela própria) é a de basear-se no uso de *colunas opcionais*, isto é, no uso de colunas que admitem valores vazios (no exemplo, as colunas `IdentH`, `Data` e `Regime` da tabela `Mulher`). Esta alternativa transfere a responsabilidade pela verificação da opcionalidade de campos do SGBD para os programas que atualizam o banco de dados. No caso da tabela `Mulher` do exemplo, nas linhas que correspondem a mulheres que não são casadas, os campos correspondentes ao casamento devem estar todos vazios. Já nas linhas correspondentes a mulheres casadas, os três campos devem estar preenchidos. Não há linhas em que, dentre os três campos, alguns estejam vazios e outros preenchidos. O controle que garante que os três campos estejam preenchidos ou vazios não é feito pelo SGBD, mas sim pela própria aplicação. Cabe observar que em alguns SGBDs que implementam restrições de integridade (cláusula *check* ou similar) é possível transferir ao SGBD a responsabilidade pelo controle das colunas opcionais. Neste caso, a segunda alternativa seria a preferida.

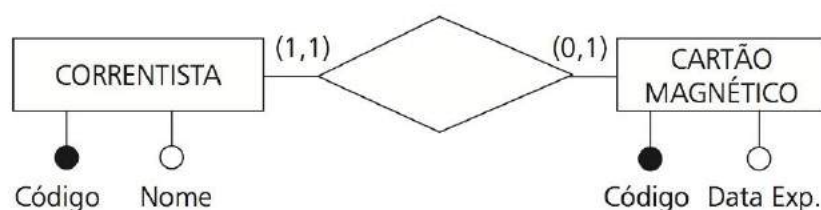
A alternativa de implementação por *fusão de tabelas* não é aplicável neste caso. A tabela resultante da fusão das tabelas referentes às duas entidades teria as seguintes colunas:

Tabela (`IdentM`, `NomeM`, `IdentH`, `NomeH`, `Data`, `Regime`)

O problema aqui é que, como ambas as entidades têm participação opcional, tanto `IdentH`, quanto `IdentM` são opcionais. Assim, nenhuma destas colunas poderia ser usada como chave primária, já que chaves primárias sempre são colunas obrigatórias.

■ uma entidade tem participação opcional e a outra tem participação obrigatória

Outro tipo de relacionamento **1:1** é aquele no qual uma das entidades tem participação *obrigatória*, enquanto que a outra entidade tem participação *opcional* (a cardinalidade mínima de uma das entidades é um, a cardinalidade mínima da outra entidade é zero). Um exemplo desta situação é apresentado na Figura 5.10.



Esquema relacional correspondente:

Correntista (CodCorrent, Nome, CodCartão, DataExp)

Figura 5.10 Implementação de relacionamento 1:1 com participação obrigatória de uma entidade e participação opcional da outra.

Neste caso, a tradução preferida é através da *fusão das tabelas* correspondentes às duas entidades.

Alternativamente, poderia ser considerada a tradução através da adição de colunas à tabela correspondente à entidade que obrigatoriamente está associada através do relacionamento em questão (no exemplo, esta entidade é cartão magnético).

Correntista (CodCorrent, Nome)

Cartão (CodCartão, DataExp, **CodCorrent**)

CodCorrent referencia Correntista

■ ambas as entidades tem participação obrigatória

O último tipo de relacionamentos 1:1 é aquele no qual ambas as entidades tem participação *obrigatória* no relacionamento (a cardinalidade mínima de ambas as entidades é um). Um exemplo desta situação é apresentado na Figura 5.11.

Neste caso, a tradução preferida é através da *fusão das tabelas* correspondentes às duas entidades.

Nenhuma das demais alternativas é adequada. Em ambas as alternativas, as entidades que participam do relacionamento seriam representadas através de duas tabelas distintas. Estas tabelas teriam a mesma chave primária e relação um-para-um entre suas linhas. Essa implementação viola os princípios de evitar junções e diminuir o número de chaves primárias estabelecidos no início do capítulo.



Esquema relacional correspondente:

Conferência (CodConf, Nome, DataInstComOrg, EnderComOrg)

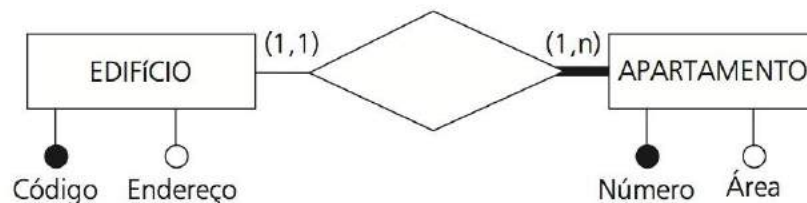
Figura 5.11 Implementação de relacionamento 1:1 com participação obrigatória de ambas as entidades.

5.2.5 relacionamentos 1:n

No caso de relacionamentos 1:n, a alternativa preferida de implementação é a de *adição de colunas* (ver na página 146).

Um exemplo desta tradução é apresentado na Figura 5.12.

Cabe observar que, neste exemplo, a coluna `CódigoEd` da tabela `Apartamento` (que implementa o relacionamento do apartamento com seu edifício), além de ser chave estrangeira, é também parte da chave primária. Esta situação é típica de uma entidade com *relacionamento identificador*, cuja tradução foi discutida na página 142.



Esquema relacional correspondente:

Edifício (CódigoEd, Endereço)

Apartamento (CódigoEd, NúmeroAp, ÁreaAp)

CódigoEd referencia Edifício

Figura 5.12 Tradução de relacionamentos 1:n através de adição de colunas.

No caso de a entidade com cardinalidade máxima **1** ser opcional, isto é, possuir cardinalidade mínima **0**, poderia ser considerada uma implementação alternativa. Um exemplo deste tipo de relacionamento é mostrado na Figura 5.13. A entidade *VENDA* está *opcionalmente* ligada à entidade *FINANCEIRA*.

Para este tipo de relacionamento, pode ser usada alternativamente a implementação através de tabela própria.

A implementação através de *adição de colunas* à tabela de entidade *Venda* (implementação preferida) é a seguinte:

```
Financeira (CodFin, Nome)
Venda (IdVend, Data, CodFin, NoParc, TxJuros)
CodFin referencia Financeira
```

As colunas em negrito na tabela *Venda* implementam o relacionamento.

A implementação através de *tabela própria* (implementação alternativa) é a seguinte:

```
Financeira (CodFin, Nome)
Venda (IdVend, Data)
Financiam (IdVend, CodFin, NoParc, TxJuros)
IdVend referencia Venda
CodFin referencia Financeira
```

A implementação por tabela própria tem duas desvantagens em relação à implementação por adição de colunas:

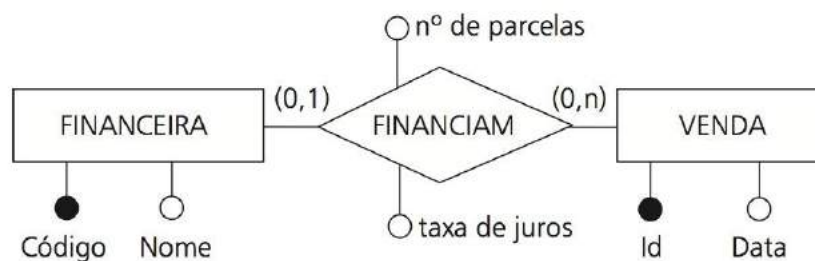


Figura 5.13 Tradução de relacionamentos **1:n** no qual a entidade com cardinalidade máxima um é opcional.

- Operações que envolvem acesso a dados de uma venda e do respectivo financiamento exigem junções. Na primeira alternativa, isto não ocorre, já que os dados da venda e de seu financiamento estão na mesma linha.
- As tabelas `Venda` e `Financiam` possuem a mesma chave primária, sendo o conjunto de valores de `Financiam` um subconjunto de `Venda`. Tem-se o problema acima mencionado de armazenamento e processamento duplicados de chave primária.

A única vantagem que a implementação por tabela própria apresenta é o fato de nela haver campos que são opcionais em certas linhas e obrigatórios em outras. Este é o caso dos campos `CodFin`, `NoParc` e `TxJuros` da tabela `Venda` na alternativa de adição de colunas. Estes campos estão obrigatoriamente preenchidos em caso de venda a prazo e vazios em caso contrário.

5.2.6 relacionamentos n:n

Independentemente da cardinalidade mínima, relacionamentos **n:n** são sempre implementados através de tabela própria. Um exemplo de implementação de relacionamentos **n:n** é apresentado na Figura 5.6.

A alternativa de adicionar colunas a uma das tabelas correspondentes às entidades que participam do relacionamento não é aplicável. Cada entidade está associada a um número variável de entidades. Para implementar o relacionamento através da adição de colunas, seria necessária uma coluna *multivalorada*, que comportasse um conjunto de valores de chaves primárias, referente à entidade associada. Entretanto, como vimos no capítulo anterior, as colunas na abordagem relacional são sempre *monovaloradas*. Assim, esta alternativa não é viável, pelas próprias características da abordagem relacional.

5.2.7 relacionamentos de grau maior que 2

As alternativas de implementação de relacionamentos apresentadas até este ponto aplicam-se somente à implementação de relacionamentos binários, isto é, que envolvem apenas duas entidades. Como visto na Seção Relacionamento ternário, na página 44, algumas variantes da abordagem ER admitem relacionamentos de grau maior que 2. Para fins de exemplo, vamos considerar o modelo ER apresentado na Figura 5.14.

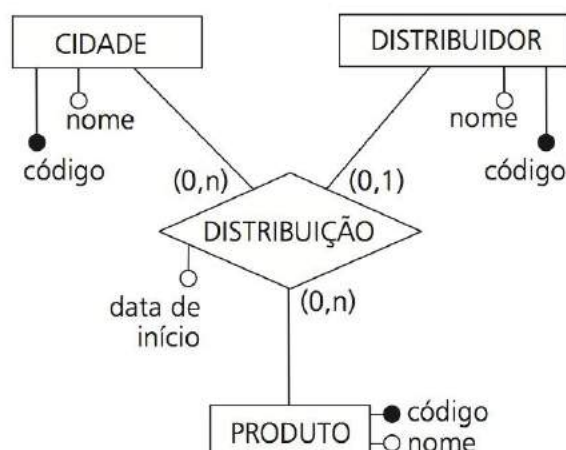


Figura 5.14 Relacionamento ternário a ser implementado.

Para relacionamentos de grau maior que 2, não são definidas regras específicas. A implementação de um relacionamento de grau maior que 2 dá-se na seguinte sequência de passos:

- 1** O relacionamento é transformado em uma entidade. Esta nova entidade é ligada através de um relacionamento binário a cada uma das entidades que participavam do relacionamento original.
- 2** As regras de implementação de entidades e relacionamentos binários apresentadas acima são aplicadas às entidades e aos relacionamentos binários assim criados.

A Figura 5.15 mostra o resultado da transformação do relacionamento ternário que aparece no modelo da Figura 5.14 em uma entidade (resultado da aplicação do passo 1 acima).

A implementação deste modelo seguindo as regras apresentadas acima resulta no seguinte esquema relacional:

```

Produto(CodProd, Nome) [Cidade (CodCid, Nome)
Distribuidor(CodDistr, Nome)
Distribuição (CodProd, CodDistr, CodCid, DataDeInicio)
    CodProd referencia Produto
    CodDistr referencia Distribuidor
    CodCid referencia Cidade
  
```

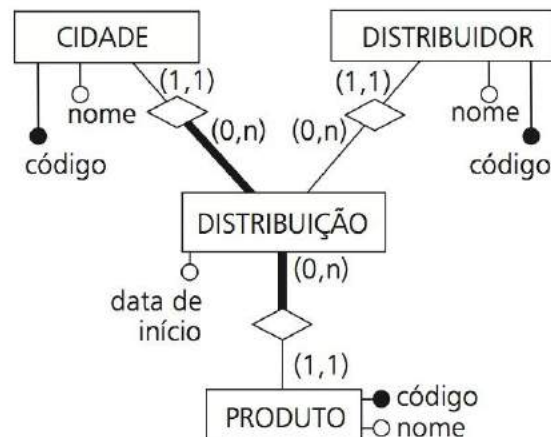


Figura 5.15 Transformação de relacionamento ternário em entidade.

5.2.8 implementação de generalização/especialização

Para a implementação de hierarquias de generalização/especialização na abordagem relacional, há duas alternativas principais a considerar: (1) uso de uma tabela para cada entidade e (2) uso de uma única tabela para toda hierarquia de generalização/especialização. A seguir apresentamos as duas alternativas para, depois, discutir quando usar cada uma.

■ uma tabela por hierarquia

Nesta alternativa, todas as tabelas referentes às especializações de uma entidade genérica são fundidas em uma única tabela, composta por:

- chave primária correspondente ao identificador da entidade mais genérica;
- caso não exista, uma coluna `Tipo`, que identifica que tipo de entidade especializada está sendo representada por cada linha da tabela;
- uma coluna para cada atributo da entidade genérica;
- colunas referentes aos relacionamentos dos quais participa a entidade genérica e que sejam implementados através da alternativa de adicionar colunas à tabela da entidade genérica;
- uma coluna para cada atributo de cada entidade especializada (estas colunas devem ser definidas como opcionais, já que somente terão valores quando a linha for referente à entidade especializada em questão);

- colunas referentes aos relacionamentos dos quais participa cada entidade especializada e que sejam implementados através da alternativa de adicionar colunas à tabela da entidade (estas colunas devem ser definidas como opcionais, já que somente terão valores quando a linha for referente à entidade especializada em questão).

Observe-se que, pela definição acima, uma entidade especializada pode não gerar uma coluna na implementação. Isto ocorrerá caso a entidade especializada não tenha atributos e caso todos os relacionamentos dos quais ela participe sejam implementados através de tabelas próprias.

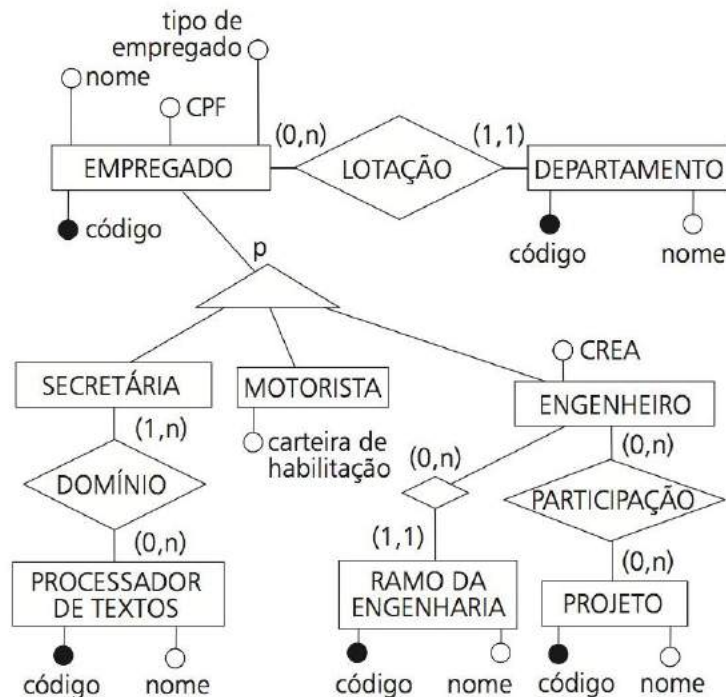
Um exemplo de implementação usando uma única tabela para toda hierarquia de especialização da entidade *EMPREGADO* aparece na Figura 5.16.

A tabela *Emp*, que implementa a entidade *EMPREGADO* e suas especializações, contém as seguintes colunas:

- *CódigoEmp*, chave primária da tabela, correspondente ao identificador da entidade;
- as colunas *Tipo*, *Nome* e *CPF* referentes aos atributos da entidade genérica;
- a coluna *CódigoDept*, que implementa o relacionamento *LOTAÇÃO*;
- a coluna *CartHabil* que implementa os atributos da entidade especializada *MOTORISTA*;
- a coluna *CREA* que implementa os atributos da entidade especializada *ENGENHEIRO*;
- a coluna *CódigoRamo*, que implementa o relacionamento entre *ENGENHEIRO* e *RAMO DA ENGENHARIA*.

Pelas regras apresentadas na seção anterior, os relacionamentos *PARTICIPAÇÃO* e *DOMÍNIO* são implementados por tabela própria, não gerando colunas na tabela correspondente à hierarquia de generalização/especialização. Além disso, a entidade *SECRETÁRIA* não gera uma coluna, já que não possui atributos, nem participa de relacionamentos que gerem colunas.

As colunas que correspondem às entidades especializadas (*CartHabil*, *CREA* e *CódigoRamo*) devem ser definidas como colunas *opcionais*. Essa definição é necessária, pois uma linha referente a um empregado, que não pertença a alguma das classes especializadas, terá todos os campos anteriormente listados



Esquema relacional correspondente:

```

Emp (CódigoEmp, Tipo, Nome, CPF, CódigoDept,
     CartHabil, CREA, CódigoRamo)
  CódigoDept referencia Depto
  CódigoRamo referencia Ramo
Depto (CódigoDept, Nome)
Ramo (CódigoRamo, Nome)
ProcessTexto (CódigoProc, Nome)
Domínio (CódigoEmp, CódigoProc)
  CódigoEmp referencia Emp
  CódigoProc referencia ProcessTexto
Projeto (CódigoProj, Nome)
Participação (CódigoEmp, CódigoProj)
  CódigoEmp referencia Emp
  CódigoProj referencia Projeto
  
```

Figura 5.16 Hierarquia de generalização/especialização e sua implementação através de tabela única.

vazios. Já uma linha correspondente a uma entidade especializada terá alguns campos vazios e outros preenchidos. Exemplificando, uma linha referente a um engenheiro teria os campos CREA e CódigoRamo preenchidos e o campo CartHabil vazio.

As demais tabelas que implementam o modelo da Figura 5.16, definidas usando as regras apresentadas nas seções anteriores, são:

- Depto, que implementa a entidade *DEPARTAMENTO*
- Ramo, que implementa a entidade *RAMO DA ENGENHARIA*
- ProcessTexto, que implementa a entidade *PROCESSADOR DE TEXTO*
- Domínio, que implementa o relacionamento *DOMÍNIO*
- Projeto, que implementa a entidade *PROJETO*
- Participação, que implementa o relacionamento *PARTICIPAÇÃO*

■ uma tabela por entidade especializada

A outra alternativa de implementação de uma hierarquia de generalização/especialização é criar uma tabela para cada entidade que compõe a hierarquia, aplicando as regras correspondentes à implementação de entidades e relacionamentos já apresentadas nas seções anteriores.

O único acréscimo que deve ser feito àquelas regras é a inclusão da chave primária da tabela correspondente à entidade genérica, em cada tabela correspondente a uma entidade especializada. Exemplificando, a implementação do modelo ER da Figura 5.16 resultaria no esquema relacional apresentado na Figura 5.17 (parte referente à hierarquia de generalização/especialização em negrito).

```

Emp (CódigoEmp, Tipo, Nome, CPF, CódigoDept)
    CódigoDept referencia Depto
Motorista (CódigoEmp, CartHabil)
    CódigoEmp referencia Emp
Engenheiro (CódigoEmp, CREA, CódigoRamo)
    CódigoEmp referencia Emp
    CódigoRamo referencia Ramo
Depto (CódigoDept, Nome)
Ramo (CódigoRamo, Nome)
ProcessTexto (CódigoProc, Nome)
Domínio (CódigoEmp, CódigoProc)
    CódigoEmp referencia Emp
    Código Proc referencia ProcessTexto
Projeto (CódigoProj, Nome)
Participação (CódigoEmp, CódigoProj)
    CódigoEmp referencia Engenheiro
    CódigoProj referencia Projeto

```

Figura 5.17 Generalização/especialização: implementação através de uma tabela por entidade especializada.

Para a entidade *EMPREGADO* e cada uma de suas especializações, foi criada uma tabela. Estas tabelas têm, todas, a mesma chave primária. A tabela *Emp* contém uma linha para cada empregado, independentemente de seu tipo. Nela aparecem as informações comuns a todos os empregados. As informações referentes a cada tipo particular de empregado estão nas tabelas *Motorista* e *Engenheiro*. Em cada uma delas, aparecem linhas somente para empregados pertencentes ao tipo representado pela tabela.

Nas tabelas referentes às entidades especializadas, a chave primária é também chave estrangeira em relação à tabela de empregados. Isso ocorre porque a toda ocorrência de uma entidade especializada corresponde uma ocorrência de entidade genérica, ou seja, a toda linha de uma tabela de entidade especializada corresponde uma linha da tabela da entidade genérica.

■ comparação entre as duas alternativas de implementação

A seguir são discutidas as vantagens de cada tipo de implementação em relação à outra.

■ *Vantagens da implementação com tabela única*

- Todos os dados referentes a uma ocorrência de entidade genérica, bem como os dados referentes a ocorrências de sua especialização, estão em uma única linha. Não há necessidade de realizar junções, quando a aplicação deseja obter dados referentes a uma ocorrência de entidade genérica juntamente com uma ocorrência de entidade especializada.
- A chave primária é armazenada uma única vez, ao contrário da alternativa com múltiplas tabelas, na qual a chave primária aparece tanto na tabela referente à entidade genérica, quanto na tabela referente à entidade especializada.

■ *Vantagens da implementação com uma tabela por entidade especializada*

- As colunas opcionais que aparecem são apenas aquelas referentes a atributos que podem ser vazios do ponto de vista da aplicação. Na solução alternativa, todas as colunas referentes a atributos e relacionamentos das entidades especializadas devem ser definidas como opcionais.
- O controle de colunas opcionais passa a ser feito pela aplicação, com base no valor da coluna TIPO e não pelo SGBD, como ocorre na solução alternativa.

O projetista deverá ponderar as vantagens e desvantagens de ambas as soluções e optar por aquela que, considerando os fatores acima, seja a mais adequada ao seu problema.

■ subdivisão da entidade genérica

Além das duas alternativas acima, alguns textos de projeto de banco de dados apresentam uma terceira implementação para a generalização/especialização. Nesta alternativa, cria-se uma tabela para cada entidade especializada que não possua outra especialização (entidade folha da árvore). Esta tabela contém tanto os dados da entidade especializada, quanto os de suas entidades genéricas. No caso do modelo ER da Figura 5.16, a implementação é a apresentada na Figura 5.18 (parte referente à hierarquia de generalização/especialização em negrito).

A diferença desta alternativa em relação a anterior é que, nesta alternativa, as tabelas correspondentes às especializações contém, não só os atributos da entidade especializada, mas também os atributos de suas generalizações. A tabela contém não só os atributos específicos da entidade *MOTORISTA*, mas também os atributos referentes à sua generalização (atributos *CódigoEmp*, *Tipo*, *Nome*, *CPF* e *CódigoDept*). De forma análoga, a tabela *Engenheiro* contém não só os atributos específicos de engenheiro, mas também os de empregado. Finalmente, a tabela *EmpOutros* contém dados de todas as demais categorias de empregados.

```

EmpOutros (CódigoEmp, Tipo, Nome, CPF, CódigoDept)
    CódigoDept referencia Depto
Motorista (CódigoEmp, Nome, CPF, CódigoDept, CartHabil)
Engenheiro (CódigoEmp, Nome, CPF, CódigoDept, CREA, CódigoRamo)
    CódigoRamo referencia Ramo
Depto (CódigoDept, Nome)
Ramo (CódigoRamo, Nome)
ProcessoTexto (CódigoProc, Nome)
Domínio (CódigoEmp, CódigoProc)
    CódigoEmp referencia EmpOutros
    CódigoProc referencia ProcessoTexto
Projeto (CódigoProj, Nome)
Participação (CódigoEmp, CódigoProj)
    CódigoEmp referencia Engenheiro
    CódigoProj referencia Projeto

```

Figura 5.18 Generalização/especialização: implementação através de subdivisão da entidade genérica.

É importante observar que, nesta alternativa, as colunas `CódigoEmp` que aparecem como chave nas tabelas referentes às diversas especializações de empregado não são chave estrangeira, já que não existe uma tabela onde todos os empregados estão reunidos, como ocorria na alternativa anterior. Além disso, quando a especialização for total (e não parcial como no exemplo), deixa de existir a tabela que coleciona as linhas referentes à entidades para as quais não há especialização (no exemplo, a tabela `EmpOutros`).

Essa alternativa tem como vantagens sobre as anteriores o fato de eliminar os problemas de colunas opcionais e chaves primárias redundantes, típicos das soluções anteriormente apresentadas.

Entretanto, esta alternativa apresenta uma desvantagem do ponto de vista funcional que supera as vantagens oferecidas quanto ao uso mais eficiente de recursos. Nesta alternativa, para garantir a unicidade da chave primária, a aplicação que faz inclusões de linhas de empregados deve verificar todas as tabelas referentes às especializações. Exemplificando, quando for incluído um novo empregado, a aplicação deverá testar a sua existência nas três tabelas (`EmpOutros`, `Motorista` e `Engenheiro`). Essa verificação fica a cargo da aplicação, já que os SGBDs relacionais não são capazes de realizá-la automaticamente.

Adicionalmente, não há como especificar ao SGBD restrições de integridade referenciais que façam referência ao conjunto de empregados como um todo (já que este não aparece no banco de dados).

5.2.9 refinamento do modelo relacional

O processo de tradução acima descrito leva a um banco de dados correto do ponto de vista da abordagem relacional e que implementa os dados que são especificados pelo modelo conceitual.

Entretanto, o banco de dados projetado não deve ser visto como final, pois pode conter redundância de dados, conforme explicado mais adiante.

Além disso, deve ser considerado que, em todo processo de engenharia, está envolvido um compromisso entre o ideal e o realizável dentro das restrições de recursos impostas pela prática. No processo de engenharia de banco de dados, particularmente na implementação de modelos conceituais, às vezes

também é necessário traçar um compromisso entre o ideal, representado pelas regras de implementação apresentadas, e o alcançável frente a limitações de performance. Algumas vezes, o esquema de BD criado através do uso dessas regras não atende plenamente os requisitos de performance impostos ao sistema. Neste caso, é necessário buscar uma alternativa de implementação que resulte em um melhor desempenho do sistema. Cabe salientar que estas alternativas somente devem ser aplicadas como último recurso, pois, do ponto de vista do desenvolvimento de programas sobre o banco de dados, elas são sempre piores que as alternativas que haviam sido apresentadas anteriormente.

Nas seções seguintes, apresentamos, inicialmente, um exemplo de situação em que as regras de tradução apresentadas introduzem redundância de dados no banco de dados e, posteriormente alguns exemplos de alternativas de implementação que podem ser adotadas para a melhoria do desempenho.

■ redundância no banco de dados projetado

Conforme mencionado acima, há situações em que a aplicação das regras de tradução apresentadas leva a um banco de dados com redundância de dados. O problema está ligado ao uso de *relacionamentos identificadores*.

Para mostrar um exemplo dessa situação, vamos considerar o modelo ER mostrado na Figura 5.19 que corresponde ao banco de dados de uma empresa que ministra cursos e que mantém informações sobre as avaliações dos cursos. A entidade *CURSO* representa os cursos ministrados, havendo

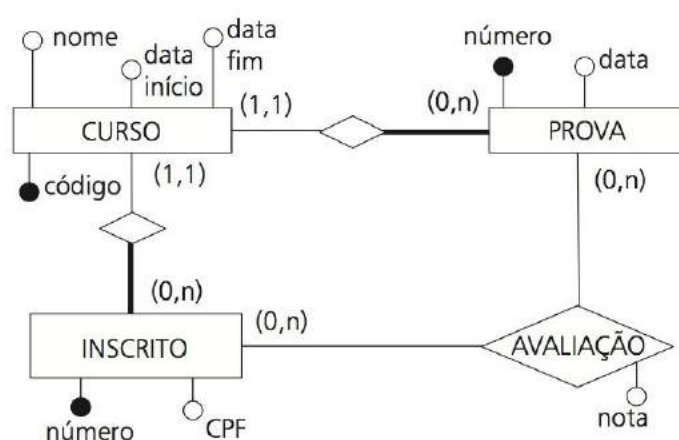


Figura 5.19 Modelo ER com relacionamentos identificadores.

uma ocorrência para cada curso. Um curso é identificado por um código, tem um nome, uma data de início e uma data de fim. A entidade *INSCRITO* representa cada um dos inscritos em um curso. Esta entidade é identificada pelo curso no qual ocorreu a inscrição e por um número seqüencial que distingue as inscrições de um curso. Já a entidade *PROVA* corresponde às provas que são realizadas em um curso, e é identificada pelo curso e por um número seqüencial da prova dentro do curso. Finalmente, o relacionamento *AValiação* armazena a nota que é atribuída a um inscrito em uma prova.

A aplicação das regras de tradução descritas acima leva ao esquema relacional mostrado na Figura 5.20.

Entretanto, o banco de dados da Figura 5.20 apresenta redundância de dados. As colunas *CodCursoProva* e *CodCursoInscrito* contêm exatamente os mesmos valores, já que os inscritos em cursos somente realizam provas do curso em que estão inscritos, e não em outros cursos. O problema é que esta restrição de integridade não está expressa no modelo ER e, portanto, não é considerada pelas regras de tradução para o modelo lógico.

Por esta razão, após aplicar as regras de tradução de ER para relacional apresentadas na seção anterior, é necessário fazer uma revisão do modelo relacional, para eliminar eventuais redundâncias de dados.

No exemplo da Figura 5.20, as colunas *CodCursoProva* e *CodCursoInscrito* devem ser fundidas em uma única coluna, o que resulta no modelo da Figura 5.21.

```

Curso(CodCurso, Nome, DataIni, DataFim)
Prova(CodCurso, NumeroProva, DataProva)
      CodCurso referencia Curso
Inscrito(CodCurso, NumeroInscrito, CPF)
        CodCurso referencia Curso
Avaliacao(CodCursoProva, NumeroProva,
          CodCursoInscrito, NumeroInscrito, Nota)
          (CodCursoProva, NumeroProva) referencia Prova
          (CodCursoInscrito, NumeroInscrito) referencia Inscrito

```

Figura 5.20 Modelo relacional resultante da aplicação das regras de projeto lógico sobre o modelo ER da Figura 5.19.

```

Curso(CodCurso, Nome, DataIni, DataFim)
Prova(CodCurso, NumeroProva, DataProva)
    CodCurso referencia Curso
Inscrito(CodCurso, NumeroInscrito, CPF)
    CodCurso referencia Curso
Avaliacao(CodCurso, NumeroProva, NumeroInscrito, Nota)
    (CodCurso, NumeroProva) referencia Prova
    (CodCurso, NumeroInscrito) referencia Inscrito

```

Figura 5.21 Modelo relacional referente ao modelo ER da Figura 5.19, após a eliminação de redundância.

■ manutenção de informações redundantes no banco de dados

Como vimos na Seção Um modelo deve ser livre de redundância página 87, as informações redundantes (informações que podem ser obtidas a partir de outras existentes no banco de dados) devem ser eliminadas do modelo conceitual. Na seção precedente, vimos também a eliminação de informações redundantes do modelo relacional.

Entretanto, por questões de desempenho das aplicações que acessam o banco de dados, às vezes, informações redundantes são desejáveis no banco de dados. Isto pode ocorrer com atributos redundantes, eventualmente resultantes de computações que envolvam vários acessos ao banco de dados, e que são acessados muito freqüentemente, mas sofrem poucas alterações. Neste caso, considerando o desempenho do sistema como um todo, pode ser mais eficiente manter o atributo redundante armazenado no banco de dados.

A Figura 5.22 apresenta um exemplo, no qual aparece um atributo redundante em um fragmento do modelo de dados de um sistema de reserva de passagens aéreas. Trata-se do atributo *número de reservas*, que pode ser obtido pela contagem de todas as reservas relacionadas ao voo. Do ponto de vista conceitual, o atributo *número de reservas* deveria ser eliminado, por ser redundante. Entretanto, do ponto de vista de desempenho global do sistema, poderia ser importante manter uma coluna com este valor, caso a consulta ao número de reservas fosse freqüente e sua computação demandasse tempo considerável.

■ simulação de atributos multivalorados

Conforme discutimos na Seção Tabela, na página 120, na abordagem relacional, não existem colunas multivaloradas.

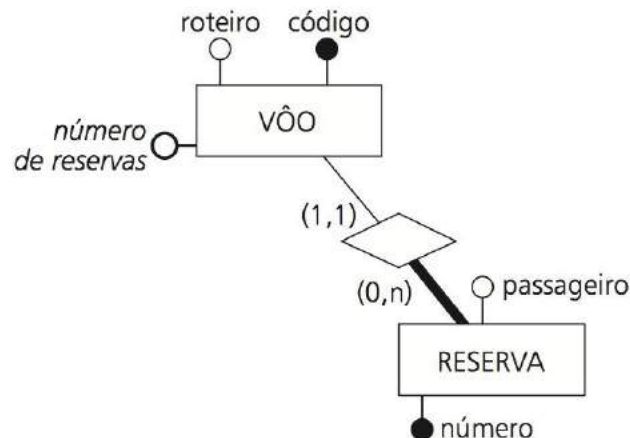


Figura 5.22 Atributo redundante mantido no projeto lógico.

Por esta razão, modelos ER destinados ao projeto de bancos de dados relacionais são, normalmente, construídos sem o uso de atributos multivalorados. A Figura 5.23 apresenta um diagrama ER com atributos multivalorados e o diagrama obtido pela transformação do atributo multivalorado em uma entidade separada.

A implementação do diagrama da Figura 5.23, caso sejam seguidas as regras apresentadas, é a seguinte:

```

Cliente (CodCli, Nome)
Telefone (CodCli, Número)
    CodCli referencia Cliente
  
```

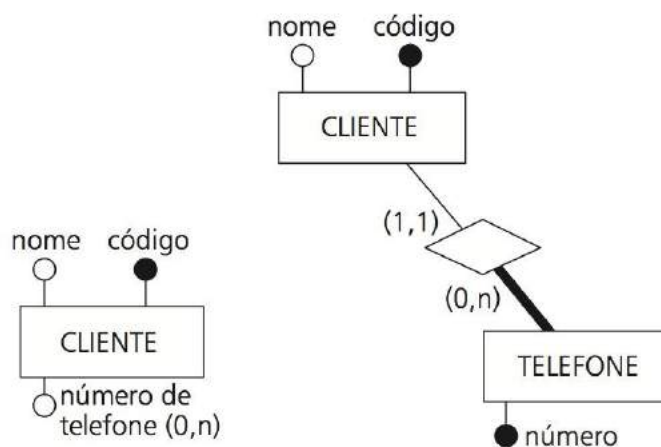


Figura 5.23 Eliminando atributos multivalorados.

Entretanto, esta implementação pode trazer problemas de desempenho, caso seja necessário obter todos os telefones de um cliente com frequência. Esta operação implica em busca de várias linhas de `Telefone`, tantas linhas quantos telefones o cliente possuir.

Dadas certas condições de contorno, é possível conceber uma implementação simplificada, que atenda com mais eficiência às buscas aos telefones de um cliente. Consideremos as seguintes condições de contorno:

- São raros os clientes que possuem mais que dois telefones. Quando isso ocorrer, é suficiente armazenar apenas dois números.
- Não há consultas ao banco de dados usando o número de telefone como critério de seleção. Os números de telefone são apenas exibidos ou impressos junto às demais informações de cliente.

Neste caso, uma implementação “desnormalizada”¹ como a mostrada a seguir pode permitir um maior desempenho das aplicações:

Cliente (CodCli, Nome, NumTel1, NumTel2)

Nesta implementação, optou-se por simular uma coluna multivalorada através da criação de diversas colunas `NumTel` sufixadas por um número para distingui-las. Essa alternativa permite que os telefones de um cliente sejam obtidos mais rapidamente, já que encontram-se todos dentro da mesma linha da tabela. Além disso, implica em menos espaço ocupado, já que o espaço necessário à implementação da chave primária da tabela *Telefone*, na primeira alternativa, é considerável.

Do ponto de vista funcional, o inconveniente desta alternativa é que operações que tratam com os telefones de um cliente como uma coleção ficam mais complexos em SQL. Por exemplo, uma eventual consulta usando o número de telefone como critério de busca torna-se mais complicada, já que devem ser referenciados todos os nomes das colunas referentes ao atributo multivalorado.

■ relacionamentos mutuamente exclusivos

Outro caso que permite uma implementação alternativa à especificada pelas regras de projeto é aquele no qual uma entidade participa de forma

¹ No próximo capítulo veremos o conceito de *normalização*.

mutuamente exclusiva de dois ou mais relacionamentos. Participar de forma mutuamente exclusiva significa que uma ocorrência da entidade que participa de um dos relacionamentos em questão, não participa dos demais relacionamentos. Um exemplo é apresentado na Figura 5.24. Pode-se supor que uma ocorrência da entidade *VENDA* participe de exatamente um dos dois relacionamentos e de somente um deles. Observe que esta informação não está contida no diagrama, já que não há uma convenção para registrá-la no DER².

A implementação para este modelo, caso forem seguidas as regras apresentadas, é a seguinte:

```
PessFis (CPF, Nome)
PessJur (CNPJ, RazSoc)
Venda (No, data, CPF, CNPJ)
    CPF referencia PessFis
    CNPJ referencia PessJur
```

Nesta implementação, as colunas *CPF* e *CNPJ* são especificadas como opcionais, já que, em cada linha, um ou outro campos serão vazios. Aparecem

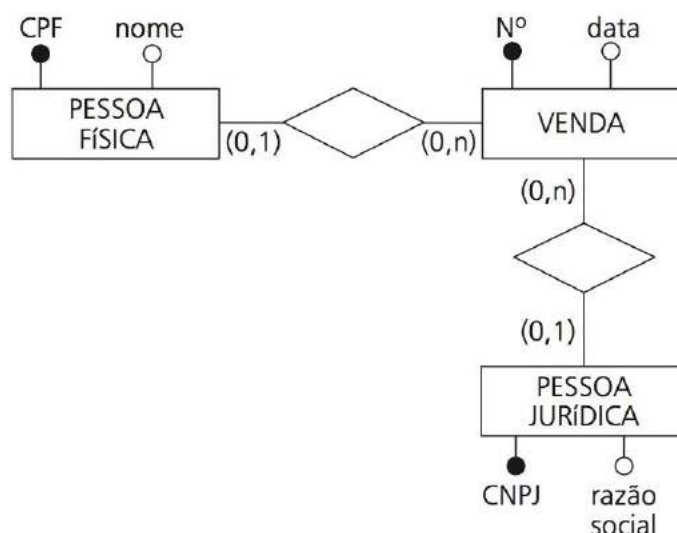


Figura 5.24 Relacionamentos mutuamente exclusivos.

² Algumas convenções de diagramas ER prevêem notações específicas para a mútua exclusão entre relacionamentos.

assim os problemas típicos de colunas opcionais. Uma implementação alternativa é criar uma única coluna na qual aparece o CPF ou o CNPJ do comprador, conforme mostrado abaixo.

```
PessFis (CPF, Nome)
PessJur (CNPJ, RazSoc)
Venda (No, data, CPF/CNPJ, TipoCompr)
```

Além da fusão das colunas CPF e CNPJ, deve ser criada uma coluna TipoCompr, que informa se o campo na coluna CPF/CNPJ é referente a um comprador pessoa física ou a uma pessoa jurídica. Através desta alternativa evita-se o uso de colunas opcionais.

Entretanto, a alternativa apresenta igualmente uma desvantagem pois não é possível especificar ao SGBD que o campo CPF/CNPJ é chave estrangeira, já que ele não referencia uma única tabela, mas duas (PessFis e PessJur), de forma alternativa de acordo com o valor do campo TipoCompr.

Apresentadas as alternativas de projeto acima, cabe reforçar a observação que já foi feita anteriormente: alternativas de implementação que fogem às regras de projeto lógico apresentadas somente devem ser tentadas em último caso, quando outras alternativas para a melhoria do desempenho não solucionarem o problema. Normalmente, bases de dados que não foram projetadas de acordo com as regras implicam em custos mais altos de desenvolvimento e de manutenção das aplicações desenvolvidas sobre o banco de dados.

5.3

→ engenharia reversa de modelos relacionais

De forma geral, um processo de *engenharia reversa* pode ser definido como um processo de abstração, que parte de um modelo de implementação e resulta em um modelo conceitual, que descreve abstratamente a implementação em questão. O termo *engenharia reversa* vem do fato de usar-se, como ponto de partida do processo, um produto implementado (o modelo de implementação) para obter sua especificação (o modelo conceitual).

No caso de banco de dados, fala-se de engenharia reversa quando modelos de dados mais ricos em detalhes de implementação são transformados em modelos de dados mais abstratos.

Um caso específico de engenharia reversa de banco de dados é o da *engenharia reversa de modelos relacionais*. Neste tipo de engenharia reversa, tem-se, como ponto de partida, um modelo lógico de um banco de dados relacional e, como resultado, um modelo conceitual, no caso deste livro, na abordagem ER. Este é o processo inverso ao de projeto lógico (Figura 5.1).

A engenharia reversa de modelos relacionais pode ser útil quando não se tem um modelo conceitual para um banco de dados existente. Isso pode acontecer quando o banco de dados foi desenvolvido de forma empírica, sem o uso de uma metodologia de desenvolvimento, ou quando o esquema do banco de dados sofreu modificações ao longo do tempo, sem que as mesmas tenham sido registradas no modelo conceitual.

Como veremos no capítulo 6, a engenharia reversa de modelos relacionais é um passo dentro de um processo mais amplo de engenharia reversa de arquivos e documentos convencionais.

O processo de engenharia reversa de um modelo relacional consta dos seguintes passos:

- 1** Identificação da construção ER correspondente a cada tabela
- 2** Definição de relacionamentos **1:n** e **1:1**
- 3** Definição de atributos
- 4** Definição de identificadores de entidades e relacionamentos

Estes passos são detalhados nas seções que seguem.

O processo será exemplificado sobre um banco de dados para um sistema acadêmico, cujo esquema é apresentado na Figura 5.25.

5.3.1 identificação da construção ER correspondente a cada tabela

Na primeira etapa da engenharia reversa de um banco de dados relacional, define-se, para cada tabela do modelo relacional, qual a construção que lhe corresponde no modelo ER.

Uma tabela pode corresponder a:

- uma entidade,

```

Disciplina (CodDisc, NomeDisc)
Curso (CodCr, NomeCr)
Curric (CodCr, CodDisc, Obr/Opc)
    CodCr referencia Curso
    CodDisc referencia Disciplina
Sala (CodPr, CodSl, Capacidade)
    CodPr referencia Prédio
Prédio (CodPr, Endereço)
Turma (Anosem, CodDisc, SiglaTur, Capacidade, CodPr, CodSl)
    CodDisc referencia Disciplina
    (CodPr, CodSl) referencia Sala
Laboratório (CodPr, CodSl, Equipam)
    (CodPr, CodSl) referencia Sala

```

Figura 5.25 Esquema de banco de dados acadêmico para engenharia reversa.

- um relacionamento **n:n**,
- uma entidade especializada.

O fator que determina qual a construção ER que corresponde a uma tabela é a composição de sua chave primária. Tabelas podem ser classificadas em três tipos de acordo com sua chave primária:

- **Regra 1:** Chave primária composta por mais de uma chave estrangeira – A tabela que possui uma chave primária composta de múltiplas chaves estrangeiras implementa um *relacionamento n:n* entre as entidades correspondentes às tabelas referenciadas pelas chaves estrangeiras. Um exemplo de tabela deste tipo é *Curric*, que tem como chave primária *CodCr* e *CodDisc*. Ambas as colunas são chave estrangeira em relação às tabelas *Curso* e *Disciplina* respectivamente. Portanto, a tabela *Curric* representa um relacionamento entre as entidades correspondentes às tabelas *CodCr* e *CodDisc*. No exemplo, a única tabela deste tipo é a tabela *Curric*.
- **Regra 2:** A chave primária completa forma uma chave estrangeira – Quando toda chave primária (todas as suas colunas) compõe uma única chave estrangeira, a tabela representa uma entidade que forma uma *especialização* da entidade correspondente à tabela referenciada pela chave estrangeira. Um exemplo de tabela deste tipo é a tabela *Laboratório*, que possui como chave primária as colunas *CodPr* e *CodSl*, as quais são chave estrangeira da tabela de salas. A restrição de integridade referencial em questão especifica que uma linha na tabela de laboratórios somente existe quando uma linha com a mesma chave existir na tabela de salas. No

modelo ER, isso significa que uma ocorrência da entidade laboratório somente pode existir quando a ocorrência correspondente da entidade sala existir, ou seja, significa que a entidade laboratório é uma especialização de sala. No exemplo, a única tabela deste tipo é a tabela Laboratório.

- **Regra 3: Demais casos** – Quando a chave primária da tabela não for composta de múltiplas chaves primárias (Regra 1), nem for toda ela uma chave estrangeira (Regra 2), a tabela representa uma *entidade*. Exemplificando, a tabela Curso, cuja chave primária, a coluna CodCr não contém chaves estrangeiras, representa uma entidade. Da mesma forma, a tabela Sala também representa uma entidade. Sua chave primária (colunas CodPr e CodSl) contém apenas uma chave estrangeira (coluna CodSl). Assim, não obedece ao requisito da multiplicidade de chaves estrangeiras (Regra 1), nem ao requisito de toda chave primária ser estrangeira (Regra 2) e enquadra-se na presente regra. O mesmo é válido para as tabelas Disciplina, Prédio e Turma.

Tendo feito a classificação de tabelas segundo a composição da chave primária e com isso identificado as construções de ER correspondentes a cada tabela, é possível montar um diagrama ER inicial, conforme mostra a Figura 5.26.

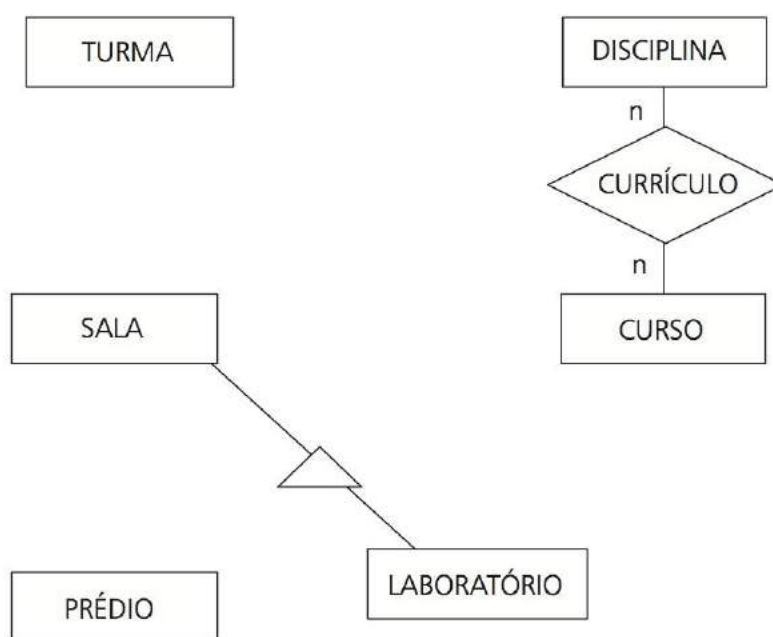


Figura 5.26 Diagrama ER inicial para o BD acadêmico.

5.3.2 definição de relacionamentos 1:n ou 1:1

Toda chave estrangeira que não se enquadra nas regras 1 e 2 apresentadas na seção anterior representa um relacionamento **1:n** ou **1:1**. Em outros termos, toda chave estrangeira que não corresponde a um relacionamento **n:n** (Regra 1), nem a uma entidade especializada (Regra 2) representa um relacionamento **1:n** ou **1:1**.

Assim, nesta regra, estão sendo tratadas chaves estrangeiras que:

- contêm colunas que não fazem parte de chaves primárias, ou
- fazem parte de uma chave primária, e esta chave primária não contém outras chaves estrangeiras e contém colunas que não são chaves estrangeiras.

A regra nem sempre permite definir se a cardinalidade do relacionamento é **1:n** ou **1:1**. Quando a chave estrangeira é parte de uma chave primária, trata-se de um relacionamento **1:n**. Entretanto, nos demais casos, para definir a cardinalidade, é necessário verificar os possíveis conteúdos do banco de dados.

No exemplo, as chaves estrangeiras que representam relacionamentos **1:n** ou **1:1** são as seguintes:

Tabela Sala:

CodPr referencia Prédio

Tabela Turma:

CodDisc referencia Disciplina
(CodPr,CodSl) referencia Sala

Com este passo, podemos completar a definição dos relacionamentos no diagrama ER, conforme mostra a Figura 5.27. No exemplo, todos os relacionamentos referentes às chaves estrangeiras acima são do tipo **1:n**.

5.3.3 definição de atributos

Neste passo, para cada coluna que não seja chave estrangeira, é definido um atributo na entidade/relacionamento correspondente à tabela que contém a coluna. Observe-se que colunas de chaves estrangeiras não correspondem a atributos no diagrama ER, mas sim a relacionamentos, e por isso já foram

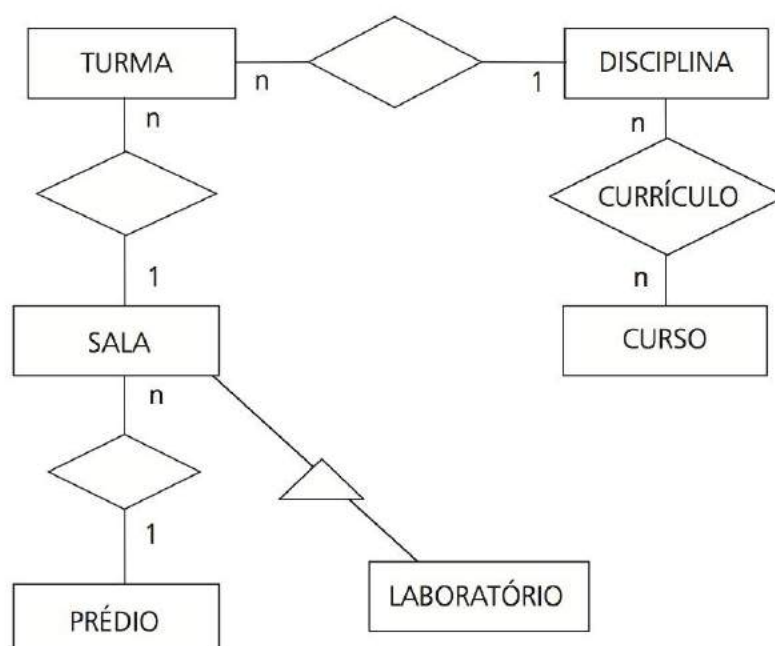


Figura 5.27 Definição dos relacionamentos 1:n e 1:1.

tratadas nas etapas anteriores. Para o exemplo, a execução deste passo da engenharia reversa resulta no diagrama ER da Figura 5.28.

5.3.4 definição de identificadores de entidades

No último passo da engenharia reversa, são definidos os identificadores das entidades e dos relacionamentos. A regra para a definição dos identificadores é a seguinte:

- Coluna da chave primária que não é chave estrangeira – Toda coluna que faz parte da chave primária e que *não* é chave estrangeira corresponde a um *atributo identificador* da entidade ou relacionamento.
- Coluna da chave primária que é chave estrangeira – Toda coluna que faz parte da chave primária e que é chave estrangeira corresponde a um identificador externo da entidade. Exemplificando, a coluna *CodDisc*, que é parte da chave primária da tabela *Turma* é também chave estrangeira em relação à tabela *Disciplina*. Portanto, a entidade *TURMA* é identificada também pelo relacionamento com *DISCIPLINA*.

Executando este passo da engenharia reversa sobre o modelo do exemplo chegamos à Figura 5.29.

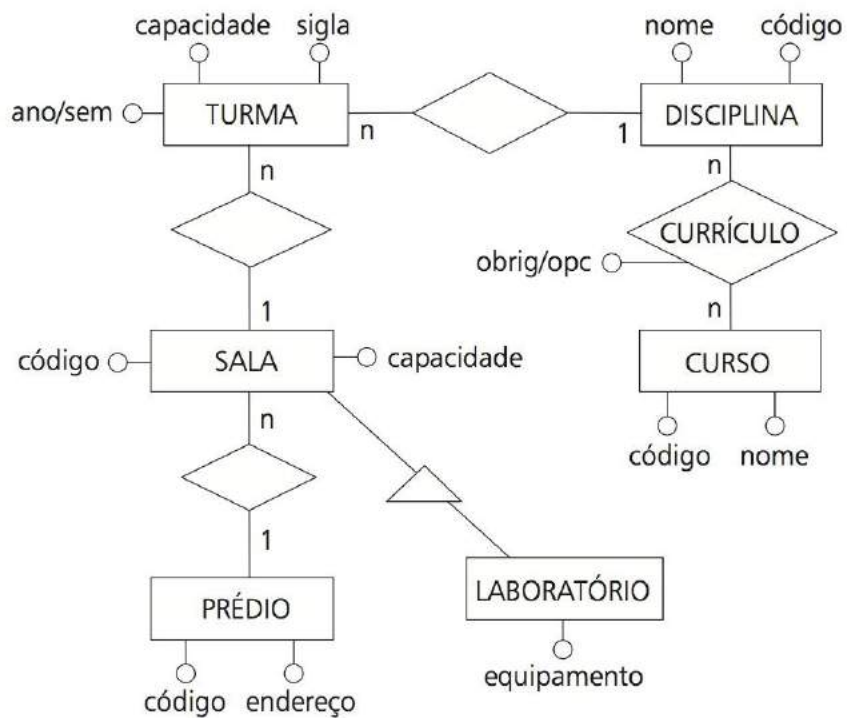


Figura 5.28 Definição dos atributos.

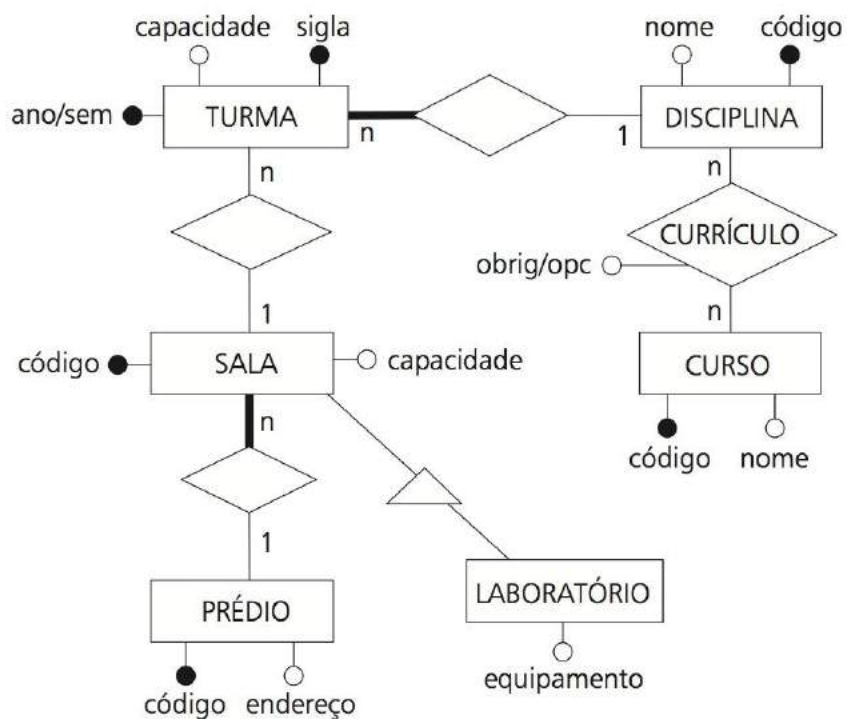


Figura 5.29 Definição dos identificadores de entidades.

5.4 → exercícios

exercício 1 Considere as seguintes alternativas de implementação de um banco de dados relacional:

Alternativa 1:

Aluno (CodAl, Nome, CodCurso, Endereco)

Alternativa 2:

Aluno (CodAl, Nome, CodCurso)

EnderecoAluno (CodAl, Endereco)

CodAl referencia Aluno

Em ambos os casos está sendo representado um conjunto de alunos e informações (código, nome, código de curso, endereço) a ele referentes. À luz dos princípios que baseiam as regras de tradução de diagramas ER para modelo relacional, discuta qual das duas alternativas é preferível.

exercício 2 Usando as regras de transformação de modelos ER para modelo lógico relacional apresentadas neste capítulo, projete um BD relacional para o modelo ER da Figura 5.30. Para não sobrecarregar o diagrama os atributos das entidades são listados a seguir. Os atributos identificadores estão sublinhados.

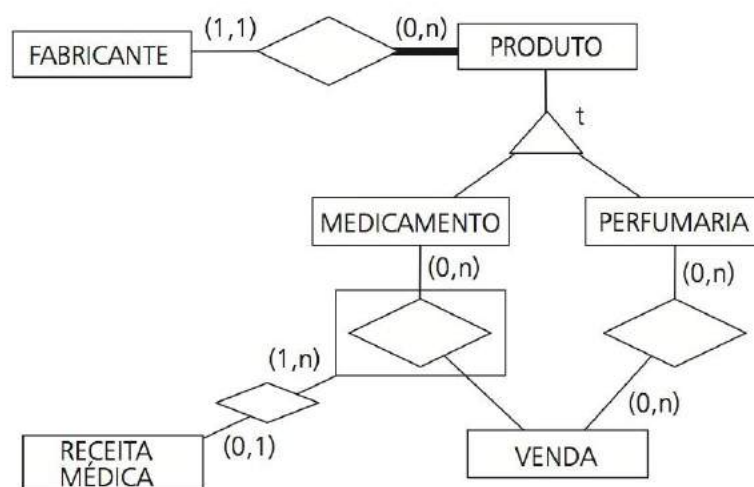


Figura 5.30 Modelo ER para uma farmácia.

Produto(Número, NomeComercial, TipoEmbalagem,
 Quantidade, PreçoUnitário)
 Fabricante(CNPJ, Nome, Endereço)
 Medicamento(Tarja, Fórmula)
 Perfumaria(Tipo)
 Venda(Data, NúmeroNota, NomeCliente, CidadeCliente)
 PerfumariaVenda(Quantidade, Imposto)
 MedicamentoReceitaVenda(Quantidade, Imposto)
 ReceitaMédica(CRM, Número, Data)

exercício 3 Usando as regras de transformação de modelos ER para modelo lógico relacional apresentadas neste capítulo, projete um BD relacional para o modelo ER da Figura 5.31. Para não sobrecarregar o diagrama os atributos das entidades são listados a seguir. Os atributos identificadores estão sublinhados.

Escritório(Número, Local)
 Cliente(NúmeroCartMotorista, EstadoCartMotorista,
 Nome, Endereço, Telefone)
 Contrato aluguel(Número, Data, Duração)
 Veículo(Número, DataPróximaManutenção, Placa)
 Tipo de Veículo(Código, Nome, ArCondicionado)
 Automóvel(NúmeroPortas, DireçãoHidráulica,
 CâmbioAutomático, Rádio)
 Ônibus(NúmeroPassageiros, Leito, Sanitário)

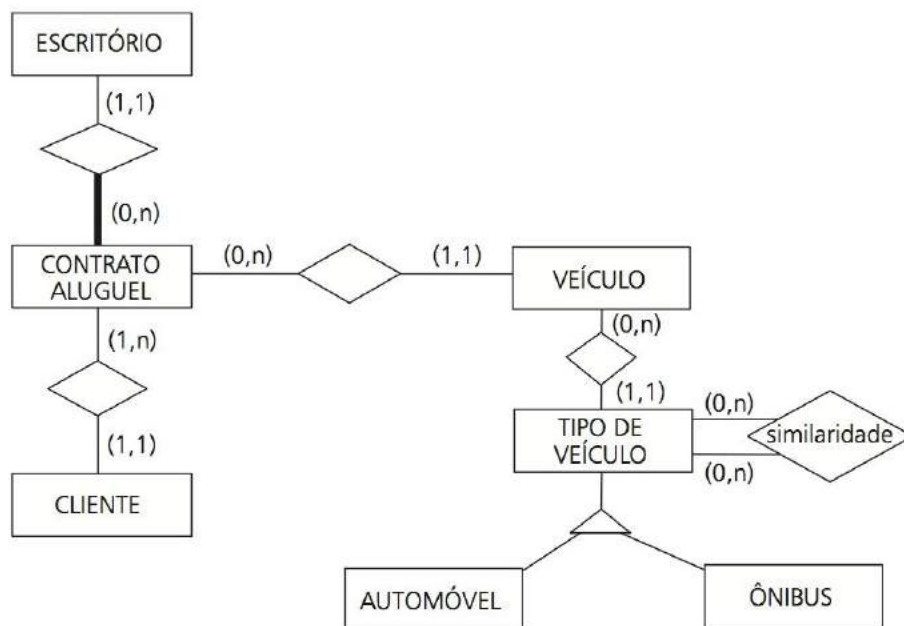


Figura 5.31 Modelo ER para locadora de veículos.

exercício 4 Abaixo é apresentado um esquema lógico de um BD relacional que armazena dados sobre produtos e vendas em uma loja. Usando as regras de engenharia reversa apresentadas anteriormente, construa um diagrama ER para este BD.

```
Produto (CodigoTipoProd, NumeroProd, DescricaoProd, PreçoProd)
        CodigoTipoProd referencia TipoProd
```

/* tabela de produtos de uma loja – CodigoTipoProd é o código do tipo do produto, NumeroProd é seu código, DescriçãoProd é uma descrição do produto e PreçoProd é seu preço */

```
Similaridade (CodigoTipoProd, NumeroProd,
               CodigoTipoProdSim, NumeroProdSim
               (CodigoTipoProd, NumeroProd) referencia Produto
               (CodigoTipoProdsim, NumeroProdSim) referencia Produto
```

/* tabela de similaridade de produtos – para cada produto, informa quais são seus produtos similares*/

```
TipoProd (CodigoTipoProd, DescricaoTipoProd)
```

/* tabela de tipos de produtos, com código e descrição */

```
Venda (NúmeroNF, DataVenda, CodReg, CodEmp)
        (CodigoReg) referencia Registradora
        (CodEmp) referencia Empregado
```

/* tabela que informa as vendas que ocorreram na loja – informa o número da nota fiscal, a data da venda, a registradora na qual ocorreu, bem como o empregado que a realizou */

```
ItemVenda (NúmeroNF, CodigoTipoProd, NumeroProd,
            QtdeItem, PreçoItem)
        (NúmeroNF) referencia Venda
        (CodigoTipoProd, NumeroProd) referencia Produto
```

/* tabela com informações dos itens de uma venda, isto é, que produtos e em que quantidade e com que preço foram vendidos em uma venda */

```
Registradora(CodReg, SaldoReg)
```

```
/* tabela com código e saldo de cada registradora na loja */
```

```
Empregado(CodEmp, NomeEmp, SenhaEmp)
```

```
/* tabela com código, nome e senha de cada empregado da loja */
```

exercício 5 Abaixo é apresentado um esquema lógico de um BD relacional que armazena dados genealógicos. Usando as regras de engenharia reversa apresentadas anteriormente, construa um diagrama ER para este BD.

```
Pessoa(PessID, PessNome, NascLocID, DataNasc,
        FalecLocID, DataFalec, ProfID, FilhoCasamID, Sexo)
    NascLocID referencia Local
    FalecLocID referencia Local
    ProfID referencia Profiss
    FilhoCasamID referencia Casam
```

```
/* Tabela de pessoas: contém o identificador da pessoa, seu nome, local (iden-
tificador) e data de nascimento, local (identificador) e data de falecimento,
profissão, identificador do casamento que gerou a pessoa e sexo */
```

```
Local(LocID, Cidade, País)
```

```
/* Tabela de locais */
```

```
Profiss(ProfID, ProfNome)
```

```
/* Tabela de profissões */
```

```
Casam(CasamID, MaridoPessID, EsposaPessID,
        DataCasam, CasamLocID)
    MaridoPessID referencia Pessoa
    EsposaPessID referencia Pessoa
    CasamLocID referencia Local
```

```
/* Tabela de casamentos: contém identificador do casamento, identificador do
marido, identificador da esposa, data do casamento e local (identificador) */
```

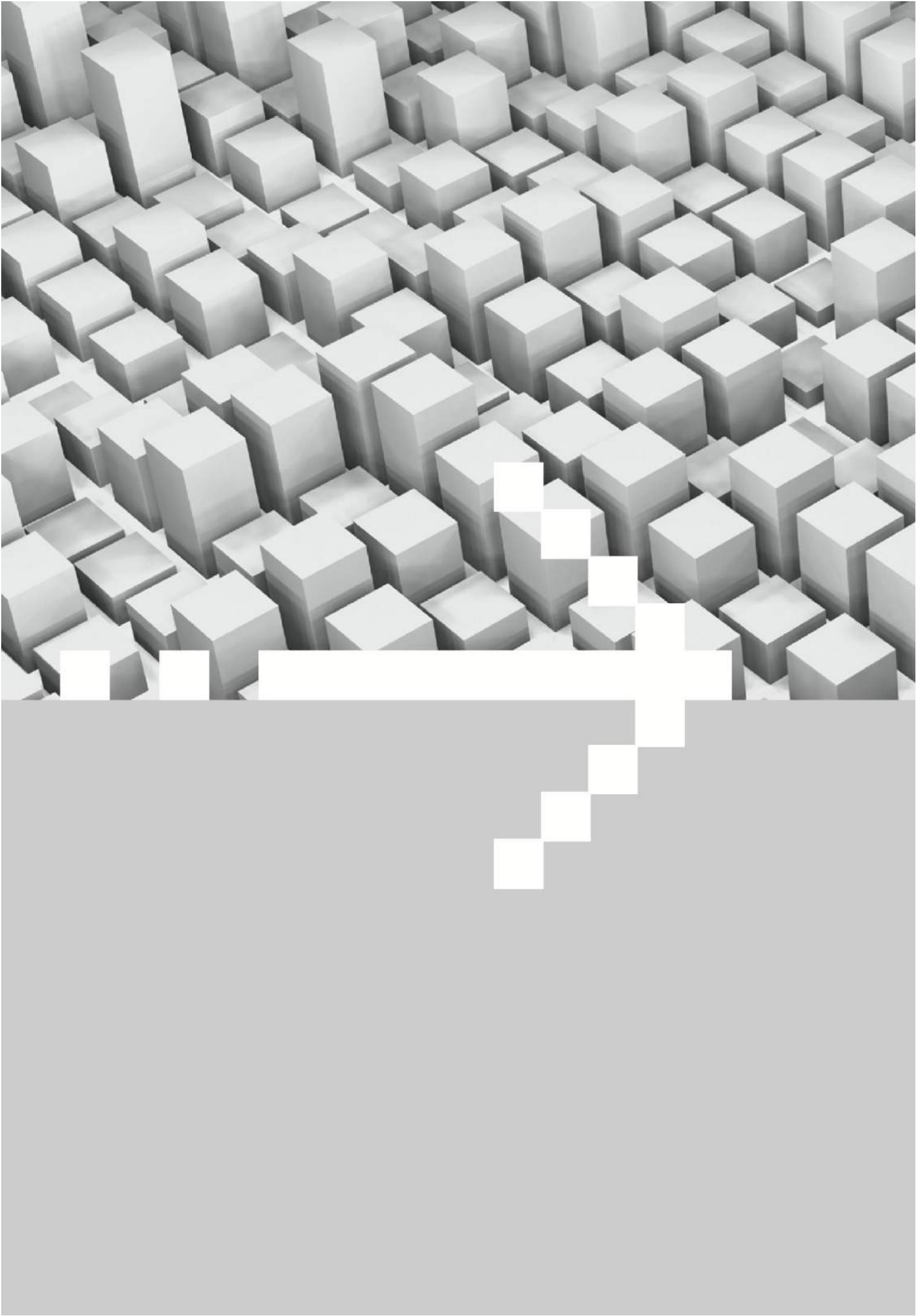
5.5

→ leituras recomendadas

Muitos dos livros que tratam da abordagem relacional dedicam um capítulo ao projeto de banco de dados relacional a partir de modelos ER. Este é o caso dos livros ELMASRI; NAVATHE (2002) e SILBERSCHATZ; KORTH; SUDARSHAN (1999) que já havíamos citado no capítulo referente a abordagem relacional.

Da mesma forma, livros sobre modelagem e projeto de banco de dados (BATINI; CERI; NAVATHE (1992); SETZER (1987); TEOREY (1994)) abordam o problema do projeto lógico. O livro de BATINI; CERI; NAVATHE (1992) contém uma cobertura completa, tanto do projeto do banco de dados, quanto da engenharia reversa a partir de modelos relacionais.

Os artigos DUMPALA; ARORA (1983), TEOREY; YANG; FRY: (1986) e WONG; KATZ (1979) são os pioneiros no projeto lógico de banco de dados relacionais a partir de modelos ER. Já os artigos JOHANNESSON; KALMAN (1989) e NAVATHE; AWONG (1987) apresentam as idéias básicas sobre como deve ser feita a engenharia reversa de modelos relacionais para modelos ER.





capítulo

6

engenharia reversa de arquivos e normalização

- ■ No capítulo 5, foi apresentado um processo de engenharia reversa de modelos relacionais, que permite obter um modelo conceitual para um modelo lógico relacional.

Neste capítulo, veremos a engenharia reversa de arquivos, que possibilita a obtenção de um modelo lógico relacional a partir do modelo lógico de um banco de dados não-relacional. Ele permite a engenharia reversa de qualquer conjunto de dados para os quais se disponha de uma descrição, como documentos, arquivos manuais, arquivos convencionais em computador ou bancos de dados gerenciados por SGBD não-relacional. Como base teórica para este processo será usado o conceito de normalização, uma técnica que elimina a redundância de dados de arquivos.

6.1

→ introdução

Muitos dos sistemas de informação hoje usados foram desenvolvidos ao longo dos últimos 20 anos e não utilizam bancos de dados relacionais, sendo chamados de *sistemas legados* (*legacy systems*). Os dados desses sistemas estão armazenados em arquivos de linguagens de terceira geração, como COBOL ou Basic, ou então em bancos de dados da era pré-relacional, como IMS ou ADABAS. Raramente, os arquivos destes sistemas estão documentados através de modelos conceituais.

Adicionalmente, há bancos de dados relacionais que não possuem documentação na forma de um modelo conceitual.

No entanto, há situações no ciclo de vida de um sistema nas quais um modelo conceitual é de grande valia.

Um exemplo é a manutenção rotineira do *software* de um sistema de informações. Neste caso, o modelo conceitual pode ser usado como documentação abstrata dos dados, durante discussões entre usuários, analistas e programadores. A existência de um modelo conceitual permite que pessoas que não conheçam o sistema possam aprender mais rapidamente o seu funcionamento.

Outro exemplo no qual é importante possuir o modelo conceitual de um banco de dados já implementado é o da migração do banco de dados para uma nova plataforma de implementação, por exemplo, usando um SGBD relacional. A disponibilidade de uma documentação abstrata, na forma de um modelo conceitual dos dados do sistema existente, pode acelerar o processo de migração, pois permite encurtar a etapa de modelagem de dados do novo banco de dados.

O presente capítulo mostra como executar o processo de engenharia reversa de arquivos convencionais ou de banco de dados pré-relacionais. De forma geral, o processo apresentado serve para a construção de um modelo entidade-relacionamento dentro da estratégia *ascendente*, estratégia de modelagem recomendada para o caso em que se parte de descrições de dados existentes (Seção Partindo de descrições de dados existentes, na página 102).

6.2

→ visão geral do processo de engenharia reversa

A Figura 6.1 apresenta uma visão geral do processo de engenharia reversa de arquivos convencionais. O processo parte das descrições dos arquivos que compõem o sistema existente. Portanto, é um processo que segue a estratégia ascendente de construção de modelos ER referida na Seção Partindo de descrições de dados existentes (página 102).

O primeiro passo é a representação da descrição de cada arquivo existente na forma de um esquema de uma tabela relacional não-normalizada. Este

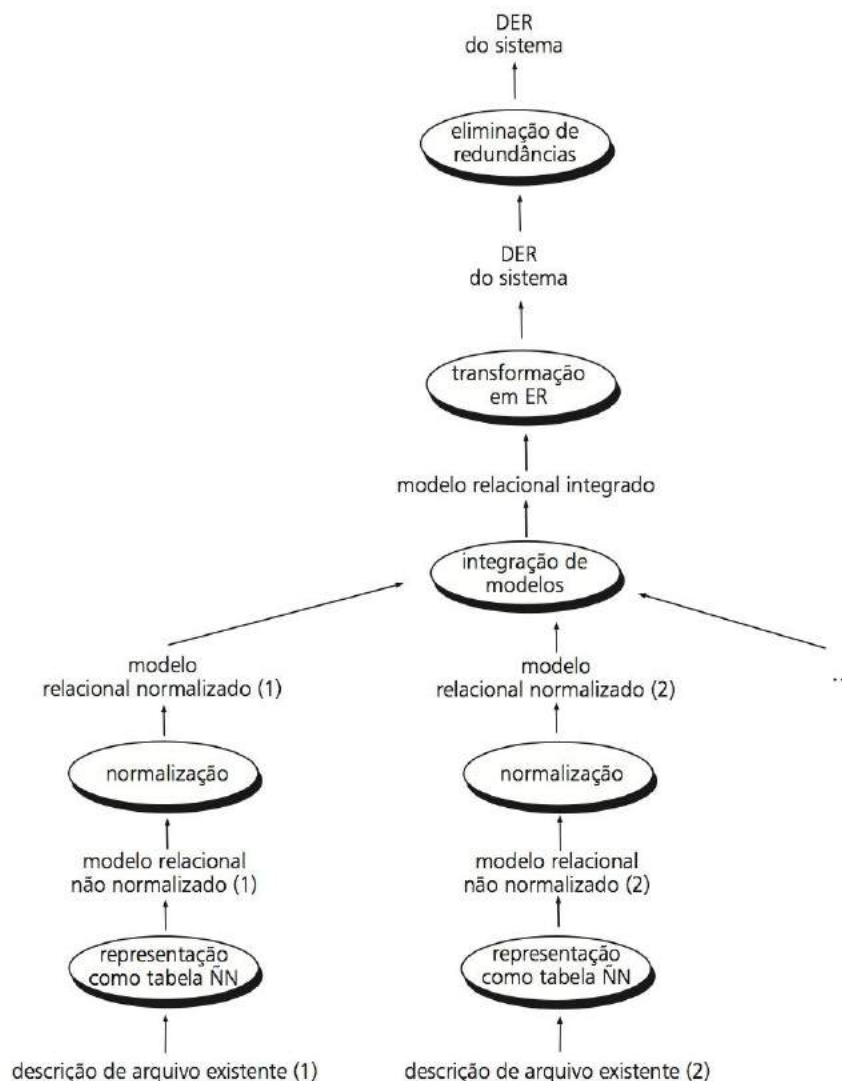


Figura 6.1 Visão geral do processo de engenharia reversa.

primeiro passo obtém uma descrição independente do tipo de arquivo que está sendo utilizado. A partir daí, o processo trabalha apenas com tabelas relacionais, o que o torna independente do tipo de arquivo que está sendo usado como entrada ao processo de normalização.

A seguir, este esquema de tabela não-normalizada passa por um processo conhecido por *normalização*, através do qual é obtido um modelo relacional, contendo as descrições das tabelas correspondentes ao arquivo em questão. O objetivo do processo de normalização é:

- reagrupar informações de forma a eliminar redundâncias de dados que possam existir nos arquivos, e
- reagrupar informações de uma forma que permita a obtenção de um modelo ER.

Uma vez normalizados todos os arquivos do sistema, os diferentes esquemas relacionais obtidos pela normalização são integrados, gerando o esquema relacional do banco de dados do sistema. Nesta etapa, as informações comuns a diferentes arquivos são identificadas e representadas uma única vez.

Finalmente, a partir do esquema relacional obtido, usando as regras para a engenharia reversa de um modelo relacional mostradas no capítulo anterior, temos o modelo ER do sistema existente.

6.3

→ documento exemplo

O processo de normalização pode ser executado sobre qualquer tipo de representação de dados. Pode partir da descrição de um arquivo em computador, do *lay-out* de um relatório ou de uma tela, etc.

Para exemplificar o processo de normalização descrito a seguir, usamos um documento (Figura 6.2), que poderia fazer parte de um sistema de gerência de projetos. Para cada projeto, são informados o código, a descrição e o tipo do projeto, bem como os empregados que atuam no projeto. Já para cada empregado do projeto, são informados o seu número, nome e categoria funcional, bem como a data em que o empregado foi alocado ao projeto e o tempo pelo qual o empregado foi alocado ao projeto. A Figura 6.2 apresenta um exemplo de um possível conteúdo deste documento.

RELATÓRIO DE ALOCAÇÃO A PROJETO					
CÓDIGO DO PROJETO: LSC001			TIPO: Novo Desenv.		
DESCRIÇÃO: Sistema de Estoque					
CÓDIGO DO EMPREGADO	NOME	CATEGORIA FUNCIONAL	SALÁRIO	DATA DE INÍCIO NO PROJETO	TEMPO ALOCADO AO PROJETO
2146	João	A1	4	1/11/91	24
3145	Sílvia	A2	4	2/10/91	24
6126	José	B1	9	3/10/92	18
1214	Carlos	A2	4	4/10/92	18
8191	Mário	A1	4	1/11/92	12
CÓDIGO DO PROJETO: PAG02			TIPO: Manutenção		
DESCRIÇÃO: Sistema de RH					
CÓDIGO DO EMPREGADO	NOME	CATEGORIA FUNCIONAL	SALÁRIO	DATA DE INÍCIO NO PROJETO	TEMPO ALOCADO AO PROJETO
8191	Mário	A1	4	1/05/93	12
4112	João	A2	4	4/01/91	24
6126	José	B1	9	1/11/92	12

Figura 6.2 Exemplo de documento a ser normalizado.

6.4

→ representação na forma de tabela não-normalizada

O primeiro passo do processo de engenharia reversa consta em transformar a descrição do documento ou arquivo a ser normalizado em um esquema de uma tabela relacional. A Figura 6.3 apresenta a tabela correspondente ao documento da Figura 6.2. Essa tabela é dita *não-normalizada* (ou mais precisamente *não-primeira-forma-normal*) pois possui uma *tabela aninhada*. Usaremos a abreviatura *NN* para identificar esta forma de representar a tabela. Uma *tabela aninhada* (também chamada por outros autores de *grupo repetido* ou *coluna multivalorada* ou ainda *coluna não atômica*) é uma tabela que aparece como valor de campo dentro de outra tabela.

No caso da Figura 6.3, na coluna *Emp*, aparece, para cada linha de departamento, uma tabela aninhada, que contém os dados dos empregados referentes ao departamento em questão. O conceito de tabela aninhada é o correspondente, na abordagem relacional, aos conceitos de vetor ou *array* de

CódProj	Tipo	Descr	Emp					
			CodEmp	Nome	Cat	Sal	DataIni	TempAl
LSC001	Novo Desenv.	Sistema de Estoque	2146	João	A1	4	1/11/91	24
			3145	Sílvio	A2	4	2/10/91	24
			6126	José	B1	9	3/10/92	18
			1214	Carlos	A2	4	4/10/92	18
			8191	Mário	A1	4	1/11/92	12
PAG02	Manutenção	Sistema de RH	8191	Mário	A1	4	1/05/93	12
			4112	João	A2	4	4/01/91	24
			6126	José	B1	9	1/11/92	12

Figura 6.3 Documento exemplo na forma de tabela não-normalizada.

linguagens, como Java e Pascal, ou de item de grupo repetido (especificado com cláusula OCCURS) da linguagem COBOL.

A tabela da Figura 6.3 pode ser descrita pelo seguinte esquema de tabela relacional:

```
Proj (CodProj, Tipo, Descr,
      (CodEmp, Nome, Cat, Sal, DataIni, TempAl))
```

No esquema acima, foi usada uma extensão da notação de esquema relacional apresentada no capítulo 5. A extensão consta do uso de parênteses aninhados para descrever tabelas aninhadas. No caso de tabelas aninhadas, as colunas sublinhadas indicam a coluna ou grupo de colunas que servem para distinguir diferentes linhas da tabela aninhada referente a uma linha da tabela de seu nível externo. Assim, a coluna CodProj distingue as linhas (cada uma referente a um projeto) da tabela *Proj*. Já a coluna CodEmp serve para distinguir diferentes linhas de empregado dentro do grupo referente a um projeto.

Para apresentar outro exemplo de representação não-normalizada de documentos, mostramos, nas Figuras 6.4 e 6.5, a definição de um arquivo convencional que contém dados de alunos usando as notações de Pascal e de COBOL, respectivamente.

O arquivo contém dados referentes a alunos de uma universidade. Cada registro contém informações referentes a um aluno. O registro do aluno contém seu código, seu nome, uma lista de ingressos em cursos e uma lista de disciplinas cursadas. A lista de ingressos em curso contém o código do curso em que o

```

type reg_aluno = record
  cod_al: integer;
  nome_al: char_60;
  ingressos_cursos_al: array [1..10] of record
    cod_curso: integer;
    semestre_ingresso: integer
  end;
  disciplinas_cursadas_al: array [0..200] of record
    cod_disc: integer;
    semestres_cursados: array [1..20] of record
      semestre_disc: integer;
      nota_disc: integer
    end
  end
end;
end;
arq_aluno = file of reg_aluno;

```

Figura 6.4 Registro de aluno (Pascal).

```

FD   Arq-Alunos 01   Reg-Al.
03   Cod-Al
03   Nome-Al
03   Ingr-Cursos-al OCCURS 1 TO 10
      05   Cod-Curso
      05   Sem-ingresso
03   Disc-Curs-Al OCCURS 0 to 200
      05   Cod-Disc
      05   Sem-Cursado OCCURS 1 TO 20
          07   Sem-Disc-Cursada
          07   Nota-Disc

```

Figura 6.5 Registro de aluno (COBOL).

aluno ingressou, junto com o semestre de ingresso no curso. A lista de disciplinas cursadas contém uma entrada para cada disciplina que o aluno cursou. Para cada disciplina cursada, é informado seu código, junto com os diversos semestres em que o aluno cursou a disciplina. Para cada semestre, é informado o semestre e a nota que o aluno obteve na disciplina no semestre em questão.

Para o arquivo em questão, a representação não-normalizada seria a seguinte:

```

Arq-Alunos (Cod-Al, Nome-Al,
            (Cod-Curso, Sem-ingresso)
            (Cod-Disc,
              (Sem-Disc-Cursada, Nota-Disc)))

```

Observe que as descrições de arquivos não fornecem as colunas que são chave primária, já que este conceito não está presente nas linguagens referidas. Cabe observar ainda que, na representação não-normalizada, não são associados nomes às tabelas aninhadas, já que estes não são necessários para o processo de engenharia reversa.

6.5

→ normalização

Obtido o esquema relacional correspondente ao documento, passa-se ao processo de *normalização*. Este processo baseia-se no conceito de *forma normal*. Uma forma normal é uma regra que deve ser obedecida por uma tabela para que esta seja considerada “bem projetada”. Há diversas formas normais, isto é, diversas regras, cada vez mais rígidas, para verificar tabelas relacionais. Neste trabalho, vamos considerar quatro formas normais, denominadas simplesmente primeira, segunda, terceira e quarta forma normal, abreviadamente 1FN, 2FN, 3FN e 4FN.

6.5.1 passagem à primeira forma normal (1FN)

primeira forma normal (1FN)
=
diz-se que uma tabela está na primeira forma normal quando ela
não contém tabelas aninhadas

O primeiro passo da normalização é a transformação do esquema de tabela não-normalizada em um esquema relacional na primeira forma normal. Uma tabela encontra-se na 1FN quando ela não contém tabelas aninhadas. Portanto, a passagem à 1FN consta da eliminação das tabelas aninhadas eventualmente existentes.

Para transformar um esquema de tabela não-normalizada em um esquema na 1FN há duas alternativas:

- Construir uma única tabela com redundância de dados – Cria-se uma tabela na qual os dados das linhas externas à tabela aninhada são repetidos para cada linha da tabela aninhada. No caso da tabela da Figura 6.3, o esquema resultante seria o seguinte:

```
ProjEmp(CodProj, Tipo, Descr, CodEmp, Nome, Cat, Sal, DataIni, TempAl)
```

Nesta tabela, os dados do projeto aparecem repetidos para cada linha da tabela de empregados.

- Construir uma tabela para cada tabela aninhada – Cria-se uma tabela referente a própria tabela que está sendo normalizada e uma tabela para cada tabela aninhada. No caso da tabela da Figura 6.3, o esquema resultante seria o seguinte:

```
Proj      (CodProj, Tipo, Descr)
ProjEmp   (CodProj, CodEmp, Nome, Cat, Sal, DataIni, TempAl)
```

Considerando apenas a correção do processo de normalização, a primeira alternativa (tabela única) é a preferida. Ao decompor uma tabela em várias tabelas, como ocorre na segunda alternativa, podem ser perdidas relações entre informações.

Entretanto, para fins práticos, preferimos a segunda alternativa (*decomposição de tabelas*), mesmo sabendo que ela pode levar a modelos imperfeitos. No caso de uma tabela não-normalizada, como a do exemplo, que possui apenas uma tabela aninhada, fica fácil visualizar a tabela na 1FN, na primeira alternativa. Entretanto, quando houver diversas tabelas aninhadas, eventualmente com diversos níveis de aninhamento, fica difícil visualizar a tabela na 1FN. Para verificar o tipo de problema que pode ser causado pela decomposição em várias tabelas, o leitor pode estudar o Exercício 17 deste capítulo.

A passagem à primeira forma normal por decomposição de tabelas é feita nos seguintes passos:

- 1 É criada uma tabela na 1FN referente à tabela não-normalizada e que contém apenas as colunas com valores atômicos, isto é, sem as tabelas aninhadas. A chave primária da tabela na 1FN é idêntica à chave da tabela não-normalizada.
- 2 Para cada tabela aninhada, é criada uma tabela na 1FN composta pelas seguintes colunas:
 - a chave primária de cada uma das tabelas nas quais a tabela em questão está aninhada,
 - as colunas da própria tabela aninhada.

- 3** São definidas as chaves primárias das tabelas na 1FN que correspondem a tabelas aninhadas.

Conforme mencionado, na tabela do exemplo, a decomposição gera duas tabelas com o seguinte esquema:

1FN

Proj (CódProj, Tipo, Descr)

ProjEmp (CódProj, CodEmp, Nome, Cat, Sal, DataIni, TempAl)

O conteúdo destas tabelas, caso considerarmos o conteúdo do documento original que está sendo normalizado, é apresentado na Figura 6.6.

Observe que a tabela ProjEmp que refere-se à tabela aninhada contida na tabela não-normalizada de partida. Ela contém as seguintes colunas:

- Coluna CódProj, que é a chave primária da tabela não-normalizada externa à tabela aninhada em questão.
- Colunas da tabela aninhada em questão.

Já a chave primária da tabela ProjEmp é composta pelas colunas CódProj e CodEmp. Isto ocorre pelo fato de o mesmo empregado poder trabalhar em múltiplos projetos. Assim, na tabela ProjEmp aparecem múltiplas linhas para

Proj:

CódProj	Tipo	Descr
LSC001	Novo Desenv.	Sistema
PAG02	Manutenção	Sistema de RH

ProjEmp:

CódProj	CodEmp	Nome	Cat	Sal	DataIni	TempAl
LSC001	2146	João	A1	4	1/11/91	24
LSC001	3145	Sílvia	A2	4	2/10/91	24
LSC001	6126	José	B1	9	3/10/92	18
LSC001	1214	Carlos	A2	4	4/10/92	18
LSC001	8191	Mário	A1	4	1/11/92	12
PAG02	8191	Mário	A1	4	1/05/93	12
PAG02	4112	João	A2	4	4/01/91	24
PAG02	6126	José	B1	9	1/11/92	12

Figura 6.6 Tabelas referentes ao exemplo na 1FN.

um mesmo empregado e é necessário usar o código do projeto para distingui-las.

Cabe observar que, caso cada empregado trabalhasse em um único projeto, a chave primária seria composta apenas pela coluna `CodEmp`. Neste caso, a coluna `CodProj` não faria parte da chave primária e seria apenas uma das colunas não chave da tabela em questão.

Neste ponto do processo de normalização, pode ser difícil encontrar nomes adequados para as tabelas. No exemplo, os nomes das tabelas foram inspirados nas chaves primárias das tabelas. Assim, a segunda tabela, que tem como chave primária as colunas `CodProj` e `NumEmp` foi denominada `ProjEmp`. Como os nomes das tabelas não são relevantes ao processo de normalização, a recomendação é estabelecer os nomes definitivos das tabelas apenas ao final da normalização.

Outro exemplo de passagem à primeira forma normal é o do arquivo de alunos visto acima (Figuras 6.4 e 6.5). Conforme mostrado anteriormente, sua representação não-normalizada é a seguinte:

ÑN

```
Arq-Alunos (Cod-Al, Nome-Al,
            (Cod-Curso, Sem-ingresso)
            (Cod-Disc,
             (Sem-Disc-Cursada, Nota-Disc)))
```

Como esta tabela ÑN contém três tabelas aninhadas, ela gerará quatro tabelas na 1FN (tantas tabelas quantos abre parênteses aparecem na forma ÑN). Na 1FN, as tabelas, ainda sem a chave primária, são as seguintes:

1FN

```
Alunos      (Cod-Al, Nome-Al)
AlunoCurso  (Cod-Al, Cod-Curso, Sem-ingresso)
AlunoDisc   (Cod-Al, Cod-Disc)
AlunoDiscSem (Cod-Al, Cod-Disc, Sem-Disc-Cursada, Nota-Disc)
```

A primeira tabela corresponde à tabela ÑN sem suas tabelas aninhadas. Já a tabela `AlunoCurso` na 1FN corresponde à primeira tabela aninhada na forma ÑN. Observe que esta tabela contém, além das colunas `CodCurso` e `Sem-ingresso`, a coluna `Cod-Al`, chave primária da tabela na qual a encontra-se tabela aninhada, na forma ÑN.

Pelo mesmo motivo, a tabela `AlunoDiscSem`, que corresponde à terceira tabela aninhada, contém as colunas `Cod-Al` e `Cod-Disc`. Estas são as chaves primárias das tabelas nas quais a tabela aninhada em questão encontra-se na forma $\tilde{N}N$.

As chaves primárias das tabelas na 1FN são as seguintes:

1FN

```
Alunos      (Cod-Al, Nome-Al)
AlunoCurso  (Cod-Al, Cod-Curso, Sem-ingresso)
AlunoDisc   (Cod-Al, Cod-Disc)
AlunoDiscSem (Cod-Al, Cod-Disc, Sem-Disc-Cursada, Nota-Disc)
```

Conforme mencionado, a chave primária de uma tabela na 1FN não é necessariamente a concatenação das chaves primárias das colunas chaves primárias na forma $\tilde{N}N$. A seguir, apresentamos um exemplo em que, ao contrário dos que foram mostrados até agora, a chave primária da tabela na 1FN não é a concatenação das chaves das tabelas na forma $\tilde{N}N$. Considere a tabela não-normalizada apresentada abaixo:

$\tilde{N}N$

```
Arq-Candidatos (Cod-Curso, Nome-Curso, Numero-Vagas-Curso,
                (Cod-Cand, Nome-Cand, Escore-Cand) )
```

Esta tabela representa um arquivo que armazena informações sobre um concurso vestibular. O arquivo contém um registro para cada curso, com código, nome e número de vagas do curso. Além disso, para cada curso, há uma lista dos candidatos aprovados. Supõe-se que cada candidato tenha sido aprovado em um curso somente.

A passagem à 1FN gera as tabelas abaixo:

1FN

```
Cursos      (Cod-Curso, Nome-Curso, Numero-Vagas-Curso)
Candidatos  (Cod-Curso, Cod-Cand, Nome-Cand, Escore-Cand)
```

Observe que, na segunda tabela, correspondente à tabela aninhada da forma $\tilde{N}N$, a chave primária é apenas a coluna `Cod-Cand`(código do candidato), já que um candidato pode aparecer somente uma vez no arquivo.

Para determinar a chave primária de uma tabela na 1FN, que corresponda a uma tabela aninhada na forma $\tilde{N}N$, deve-se proceder como segue:

- 1 Tomar como parte da chave primária da tabela na 1FN a chave primária da tabela ÑN.
- 2 Verificar se esta chave primária é suficiente para identificar as linhas da tabela na 1FN.
 - Caso seja suficiente, a chave primária da tabela na 1FN é a mesma que a da tabela aninhada na forma ÑN.
 - Caso contrário, deve-se determinar quais as demais colunas necessárias para identificar as linhas da tabela na 1FN, compondo assim a chave primária na 1FN.

6.5.2 dependência funcional

Para entender as duas formas normais que serão apresentadas a seguir é necessário compreender o conceito de *dependência funcional*.

dependência funcional
=

em uma tabela relacional, diz-se que uma coluna C_2 *depende funcionalmente* de uma coluna C_1 (ou que a coluna C_1 *determina* a coluna C_2) quando, em todas as linhas da tabela, para cada valor de C_1 que aparece na tabela, aparecer o mesmo valor de C_2

O conceito fica mais compreensível se considerarmos um exemplo. A tabela da Figura 6.7 contém, entre outras colunas irrelevantes ao exemplo, as colunas Código e Salário. Diz-se que a coluna Salário *depende funcionalmente* da coluna Código (ou que a coluna Código *determina* a coluna Salário) pelo fato de cada valor de Código estar associado sempre ao mesmo valor de Salário. Exemplificando, o valor "E1" da coluna Código identifica sempre o mesmo valor de Salário ("10").

Para denotar esta dependência funcional, usa-se uma expressão na forma Código → Salário. A expressão denota que a coluna Salário depende funcionalmente da coluna Código. Diz-se, também, que a coluna Código é o *determinante* da dependência funcional.

De forma geral, o determinante de uma dependência funcional pode ser um *conjunto de colunas* e não somente uma coluna como na definição acima.

...	Código	...	Salário	...
	E1		10	
	E3		10	
	E1		10	
	E2		5	
	E3		10	
	E2		5	
	E1		10	

Figura 6.7 Parte de tabela com dependências funcionais.

Exemplificando, na tabela da Figura 6.8, a coluna C depende das colunas A e B, ou seja, há uma dependência funcional $(A, B) \rightarrow C$.

Deve-se observar que uma dependência funcional é uma restrição de integridade, ou seja, ela deve ser obedecida por todos os estados do banco de dados. Assim, para determinar uma dependência funcional, não basta observar o estado atual do banco de dados. É necessário conhecer suas regras de construção.

6.5.3 passagem à segunda forma normal (2FN)

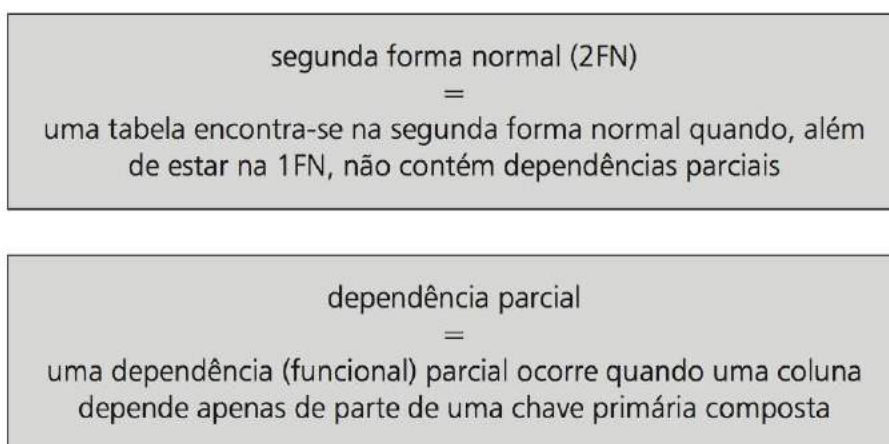
A passagem à segunda forma normal (2FN) elimina um certo tipo de redundância de dados. Para exemplificar, tomamos a tabela ProjEmp da Figura 6.3. Nesta tabela, os dados referentes a empregados (Nome, Cat e Sal) estão

A	B	C	D	E
B	5	2	20	x
C	4	2	15	c
B	6	7	20	a
B	5	2	20	y
C	2	2	15	b
C	4	2	15	k
A	10	5	18	k
A	12	3	18	f
A	10	5	18	f
B	5	2	20	c
C	4	2	15	x
A	10	5	18	r
C	4	2	15	s

Figura 6.8 Tabela com exemplos de dependências funcionais.

redundantes para os empregados que trabalham em mais de um projeto. Um exemplo é o do empregado de código "8191". A passagem à segunda forma normal elimina este tipo de redundância de dados.

Uma tabela encontra-se na segunda forma normal (2FN) quando, além de encontrar-se na primeira forma normal, cada coluna não chave depende da chave primária *completa*. Uma tabela que não se encontra na segunda forma normal contém *dependências funcionais parciais*, ou seja, contém colunas não chave que dependem apenas de uma parte da chave primária.



Obviamente, uma tabela que está na 1FN e que possui apenas uma coluna como chave primária não contém dependências parciais, já que, nesta tabela, é impossível uma coluna depender de uma parte da chave primária, visto que a chave primária não é composta por partes (por diversas colunas). Assim, toda tabela que está na 1FN e que possui apenas uma coluna como chave primária já está na 2FN. O mesmo aplica-se para uma tabela que contenha apenas colunas chave primária.

No exemplo, a tabela `Proj` encontra-se na 2FN por possuir uma chave primária simples (composta por apenas uma coluna).

Já a tabela `ProjEmp` deve ser examinada para procurar dependências parciais, pois possui uma chave primária composta de mais de uma coluna. As colunas `Nome`, `Cat` e `Sal` dependem, cada uma, apenas da coluna `CodEmp`, já que nome, categoria funcional e salário são determinados tão -somente pelo código do empregado. Por sua vez, as colunas `DataIni` e `TempAl` dependem da chave primária completa (para determinar a data em que um empregado começou a trabalhar em um projeto, bem como para determinar o tempo

pelo qual ele foi alocado ao projeto é necessário conhecer tanto o código do projeto quanto o número do empregado).

Para passar à 2FN, isto é, para eliminar as dependências de parte da chave primária é necessário dividir a tabela ProjEmp em duas tabelas com o seguinte esquema:

1FN

ProjEmp (CodProj, CodEmp, DataIni, TempAl)

Emp (CodEmp, Nome, Cat, Sal)

O processo de passagem à 2FN é ilustrado na Figura 6.9. Assim o modelo relacional correspondente ao arquivo em questão é o seguinte, na 2FN.

Tabela na primeira forma normal (1FN)
e dependências funcionais

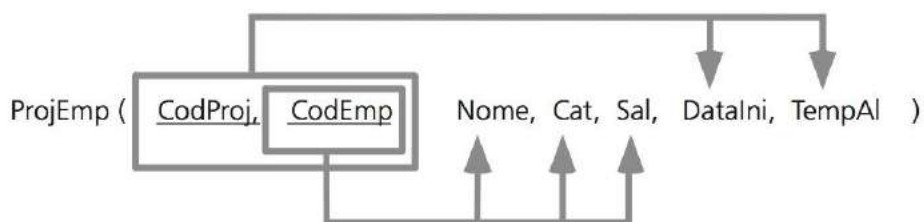


Tabela na segunda forma normal (2FN)
e dependências funcionais

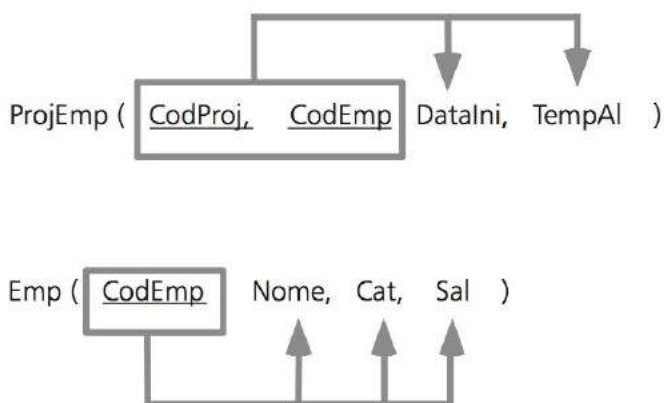


Figura 6.9 Passagem à 2FN.

2FN

Proj (CódProj, Tipo, Descr)

ProjEmp (CódProj, CodEmp, DataIni, TempAl)

Emp (CodEmp, Nome, Cat, Sal)

O conteúdo das tabelas na 2FN, caso considerarmos o conteúdo do documento original que está sendo normalizado, é apresentado na Figura 6.10.

De forma mais precisa, o processo de passagem da 1FN à 2FN é o descrito a seguir.

- 1 Copiar para a 2FN cada tabela que tenha chave primária simples ou que não tenha colunas além da chave. No exemplo, é o que acontece com a tabela Proj.

Proj:

CódProj	Tipo	Descr
LSC001	Novo Desenv.	Sistema de Estoque
PAG02	Manutenção	Sistema de RH

ProjEmp:

CódProj	CodEmp	DataIni	TempAl
LSC001	2146	1/11/91	24
LSC001	3145	2/10/91	24
LSC001	6126	3/10/92	18
LSC001	1214	4/10/92	18
LSC001	8191	1/11/92	12
PAG02	8191	1/05/93	12
PAG02	4112	4/01/91	24
PAG02	6126	1/11/92	12

Emp:

CodEmp	Nome	Cat	Sal
2146	João	A1	4
3145	Sílvio	A2	4
6126	José	B1	9
1214	Carlos	A2	4
8191	Mário	A1	4
4112	João	A2	4

Figura 6.10 Tabelas referentes ao exemplo na 2FN.

2 Para cada tabela com chave primária composta e com pelo menos uma coluna não chave (no exemplo, a tabela ProjEmp):

a Criar, na 2FN, uma tabela com as chaves primárias da tabela na 1FN (no exemplo, trata-se da tabela ProjEmp na 2FN).

b Para cada coluna não chave, fazer a seguinte pergunta:

“a coluna depende de toda a chave ou de apenas parte dela?”

■ Caso a coluna dependa de toda a chave (as colunas DataIni e TempAl da tabela ProjEmp):

■ Criar a coluna correspondente na tabela com a chave completa na 2FN (a coluna DataIni e TempAl da tabela ProjEmp na 2FN).

■ Caso a coluna dependa apenas de parte da chave (as colunas Nome, Sal e Cat da tabela ProjEmp na 2FN):

i. Criar, caso ainda não existir, uma tabela na 2FN que tenha como chave primária a parte da chave que é determinante da coluna em questão (a tabela Emp na 2FN).

ii. Criar a coluna dependente dentro da tabela na 2FN (as colunas Nome, Sal e Cat da tabela Emp na 2FN).

6.5.4 passagem à terceira forma normal (3FN)

Na passagem à terceira forma normal, elimina-se outro tipo de redundância. Para exemplificar, vamos considerar a tabela Emp da Figura 6.10. Vamos supor que o salário (coluna Sal) de um empregado seja determinado pela sua categoria funcional (coluna Cat). Neste caso, a informação de qual salário é pago a uma categoria funcional está representado redundantemente na tabela, tantas vezes quantos empregados a categoria funcional possui. A passagem à 3FN elimina este tipo de redundância de dados.

Uma tabela encontra-se na 3FN quando, além de estar na 2FN, toda coluna não chave depende *diretamente* da chave primária, isto é, quando não há dependências funcionais *transitivas* ou *indiretas*. Uma dependência funcional transitiva ou indireta acontece quando uma coluna não chave primária depende funcionalmente de outra coluna ou combinação de colunas não chave primária.

terceira forma normal (3FN)

=

uma tabela encontra-se na terceira forma normal quando, além de estar na 2FN, não contém dependências transitivas

dependência transitiva
=
uma dependência funcional transitiva ocorre quando uma
coluna, além de depender da chave primária da tabela, depende
de outra coluna ou conjunto de colunas da tabela

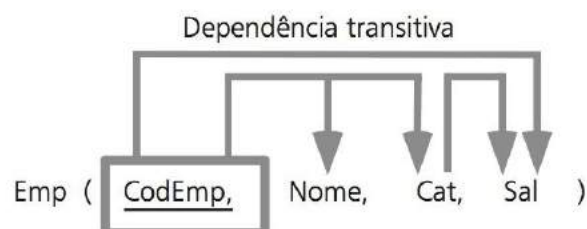
A passagem à 3FN consiste em dividir tabelas, de forma a eliminar as dependências transitivas. O processo de passagem à 3FN para o exemplo é mostrado na Figura 6.11.

Com isso, na 3FN, o modelo referente ao documento sendo normalizado é o seguinte:

3FN

Proj (CodProj, Tipo, Descr)
ProjEmp (CodProj, CodEmp, DataIni, TempAl)
Emp (CodEmp, Nome, Cat)
Cat (Cat, Sal)

Tabela na segunda forma normal (2FN)



Tabelas na terceira forma normal

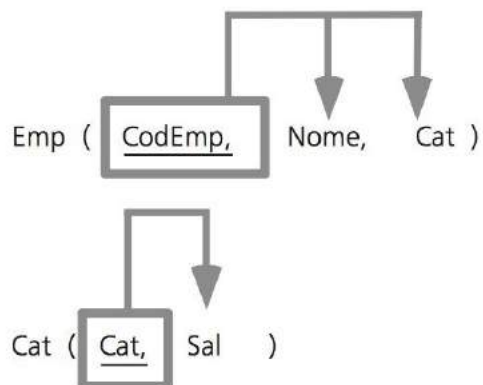


Figura 6.11 Passagem à 3FN para o exemplo.

As tabelas referentes à terceira forma normal do exemplo têm seu conteúdo mostrado na Figura 6.12.

De forma mais precisa, o processo de passagem da 2FN à 3FN é o seguinte:

- 1 Copiar para o esquema na 3FN cada tabela que tenha menos que duas colunas não-chave, pois neste caso não há como haver dependências transitivas.
- 2 Para tabelas com duas ou mais colunas não-chave:
 - a Criar uma tabela no esquema da 3FN com a chave primária da tabela em questão.
 - b Para cada coluna não-chave fazer a seguinte pergunta:
"a coluna depende de alguma outra coluna não-chave (dependência transitiva ou indireta)?"
 - Caso a coluna depender apenas da chave:
 - i. Copiar a coluna para a tabela na 3FN

Proj:

CódProj	Tipo	Descr
LSC001	Novo Desenv.	Sistema de Estoque
PAG02	Manutenção	Sistema de RH

ProjEmp:

CódProj	CodEmp	DataIni	TempAl
LSC001	2146	1/11/91	24
LSC001	3145	2/10/91	24
LSC001	6126	3/10/92	18
LSC001	1214	4/10/92	18
LSC001	8191	1/11/92	12
PAG02	8191	1/05/93	12
PAG02	4112	4/01/91	24
PAG02	6126	1/11/92	12

Emp:

CodEmp	Nome	Cat
2146	João	A1
3145	Sílvio	A2
6126	José	B1
1214	Carlos	A2
8191	Mário	A1
4112	João	A2

Cat:

Cat	Sal
A1	4
A2	4
B1	9

Figura 6.12 Tabelas referentes ao exemplo na 3FN.

- Caso a coluna depender de outra coluna:
 - i. Criar, caso ainda não exista, uma tabela no esquema na 3FN que tenha como chave primária a coluna da qual há a dependência indireta.
 - ii. Copiar a coluna dependente para a tabela criada.
 - iii. A coluna determinante deve permanecer também na tabela original.

6.5.5 passagem à quarta forma normal (4FN)

Para a maioria dos documentos e arquivos, a decomposição até a 3FN é suficiente para obter o esquema de um banco de dados correspondente ao documento. Na literatura, aparecem outras formas normais, como a forma normal de Boyce/Codd, a 4FN e a 5FN. Destas, a única que tem importância na prática da engenharia reversa é a quarta forma normal (4FN).

Para explicar a quarta forma normal, vamos utilizar um exemplo. Suponha que alguém tenha projetado um banco de dados relacional para o DER da Figura 6.13.

Neste DER, o relacionamento UTILIZAÇÃO indica que deseja-se manter a informação de qual empregado usa qual equipamento em qual projeto.

Caso seja projetado um banco de dados relacional para este exemplo usando as regras de projeto mostradas no capítulo anterior, o esquema do banco de

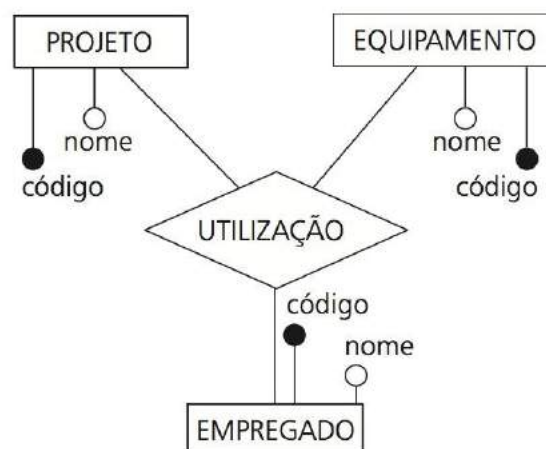


Figura 6.13 Exemplo de DER para utilização de equipamentos.

dados conterá quatro tabelas. Três servirão para implementar as entidades. Uma quarta tabela:

Util (CodProj, CodEmp, CodEquip)

servirá para implementar o relacionamento.

Agora suponha que os requerimentos da aplicação tenham mudado. O DER que representa os novos requerimentos é o apresentado na Figura 6.14. Como se observa, a granulosidade das informações que o usuário deseja manter não é mais tão grande como no caso da Figura 6.13. Agora deseja-se saber qual equipamento é usado em qual projeto (relacionamento *Proj-Eq*) e deseja-se saber que empregado está alocado a que projeto (relacionamento *Proj_Emp*). A informação de qual equipamento é usado por qual empregado passa a ser uma informação derivada: um empregado pode usar todos os equipamentos alocados aos projetos dos quais ele participa.

Continuando o exemplo, vamos supor que o banco de dados originalmente projetado para o relacionamento ternário não tenha sido modificado, apesar da existência de novos requerimentos. Um possível conteúdo para a tabela UTILIZAÇÃO é mostrado na Figura 6.15.

A tabela da Figura 6.15 obviamente contém redundância de dados. Exemplificando, a informação de que os empregados {"1","2","3"} trabalham no projeto "1" está representada duas vezes na tabela. Isso ocorre, por-

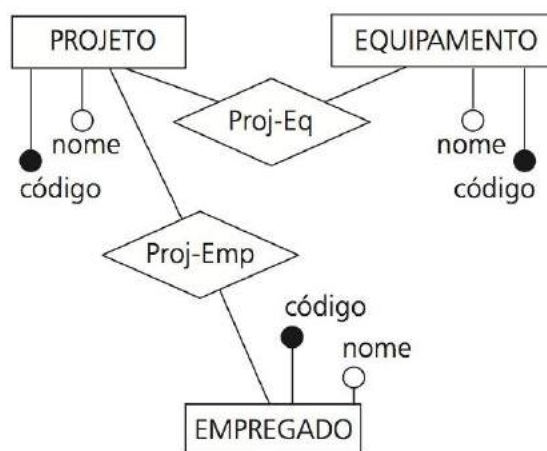


Figura 6.14 Exemplo da utilização de equipamentos – nova versão.

CodProj	CodEmp	CodEquip
1	1	1
1	2	1
1	3	1
1	1	2
1	2	2
1	3	2
2	2	2
2	2	4
3	3	1
3	4	1
3	3	3
3	4	3
3	3	5
3	4	5
4	2	5

Figura 6.15 Tabela que implementa o relacionamento UTILIZAÇÃO.

que para cada equipamento usado no projeto é necessário informar todos os seus empregados. A consequência é que também a informação de quais equipamentos são usados em um projeto está armazenada redundantemente. Exemplificando, o fato de o projeto "1" usar os equipamentos {"1","2"} está representada três vezes, já que o projeto conta com três empregados.

Entretanto, se validarmos essa tabela contra as formas normais vistas até aqui, observamos que a tabela encontra-se na 3FN. Está na 1FN por não conter tabelas aninhadas, está na 2FN por não possuir colunas não-chave (as três colunas compõem a chave) e está na 3FN pela mesma razão.

Para evitar este tipo de redundância de dados, é necessário considerar uma outra forma normal, a *quarta forma normal* (4FN), que se baseia no conceito de *dependência funcional multivalorada*.

Uma coluna ou conjunto de colunas depende multivaloradamente de uma coluna (determinante) da mesma tabela quando um valor do atributo determinante identifica repetidas vezes um *conjunto* de valores na coluna dependente. No exemplo (ver Figura 6.16), a coluna CodEmp depende multivaloradamente da coluna CodProj, já que um valor de CodProj (por exemplo "1") determina múltiplas vezes um conjunto de valores de CodEmp (no caso o conjunto de empregados do projeto, {"1","2","3"}, que aparece duas vezes).

CodProj	CodEmp	CodEquip
1	1	1
1	2	1
1	3	1
1	1	2
1	2	2
1	3	2
2	2	-

Figura 6.16 Dependência funcional multivalorada.

Para representar uma dependência funcional multivalorada usa-se uma expressão semelhante à usada para representar dependências funcionais simples, substituindo a seta simples por uma seta dupla. A tabela UTILIZAÇÃO contém as seguintes dependências funcionais multivaloradas:

$\text{CodProj} \twoheadrightarrow \text{CodEmp}$
 $\text{CodProj} \twoheadrightarrow \text{CodEquip}$

Uma tabela está na 4FN caso, além de estar na 3FN, não possua mais que uma dependência funcional multivalorada.

quarta forma normal (4FN)
 =
 uma tabela encontra-se na quarta forma normal, quando,
 além de estar na 3FN, não contém dependências
 multivaloradas

Portanto, a tabela UTILIZAÇÃO não está na 4FN e deve ser decomposta em duas tabelas:

$\text{ProjEmp} (\text{CodProj}, \text{CodEmp})$
 $\text{ProjEquip} (\text{CodProj}, \text{CodEquip})$

Estas tabelas correspondem à implementação dos dois relacionamentos da Figura 6.14.

Se considerarmos o conteúdo da tabela apresentada na Figura 6.15, o conteúdo das tabelas acima seria aquele apresentado na Figura 6.17.

ProjEquip:		ProjEmp:	
CodProj	CodEquip	CodProj	CodEmp
1	1	1	1
1	2	1	2
2	2	1	3
2	4	2	2
3	1	3	3
3	3	3	4
3	5	4	2
4	5		

Figura 6.17 Tabelas do exemplo na 4FN.

6.5.6 problemas da normalização

Nesta seção, vamos discutir alguns problemas que podem aparecer na prática da normalização de arquivos.

■ chaves primárias omitidas ou incorretas

Em arquivos convencionais, o conceito de chave primária não é obrigatório, como ocorre na abordagem relacional. Assim, é possível encontrar arquivos que não possuem chave primária. Quando um arquivo convencional não possui chave primária ou quando a chave primária nele usada difere da usual na organização, deve-se proceder como se a chave primária aparecesse no arquivo, isto é, deve-se inseri-la na forma ÑÑ.

Exemplificando, em um arquivo com dados sobre empregados de uma organização, enviado para fins de fiscalização a um órgão governamental, pode estar omitido o identificador de empregado usado na organização, já que este é irrelevante para o órgão fiscalizador.

Uma variante da situação descrita é a de documentos nos quais uma chave primária é propositadamente omitida, por não ser relevante para os leitores do arquivo. Um exemplo é apresentado no documento do Exercício 6 deste capítulo.

Outra situação análoga é o uso de uma chave alternativa, em vez da chave primária usual do arquivo. Exemplificando, no caso mencionado acima, se o órgão governamental fosse a receita federal, o arquivo poderia ter como

chave primária o CPF do empregado, em vez da chave primária normalmente usada na organização.

Em ambos os casos, se a normalização fosse executada diretamente sobre o arquivo convencional, apareceriam tabelas espúrias, com chaves primárias diferentes das usadas para as entidades representadas. Assim, tão logo uma destas situações seja detectada, deve-se introduzir, já na forma $\tilde{N}N$, a chave primária usual da tabela, como se ela aparecesse no arquivo normalizado.

■ atributos relevantes implicitamente representados

Atributos podem aparecer em arquivos convencionais implicitamente, na forma de ordenação de registros ou de listas, como ponteiros físicos, etc. Quando esta situação ocorrer, deve-se proceder como se o atributo aparecesse explicitamente no documento.

Um exemplo é o da ordenação de registros ou elementos de listas, onde a própria ordem é uma informação relevante. Para exemplificar, vamos retomar o exemplo do concurso vestibular apresentado anteriormente, agora com uma modificação. Na nova versão, não existe a coluna *Escore-Cand*, que continha a nota do aluno. Consideramos agora que os alunos aparecem ordenados no registro do curso, de acordo com sua classificação no vestibular. Seguindo os passos usuais da normalização, a tabela na forma $\tilde{N}N$ seria a seguinte:

$\tilde{N}N$

Arq-Candidatos (Cod-Curso, Nome-Curso, Numero-Vagas-Curso,
(Cod-Cand, Nome-Cand))

O processo de normalização desta tabela resultaria nas seguintes tabelas:

4FN

Cursos (Cod-Curso, Nome-Curso, Numero-Vagas-Curso)
Candidatos (Cod-Curso, Cod-Cand, Nome-Cand)

Como na abordagem relacional as linhas não estão ordenadas, a informação da classificação dos candidatos em um curso foi perdida durante o processo de normalização. Assim, o procedimento correto teria sido o de incluir explicitamente, na tabela, já na forma $\tilde{N}N$, a informação que aparece implicitamente no arquivo na forma da ordenação dos registros (coluna *Ordem-Cand*). A tabela na forma $\tilde{N}N$ que resultaria neste caso é apresentada a seguir:

ÑÑ

Arq-Candidatos (Cod-Curso, Nome-Curso, Numero-Vagas-Curso,
(Cod-Cand, Nome-Cand, Ordem-Cand))

Outro exemplo de atributo implicitamente representado são apontadores ou referências físicas a registros. Estes apontadores devem ser substituídos, quando da passagem a forma ÑÑ, pela chave primária do arquivo que está sendo referenciado.

■ atributos irrelevantes, redundantes ou derivados

Arquivos convencionais podem conter atributos que não são relevantes do ponto de vista conceitual e que existem no arquivo por questões técnicas ou de desempenho da implementação em questão. Exemplos destes atributos são campos contendo o número de ocorrências de listas, o tamanho de outros campos, estampas de tempo, etc. Estes campos devem ser eliminados já na passagem à forma não-normalizada.

6.6

→ integração de modelos

A normalização de cada um dos arquivos ou documentos de um sistema conduz à definição de um conjunto de tabelas. O passo seguinte da engenharia reversa é o de integrar os modelos obtidos para cada arquivo no modelo *global* do banco de dados (ver Figura 6.1). Esse processo é conhecido na literatura de banco de dados por *integração de visões* ou *integração de esquemas*.

Esse processo tem dois objetivos:

- Os atributos de uma mesma entidade (ou de um mesmo relacionamento) podem estar armazenados em diferentes arquivos. Após o processo de normalização, estes atributos aparecem como colunas em diferentes tabelas. O processo de integração de visões objetiva juntar estas tabelas em uma única tabela que representa a entidade ou relacionamento em questão.
- O processo de normalização garante que cada tabela, se considerada isoladamente, esteja livre de redundância de dados. Entretanto, certos casos de redundância de dados entre tabelas podem permanecer e devem ser eliminados durante o processo de integração de visões.

O processo de integração de modelos dá-se em três passos:

- 1 integração de tabelas com a mesma chave,
- 2 integração de tabelas com chave contida e
- 3 verificação de 3FN.

A seguir, cada um desses passos é descrito em detalhes.

6.6.1 integração de tabelas com mesma chave

O processo de integração de modelos inicia pela junção de tabelas que possuem a *mesma* chave primária. Aqui, quando falamos de “mesma” chave primária, estamos exigindo que os *domínios* e os *conteúdos* das colunas que compõem a chave primária sejam iguais. Quando isso ocorrer, as diferentes tabelas devem ser juntadas em uma única tabela no modelo global.

Para exemplificar, retomamos o sistema de gerência de projetos mostrado anteriormente. O documento cuja normalização foi apresentada resultou no modelo relacional abaixo:

Documento 1:

```
Proj (CodProj, Tipo, Descr)
ProjEmp (CodProj, CodEmp, DataIni, TempAl)
Emp (CodEmp, Nome, Cat)
Cat (Cat, Sal)
```

Adicionalmente, vamos considerar que outro documento tenha sido normalizado, resultando no modelo relacional abaixo:

Documento 2:

```
Proj (CodProj, DataInicio, Descr, CodDepto)
Depto (CodDepto, NomeDepto)
ProjEquipamento (CodProj, CodEquipam, DataIni)
ProjEmp (CodProj, CodEmp, FunçãoEmpProj)
Equipamento (CodEquipam, Descrição)
```

Caso a coluna *CodProj* tenha o mesmo valor nas tabelas *Proj* dos dois modelos, estas duas tabelas devem ser juntadas no modelo global. Observe que a coluna *Tipo* é proveniente do documento 1, as colunas *DataInicio* e *CodDepto* são provenientes do documento 2 e a coluna *Descr* é proveniente de ambos os documentos.

Da mesma forma, caso as colunas `CodProj` e `CodEmp` tenham os mesmos valores nas tabelas `ProjEmp` de ambos os modelos, estas duas tabelas devem ser fundidas em uma única tabela.

O modelo global resultante da integração dos dois modelos acima é o seguinte.

Modelo integrado:

```
Proj (CodProj, Tipo, Descr, DataInicio, CodDepto)
ProjEmp (CodProj, CodEmp, DataIni, TempAl, FunçãoEmpProj)
Emp (CodEmp, Nome, Cat)
Cat (Cat, Sal)
Depto (CodDepto, NomeDepto)
ProjEquipamento (CodProj, CodEquipam, DataIni)
Equipamento (CodEquipam, Descrição)
```

Como se vê no exemplo, todo o processo de integração de modelos baseia-se na comparação dos *nomes* de colunas e de tabelas dentro dos diferentes modelos. Nesta comparação, podem aparecer dois tipos de *conflitos de nomes*. *Homônimos* ocorrem quando dois objetos diferentes aparecem sob o mesmo nome. Por exemplo, diferentes campos, como data de nascimento e data de admissão de empregado aparecem em dois modelos diferentes sob o mesmo nome `DataEmp`. *Sinônimos* ocorrem quando um mesmo objeto aparece sob diferentes nomes. Por exemplo, a coluna descrição de projeto aparece em um modelo sob o nome `DescriçãoProj` e em outro modelo sob o nome `TítuloProj`.

Conflitos de nomes são resolvidos através de renomeação. Infelizmente, ainda não há técnicas na literatura que resolvam de forma sistemática o problema do conflito de nomes. Assim, a resolução deste conflito continua dependendo muito da experiência e do conhecimento do modelador.

6.6.2 integração de tabelas com chaves contidas

Outra situação na qual ocorre a integração de tabelas é quando uma tabela contém somente a chave primária e esta chave primária é subconjunto da chave primária de outra tabela. Note que, novamente, quando falamos de uma chave primária estar contida dentro da outra, estamos exigindo que a chave primária tenha o mesmo domínio e os mesmos valores.

Como exemplo, vamos considerar as duas tabelas a seguir:

AlunoDisc(Cod-Al, Cod-Disc)

AlunoDiscSem(Cod-Al, Cod-Disc, Sem-Disc-Cursada, Nota-Disc)

Vamos considerar que a primeira tabela informa que um aluno cursou uma disciplina, enquanto que a segunda tabela informa a nota obtida pelo aluno em uma disciplina em um semestre. Neste caso, as colunas Cod-Al e Cod-Disc da tabela AlunoDisc contêm exatamente os mesmos valores que as colunas CodAl e CodDisc da tabela AlunoDiscSem. As informações contidas na tabela AlunoDisc já estão na tabela AlunoDiscSem. Portanto, a tabela AlunoDisc é redundante e pode ser eliminada sem perda de informações.

Observe que, para integrar uma tabela em outra, estamos exigindo que a tabela contenha somente a chave primária. Para exemplificar um caso em que esta exigência não é cumprida, vamos considerar as duas tabelas abaixo. A primeira tabela informa se um aluno teve ou não bolsa de estudos em um semestre, enquanto que a segunda informa a nota obtida pelo aluno em uma disciplina cursada em um semestre. Consideramos que as colunas Cod-Al e Sem-Disc da tabela AlunoSem conttenham exatamente os mesmos valores que as colunas CodAl e Sem-Disc da tabela AlunoDiscSem.

AlunoSem(Cod-Al, Sem-Disc, BolsaSimNao)

AlunoDiscSem(Cod-Al, Cod-Disc, Sem-Disc, Nota-Disc)

Neste exemplo, a tabela AlunoSem não deve ser integrada à tabela AlunoDiscSem, pois, apesar de a chave primária da primeira tabela estar contida na chave primária da segunda tabela, a primeira possui atributos não-chave. Se o atributo *BolsaSimNao* fosse incluído na segunda tabela, a informação da obtenção ou não de bolsa por um estudante apareceria múltiplas vezes no banco de dados, tantos quantas disciplinas o aluno cursou no semestre.

Outro caso no qual a integração das tabelas não deve ser feita é quando os valores das colunas chave primária não são iguais, apesar de possuírem os mesmos nomes e domínios. Como exemplo, vamos considerar as tabelas abaixo:

AlunoMatric(Cod-Al, Sem-Disc)

AlunoDiscSem(Cod-Al, Cod-Disc, Sem-Disc, Nota-Disc)

Neste exemplo, consideramos que a primeira tabela (AlunoMatric) representa o fato de o aluno estar matriculado em um semestre. A segunda tabela (AlunoDiscSem) representa a nota que o aluno obteve em uma disciplina em

um semestre. Podem haver estados do banco de dados em que um aluno aparece associado a um semestre na tabela *AlunoMatric*, sem que exista informação sobre este aluno/semestre na tabela *AlunoDiscSem*. Este seria o estado do banco de dados durante o semestre letivo, quando um aluno já está matriculado, mas ainda não obteve notas. Neste caso, a primeira tabela não pode ser eliminada, pois contém informações que não aparecem na segunda tabela.

6.6.3 volta à 2FN

A integração de dois modelos, que caso considerados isoladamente estão na 4FN, pode conduzir a um modelo que está na 2FN mas não na 3FN.

Consideremos os modelos abaixo, ambos obtidos a partir de diferentes arquivos:

Arquivo 1:

Departamento (CodDepto, NomeDepto, CodGerenteDepto)

Arquivo 2:

Departamento (CodDepto, LocalDepto, NomeGerenteDepto)

A integração destes dois modelos resultaria no modelo integrado mostrado a seguir.

Modelo integrado:

Departamento (CodDepto, NomeDepto, CodGerenteDepto,
LocalDepto, NomeGerenteDepto)

No modelo integrado, a tabela *Departamento* encontra-se na 2FN, mas não na 3FN, se considerarmos que a coluna não-chave *NomeGerenteDepto* depende funcionalmente da coluna não-chave *CodGerenteDepto*.

O problema ocorrido é que o Arquivo 2 referencia o gerente de um departamento, não através da chave primária de gerentes (que é seu código), mas através do nome do gerente. Esse é mais um exemplo das consequências da omissão de chaves primárias em arquivos convencionais, que havia sido citado acima.

Assim, deve-se, após a integração de modelos, verificar as tabelas a fim de garantir que todas estejam efetivamente na 3FN.

6.7 → **finalização de modelo****6.7.1 construção do modelo ER**

Após a integração dos modelos obtidos a partir dos diversos arquivos e documentos normalizados, segue a construção do modelo ER (ver Figura 6.1). Nesta construção, usam-se as regras apresentadas no capítulo anterior para a transformação de modelos relacionais em modelos ER.

6.7.2 verificação do modelo ER – limitações da normalização

A normalização não conduz necessariamente a um modelo ER perfeito, pois ela apenas elimina campos multivalorados e redundância de dados detectadas pelas formas normais descritas.

Conforme mencionado na Seção Passagem à primeira forma normal (1FN) (página 190), na passagem à 1FN optamos pela alternativa de decompor tabelas. Esta alternativa, apesar de ser mais simples na prática, pode levar a imperfeições no modelo (ver exemplo no Exercício 17).

Além das formas normais aqui descritas, outras foram propostas na literatura, como a forma normal de Boyce/Codd e a quinta forma normal (ver referências bibliográficas no final do capítulo). Como estas formas normais tratam de casos que aparecem pouco frequentemente na prática, preferimos não apresentá-las neste texto.

Assim, o último passo da engenharia reversa é a verificação do modelo ER obtido, procurando corrigir imperfeições ainda existentes. Para ver exemplos de alguns problemas que podem permanecer no modelo normalizado, o leitor pode estudar os exemplos de engenharia reversa que aparecem nos exercícios a seguir, particularmente os referentes ao sistema de preparação de congressos e ao sistema de controle do almoxarifado.

6.8 → **exercícios**

A lista de exercícios a seguir contém, além de outras questões, dois estudos de caso de engenharia reversa do banco de dados de um sistema a partir de descrições de documentos e de arquivos convencionais.

Um sistema é o de preparação de congressos da IFIP que foi apresentado no Exercício 11 do capítulo 3. A ele, referem-se os exercícios 4 até 12 deste capítulo. O outro sistema é o de controle de um almoxarifado que foi apresentado no Exercício 12 do capítulo 3. A ele, referem-se os exercícios 13 até 22 deste capítulo. Para resolver estes exercícios, é recomendável que o leitor tenha lido a descrição do respectivo sistema no capítulo 3.

exercício 1 Considerando apenas o conteúdo atual da tabela apresentada na Figura 6.8, identifique quais as dependências funcionais nela contidas.

exercício 2 Considere a tabela abaixo:

```
ItemVenda (NúmeroNF, CódigoTipoProd, NúmeroProd,
           DescriçãoProd, DataVenda, CodReg, CodEmp, QtdeItem,
           PreçoItem, NomeEmp, DescriçãoTipoProd)
```

O significado das colunas acima é aquele apresentado no Exercício 4 do capítulo. A tabela apresentada está na primeira forma normal. Apresente a segunda e terceira formas normais.

exercício 3 No contexto de um sistema de controle acadêmico, considere a seguinte tabela:

```
Matricula (CodAluno, CodTurma, CodDisciplina, NomeDisciplina,
           NomeAluno, CodLocalNascAluno, NomeLocalNascAluno)
```

As colunas possuem o seguinte significado:

- CodAluno-código do aluno matriculado
- CodTurma-código da turma na qual o aluno está matriculado (código é o identificador de turma)
- CodDisciplina-código que identifica a disciplina da turma
- NomeDisciplina-nome de uma disciplina da turma
- NomeAluno-nome do aluno matriculado
- CodLocalNascAluno-código da localidade em que nasceu o aluno
- NomeLocalNascAluno-nome da localidade em que nasceu o aluno

Verifique se a tabela obedece a segunda e a terceira forma normais. Caso não obedeça, faça as transformações necessárias.

exercício 4 (Refere-se ao sistema de preparação de congressos descrito no Exercício 11 do capítulo 3.) A Figura 6.18 apresenta uma lista de todos os artigos submetidos a um congresso.

No cabeçalho, aparece o código e o nome do congresso. A seguir, são listados os códigos e nomes dos GTs que promovem o congresso. Posteriormente, em várias colunas são listados o código do artigo, seu título,

Relação de artigos submetidos ao congresso

Congresso: DB25 – *Advances in Database Systems*

GTs promotores: GT3.1 – *Database Systems*
GT3.3 – *Database Conceptual Modeling*

Código do artigo	Título do artigo	Assunto principal	Código do autor	Nome do autor
1	<i>Semantic Integration in Heterogeneous Databases</i>	<i>Heterogeneous Databases</i>	2	<i>Wen-Suan Li</i>
			4	<i>Chris Clifton</i>
2	<i>Providing Dynamic Security in a Federated Database</i>	<i>Heterogeneous Databases</i>	21	<i>N.B. Idris</i>
			7	<i>W.A. Gray</i>
			32	<i>R.F. Churchhouse</i>
3	<i>Efficient and effective clustering methods</i>	<i>Spatial databases</i>	12	<i>Raymond R. Ng</i>
			14	<i>Jiawei Han</i>
4	<i>Automated Performance Tuning</i>	<i>Performance and Optimization</i>	36	<i>Kurt. P. Brown</i>
5	<i>Bulk Loading into an OODB</i>	<i>Object oriented databases</i>	1	<i>Janet L. Wiener</i>

Congresso: OO03 – *Object Oriented Modeling*

GTs promotores: GT4.6 – *Software Engineering*

Código do artigo	Título do artigo	Assunto principal	Código do autor	Nome do autor
1	<i>Temporal aspects in OO models</i>	<i>Temporal modeling</i>	2	<i>Wen-Suan Li</i>

Figura 6.18 Lista de artigos submetidos aos congressos.

seu assunto principal, seu código de autor e os códigos e nomes dos vários autores do artigo. Observar que o mesmo código de artigo pode aparecer em diferentes congressos, já que a numeração de artigos inicia em um em cada congresso diferente.

Execute a normalização do documento, mostrando cada uma das formas normais.

exercício 5 (Refere-se ao sistema de preparação de congressos descrito no Exercício 11 do capítulo 3) A Figura 6.19 apresenta a definição de um arquivo convencional que armazena dados dos artigos submetidos aos vários congressos. A definição do artigo está na linguagem COBOL, na qual muitos sistemas legados estão escritos. O leitor poderá fazer este exercício mesmo que não conheça COBOL, apenas com base nas explicações a seguir.

O arquivo contém um registro para cada artigo submetido ao congresso. A exemplo do exercício precedente, um artigo é identificado pelo código do congresso ao qual foi submetido e pelo seu código. A descrição em COBOL nos informa que cada registro de artigo contém: o código do congresso, o nome do congresso, o código do artigo, o título do artigo, uma lista com

```

FD  ARQ-ARTIGOS.
01  REG-ARTIGO.
    03  COD-CONGR
    03  NOME-CONGR
    03  NUMERO-ART
    03  TIT-ART
    03  NO-DE-AUTORES
    03  NO-DE-REVISORES
    03  NO-DE-TEMAS
    03  AUTOR OCCURS 1 TO 20 TIMES
        DEPENDING ON NO-DE-AUTORES.
        05  COD-AUTOR
        05  NOME-AUTOR
    03  TEMA OCCURS 1 TO 5 TIMES
        DEPENDING ON NO-DE-TEMAS.
        05  COD-TEMA
        05  NOME-TEMA
    03  REVISOR OCCURS 1 TO 5 TIMES
        DEPENDING ON NO-DE-REVISORES.
        05  COD-REVISOR
        05  NOME-REVISOR
        05  STATUS-REVISAO

```

Figura 6.19 Arquivo COBOL que armazena os artigos submetidos.

os códigos e nomes dos diversos autores, uma lista com código e nome dos temas de que trata o artigo e uma lista com código, nome e *status* da revisão dos vários especialistas que estão julgando ou julgaram o artigo. As listas são especificadas em COBOL na forma de cláusulas OCCURS. Os campos NO-DE-AUTORES, NO-DE-TEMAS e NO-DE-REVISORES apenas servem para controlar as respectivas listas de campos. Portanto, não devem ser considerados na normalização, já que são redundantes (ver Seção Problemas da normalização). O campo de *status* da revisão pode conter dois valores diferentes e informa se o artigo já recebeu parecer do revisor ou não.

Execute a normalização do documento, mostrando cada uma das formas normais.

exercício 6 (Refere-se ao sistema de preparação de congressos descrito no Exercício 11 do capítulo 3.) A Figura 6.20 apresenta o programa de um congresso.

No cabeçalho constam o código e o nome do congresso. A seguir é listada a data a que se referem as informações seguintes (um congresso pode ocorrer em vários dias subseqüentes). Depois, são listadas as seções que são apresentadas no dia. Lembre que uma seção é um horário do congresso. Algumas seções, como a cerimônia de abertura (*Opening Ceremony*) ou o intervalo (*break*), não têm apresentação de artigos. Outras possuem apresentação de artigos, que são listados em ordem de apresentação após o título da seção. Esta seções possuem um moderador (*chair*) que aparece nomeado junto ao título da seção.

Este documento exemplifica vários casos especiais que devem ser tratados na normalização.

O documento omite vários códigos de entidades (ver Seção Problemas da normalização na página 207), que não são importantes para o leitor alvo do documento, mas são importantes na normalização. Estão omitidos o código do artigo e os códigos das pessoas que aparecem no modelo (autor e moderador).

Além disso, o documento apresenta de forma implícita (ver Seção Problemas da normalização na página 207) a informação da seqüência de apresentação dos artigos em uma seção.

Execute a normalização do documento, mostrando cada uma das formas normais.

**DB25 – Advances in Database Systems
Conference Program**

Monday September 12

09:00-10:30 Registration

10:00-10:45 Opening Ceremony

10:45-11:00 Break

11:00-12:30 Heterogeneous and Federated Databases (chair: Hervé Gallaire)

Semantic Integration in Heterogeneous Databases Using Neural Networks, Wen-Syan Li and Chris Clifton – USA

Providing Dynamic Security Control in a Federated Database, N.B. Idris, W.A. Gray and R.F. Churchhouse – UK

An approach for Building Secure Database Federations, Dirk Jonscher and Klaus R. Dittrich- Switzerland

12:30-14:00 Lunch

14:00-15:30 Issues on Architectures (Chair: Claudia Medeiros)

Optimization of Algorithms for Exploiting the Parallelism-Communication Tradeoff in Pipelined Parallelism, Waqar Hasan and Rajeev Motwani-USA

Dali: A High Performance Main Memory Storage Manager, H.V. Jagadish, Daniel Lieuwen, Rajeev Rastogi, Avi Silberschatz and S. Sudarshan- USA

Some Issues in Design of Distributed Deductive Databases, Mukesh K. Mohania and N.L. Sarda- India

15:30-16:30 Break

16:00-17:30 Performance and Optimization (Chair: Avi Silberschatz)

Towards Automated Performance Tuning for Complex Workloads, Kurt P. Brown, Manish Mehta, Michael Carey and Miron Livny- USA

...

Tuesday September 13

09:00-10:30 Object Oriented Databases (chair: Malkolm Atkinson)

Supporting Exceptions to Schema Consistency to Ease Schema Evolution in OODBMS, Eric Amiel, Maria-Jo Bellosta, Eric Dujardin and Eric Simon-France

...

Figura 6.20 Programa de um congresso.

exercício 7 (Refere-se ao sistema de preparação de congressos descrito no Exercício 11 do capítulo 3.) A Figura 6.21 apresenta a carta de aceitação de um artigo. Esta carta é enviada para o autor principal, portanto, há uma carta por artigo. Como cada artigo tem como identificador o código de seu congresso e seu código, estes dois campos são também os identificadores de uma carta de aceitação. Como no exercício anterior, há várias chaves primárias omitidas no documento. Deixamos ao leitor que as identifique e as inclua na normalização.

June 30, 1994

Carlos A. Heuser
Instituto de Informática – UFRGS
Av. Bento Gonçalves 9500,
Bloco IV Agronomia- Campus do Vale CEP 91501-970
Porto Alegre – RS. C.P. 15064 BRAZIL
Re: Paper #97 – “Partitioning and aggregation in object oriented modeling”

Dear Colleague:

I am pleased to inform you that the above referenced paper has been accepted for the DBOO94 - Database Modeling 94 . As you know, the conference will be held September 19-23, 1994 at ITESM, Estado de Mexico.

The instructions for the camera-ready version are attached to this letter. I'd like to draw your attention to the referees' report enclosed and to the 12-page limit required for publication in the Proceedings. **The final camera-ready copy of the manuscript should be received by us no later than July 25 .**

Your paper will only be included in the Conference Proceedings and in the final program if we receive along with the final manuscript the registration form and payment of the appropriate registration fee of the paper's presenter.

A preliminary program is enclosed. Any additional up-to-date information will be sent to you by e-mail or fax. If you have any questions about the conference or your participation, please contact the organizers at the address below. Alternatively, you can contact the IFIP representative in your country.

I am looking forward to seeing you at the conference.

Yours sincerely

Dirk Rastogi
Program Committee Chairman

Figura 6.21 Carta de aceitação de artigo.

Execute a normalização do documento, mostrando cada uma das formas normais.

exercício 8 (Refere-se ao sistema de preparação de congressos descrito no Exercício 11 do capítulo 3.) A Figura 6.22 apresenta a definição de um arquivo convencional que armazena dados dos congressos. A definição está em COBOL. Valem aqui as mesmas observações feitas com referência ao arquivo de artigos apresentado anteriormente.

```

FD    ARQ-CONGRESSO.
01    REG-CONGRESSO.
      03    COD-CONG
      03    NOME-CONG
      03    DATA-INSC-CONG
      03    NO-DE-GTS
      03    NO-DE-INSCR
      03    GT OCCURS 1 TO 3 TIMES
      DEPENDING ON NO-DE-GTS.
          05    COD-GT
          05    NOME-GT
      03    INSCRITO OCCURS 0 TO 200 TIMES
          DEPENDING ON NO-DE-INSCR.
          05    COD-INSCR
          05    NOME-INSCR
          05    PAÍS-INSCR

```

Figura 6.22 Arquivo COBOL de congressos.

Execute a normalização do documento, mostrando cada uma das formas normais.

exercício 9 Realize a integração dos modelos referentes aos cinco documentos e arquivos que você normalizou nos exercícios precedentes.

exercício 10 Identifique as chaves estrangeiras no modelo resultante da integração feita no exercício anterior.

exercício 11 Construa um diagrama ER a partir do modelo relacional obtido no exercício anterior. Utilize o processo de engenharia reversa descrito na Seção Engenharia reversa de modelos relacionais (página 169 em diante).

exercício 12 Identifique as diferenças do modelo ER construído no exercício anterior com o modelo ER construído no Exercício 11 do capítulo 3. Comente a causa das diferenças.

exercício 13 (Refere-se ao sistema de controle de almoxarifado descrito no Exercício 12 do capítulo 3.) A Figura 6.23 apresenta uma lista de estoque. Esta lista está organizada por corredor e apresenta, para cada receptáculo, qual a peça armazenada (código e descrição), juntamente com a quantidade de unidades da peça estocadas no receptáculo.

Execute a normalização do documento, mostrando cada uma das formas normais.

Lista de estoque em dd/mm/aa**Corredor:** (---NoCorredor---)

Receptáculo	Código da Peca	Descricao	Quantidade
(---NoRecptac)	(---CodPeca---)	(---DescricaoPeca---)	(---Quant---)
(---NoRecptac)	(---CodPeca---)	(---DescricaoPeca---)	(---Quant---)

...

Lista de estoque em dd/mm/aa**Corredor:** (---NoCorredor---)

Receptáculo	Código da Peca	Descricao	Quantidade
(---NoRecptac)	(---CodPeca---)	(---DescricaoPeca---)	(---Quant---)
(---NoRecptac)	(---CodPeca---)	(---DescricaoPeca---)	(---Quant---)

...

Figura 6.23 Lista de estoque por corredor do almoxarifado.

exercício 14 (Refere-se ao sistema de controle de almoxarifado descrito no Exercício 12 do capítulo 3) A Figura 6.24 apresenta a estrutura de um arquivo COBOL que armazena as listas de distribuição. Este arquivo registra, para cada entrega, identificada pelo código da ordem de compra (COD-OC) e pela data da entrega, os seguintes dados: o número do estrado que será usado na distribuição, o código da empilhadeira a ser usada, o operador da empilhadeira e uma lista de itens que identificam os corredores a serem visitados durante a distribuição. A lista informa, para cada corredor, quais os receptáculos

```

FD LISTA-DISTRIB.
01 REG-LISTA.
   03 COD-OC
03 DATA-ENTREGA
03 NO-ESTRADO
03 NO-DE-CORREDORES
   03 COD-EMPLHADEIRA
   03 OPER-EMPLHADEIRA
   03 ITEM-CORR OCCURS 1 TO 20 TIMES
       DEPENDING ON NO-DE-CORREDORES.
       05 NO-CORR
       05 NO-DE-RECEPTACULOS
       05 ITEM-RECEPT OCCURS 1 TO 5 TIMES
           DEPENDING ON NO-DE-RECEPTACULOS.
           07 NO-RECEPT
           07 COD-PECA
           07 DESCR-PECA
           07 QTDE

```

Figura 6.24 Lista de distribuição (arquivo COBOL).

que serão visitados e, para cada receptáculo qual o código e a descrição da peça a ser armazenada, juntamente com a quantidade de unidades a serem colocadas no receptáculo.

Execute a normalização do documento, mostrando cada uma das formas normais.

exercício 15 (Refere-se ao sistema de controle de almoxarifado descrito no Exercício 12 do capítulo 3.) A Figura 6.25 mostra um boleto que é emitido para cada pedido feito por um cliente. Este boleto informa ao cliente em que rampa seu pedido será entregue.

Execute a normalização do documento, mostrando cada uma das formas normais.

exercício 16 (Refere-se ao sistema de controle de almoxarifado descrito no Exercício 12 do capítulo 3.) A Figura 6.26 apresenta a estrutura de um documento que lista a situação de atendimento de ordens de compra (OC). Uma OC pode estar parcialmente atendida, isto é, para cada OC podem ocorrer entregas parciais (apenas algumas peças, parte da quantidade).

O documento lista em seu cabeçalho o código da OC, sua data e seu fornecedor (código e nome). Para cada peça encomendada na OC, o documento lista o código, a descrição e a quantidade pedida. A seguir, para cada entrega é listada a data da entrega e a quantidade entregue. A soma da quantidade já entregue é listada a seguir. Observe que certas peças podem não ter tido entregas.

exercício 17 (Refere-se ao sistema de controle de almoxarifado descrito no Exercício 12 do capítulo 3.) A Figura 6.27 apresenta a estrutura de um pedido. No cabeçalho aparece o número de pedido, a data e o cliente que o realizou (código, nome e uma lista com número variado de telefones). Depois são listados, para cada peça pedida, seu código, sua descrição e a quantidade pedida.

Ao Cliente

(---codCliente---) (---nomeCliente---)

Informamos que seu pedido (---noPedido---) será entregue na rampa (---noRampa---)

Figura 6.25 Boleto indicativo de rampa de saída de peças.

Situação de atendimento de Ocs pendentes em dd/mm/aa.

OC número: (---noOC---) Data da OC: (---data---)

Código do Fornecedor: (---codFornec---)

Nome do Fornecedor: (---NomeFornec)

Código Peça (---)	Descrição Peça (---)	Quantidade Pedida (---)	Data Entrega (---data---) (---data---)	Quantidade Entregue (---) (---)
		<i>Total entregue:</i> (---)		
(---)	(---)	(---)	(---data---)	(---)
		<i>Total entregue:</i> (---)		
(---)	(---)	(---)	(---data---)	(---)
			(---data---)	(---)
			(---data---)	(---)
		<i>Total entregue:</i> (---)		
(---)	(---)	(---)		
		<i>Total entregue:</i> 0		

OC número: (---noOC---) Data da OC: (---data---)

Código do Fornecedor: (---codFornec---)

Nome do Fornecedor: (---NomeFornec)

Código Peça (---)	Descrição Peça (---)	Quantidade Pedida (---)	Data Entrega (---data---) (---data---)	Quantidade Entregue (---) (---)
		<i>Total entregue:</i> (---)		

Figura 6.26 Situação de atendimento de ordens de compra.

Pedido

Pedido: (---noPed---) Data do pedido: (---data---)

Código do Cliente: (---codCliente---)

Nome do Cliente: (---nomeCliente)

Telefones p/ contato: (---noTel---) (---noTel---) (---noTel---)

Código Peça (---) (---)	Descrição Peça (---) (---)	Quantidade Pedido (---) (---)
----------------------------------	-------------------------------------	--

Figura 6.27 Pedido.

Lista de Busca

Pedido: (---noPed---) **Data do pedido:** (---data---)

Código do Cliente: (---codClie---)

Nome do Cliente: (---NomeCliente)

Corredor: (---noCorr---)

Número Receptáculo	Código Peça	Descrição Peça	Quantidade a Buscar
(---)	(---)	(---)	(---)
(---)	(---)	(---)	(---)
(---)	(---)	(---)	(---)

Corredor: (---noCorr---)

Número Receptáculo	Código Peça	Descrição Peça	Quantidade a Buscar
(---)	(---)	(---)	(---)
(---)	(---)	(---)	(---)

Figura 6.28 Lista de busca.

Execute a normalização do documento, mostrando cada uma das formas normais.

exercício 18 (Refere-se ao sistema de controle de almoxarifado descrito no Exercício 12 do capítulo 3.) A Figura 6.28 apresenta a estrutura da lista de busca. Para cada pedido é emitida uma lista de busca. O cabeçalho da lista de busca contém o número de pedido, sua data e o seu cliente (código e nome). Após, para cada corredor, são listados o receptáculo visitado, o código da peça a apanhar, sua descrição e a quantidade a buscar.

exercício 19 Realize a integração dos modelos referentes aos documentos e arquivos do sistema de almoxarifado que você normalizou nos exercícios precedentes.

exercício 20 Identifique as chaves estrangeiras no modelo resultante da integração feita no exercício anterior.

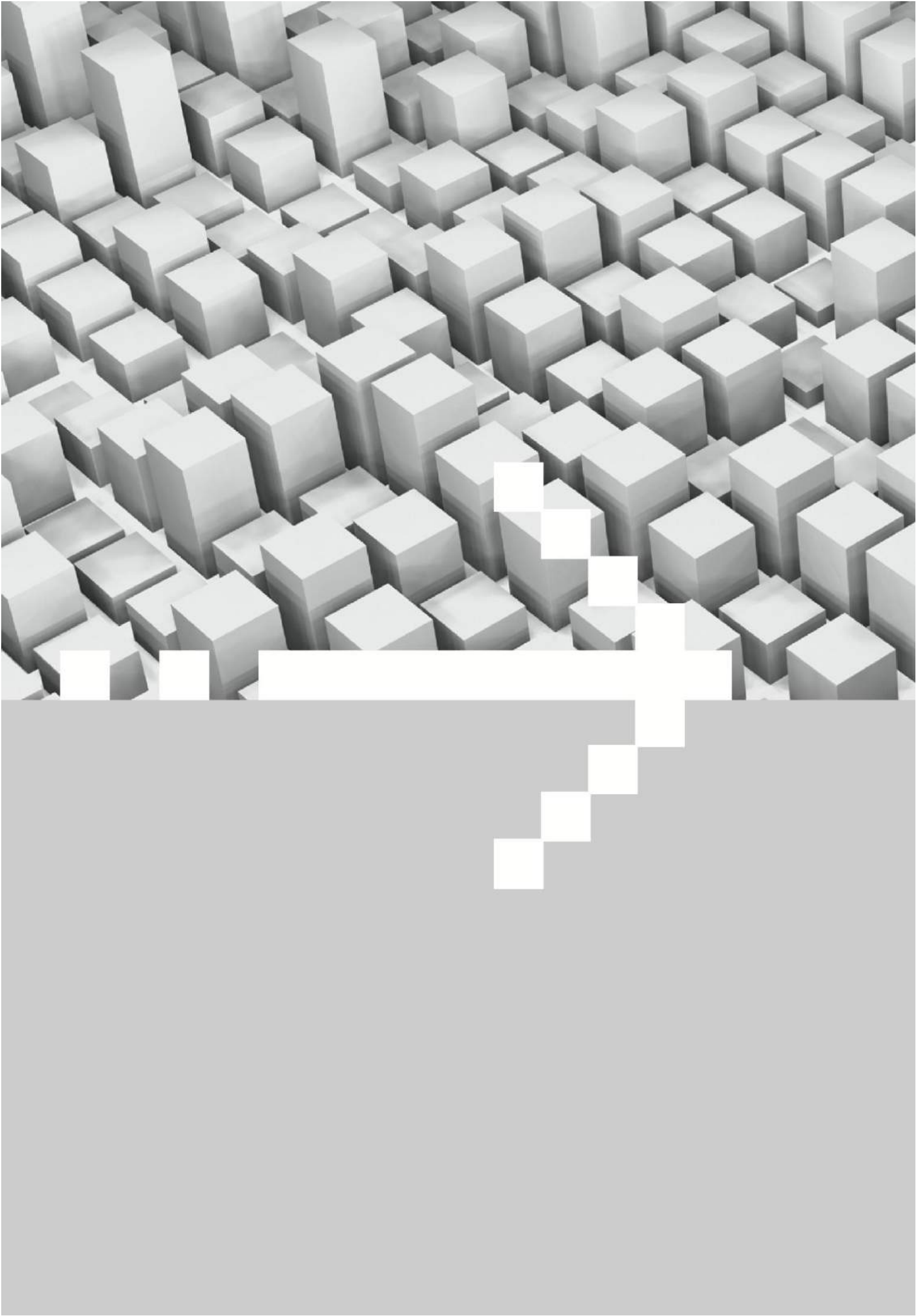
exercício 21 Construa um diagrama ER a partir do modelo relacional obtido no exercício anterior. Utilize o processo de engenharia reversa descrito na Seção Engenharia reversa de modelos relacionais (página 169 em diante).

exercício 22 Identifique as diferenças do modelo ER construído no exercício anterior com o modelo ER construído no Exercício 12 do capítulo 3. Comente a causa das diferenças.

6.9**→ leituras recomendadas**

A idéia de normalização que serve de base a este capítulo surgiu juntamente com a abordagem relacional e foi proposta por Ted Codd, o criador da abordagem relacional. As três primeiras formas normais aparecem nos artigos originais de Codd (Codd (1971) e Codd (1972)). A normalização é apresentada na maioria dos textos introdutórios de banco de dados como os já citados Elmasri; Navathe (2002) e Silberschatz; Korth; Sudarshan (1999).

A engenharia reversa é apresentada no livro de Batini; Ceri; Navathe (1992). Uma boa cobertura do assunto aparece também no livro de Setzer (1987).





capítulo

soluções de exercícios selecionados

■ ■ Este capítulo apresenta as soluções de
exercícios selecionados. Procuramos privilegiar os
problemas mais complicados.

Em muitos exercícios, particularmente nos estudos de caso, não há uma única solução correta. Como as descrições dos sistemas são informais, elas se prestam a diferentes interpretações. Por este motivo, em alguns estudos de caso mais complexos, não é apresentada somente uma solução, mas são discutidas alternativas, tanto corretas quanto incorretas.

■ exercício 2.4

Grande parte do exercício está resolvida na Seção modelos ER têm poder de expressão limitado (página 73). Lá é apresentado um diagrama de ocorrências no qual:

- uma pessoa pode estar casada com ela mesma e
- uma pessoa pode participar de dois casamentos, desde que no papel de marido em um casamento e no papel de esposa em outro.

A Figura 7.1 mostra como é possível excluir essas duas possibilidades do modelo ER. A mudança que foi feita em relação ao modelo original (Figura 2.7) foi a de transformar *CASAMENTO* em entidade e, com isso, abrir a possibilidade de especificar a cardinalidade do relacionamento de casamento com pessoa.

Outra alternativa que poderia ser considerada é a de especializar a entidade *PESSOA* em homens e mulheres. Essa solução somente deve ser considerada caso existam atributos diferentes para homens e mulheres.

■ exercício 2.5

A Figura 7.2 apresenta um possível diagrama de ocorrências para o relacionamento de *SUPERVISÃO*. Este diagrama modela as seguintes instâncias do relacionamento:

- e1 é supervisor de e2
- e1 é supervisor de e3
- e3 é supervisor de e4
- e3 é supervisor de e5

Na Figura 3.2 (página 75), é mostrado outro exemplo de diagrama de ocorrências, contendo uma situação que, apesar de permitida pelo diagrama, não corresponde à nossa intenção ao modelar o relacionamento de *SUPERVISÃO*.



Figura 7.1 Implementando restrições no casamento.

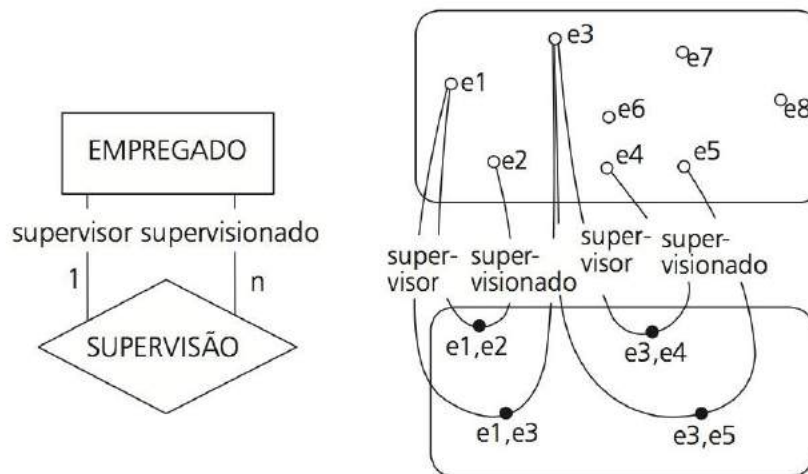


Figura 7.2 Diagrama de ocorrências para o relacionamento *SUPERVISÃO*.

■ exercício 2.7

A Figura 7.3 mostra o resultado da transformação do modelo ER da Figura 2.11 em um modelo que não contém relacionamentos ternários.

Observe que uma ocorrência da entidade *DISTRIBUIÇÃO* é identificada pelas ocorrências de *CIDADE* e de *PRODUTO* aos quais está relacionada. Esta é a tradução para a cardinalidade do relacionamento *DISTRIBUIÇÃO*, que aparece no modelo entidade-relacionamento original. Esta cardinalidade especifica que, para cada par formado por um produto e uma cidade, pode haver somente um distribuidor.

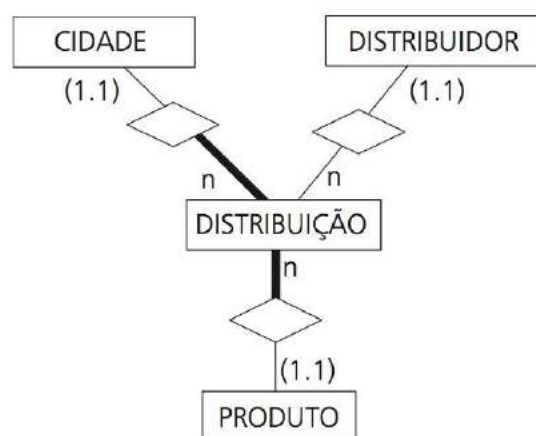


Figura 7.3 Transformação de relacionamento ternário em entidade.

Uma solução que, à primeira vista, pode parecer equivalente ao relacionamento ternário é a apresentada na Figura 7.4. Esta solução não é equivalente à original. O modelo representando na nova solução contém menos informações que o modelo original. O leitor poderá certificar-se deste fato se construir e comparar exemplos com ocorrências para o modelo original e para o modelo da Figura 7.4.

■ exercício 2.11

A transformação do relacionamento *ATUAÇÃO* da Figura 2.16 em entidade resulta no modelo ER da Figura 7.5.

Observe que uma ocorrência de *ATUAÇÃO* é identificada pelos relacionamentos com as entidades *PROJETO* e *ENGENHEIRO*.

■ exercício 2.12

A Figura 7.6 apresenta um modelo ER que resulta da modificação do modelo da Figura 2.20. A modificação consta em possibilitar que um dependente seja empregado. Caso se mantivesse o modelo original, o nome do dependente seria armazenado redundantemente. A solução adotada foi a de especializar

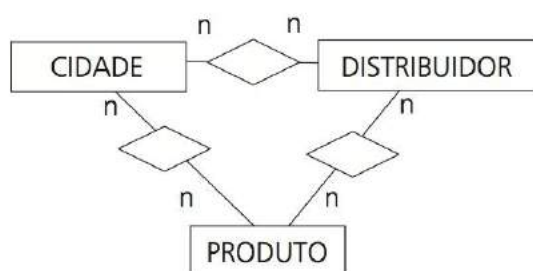


Figura 7.4 Tentativa de transformação de relacionamento ternário.

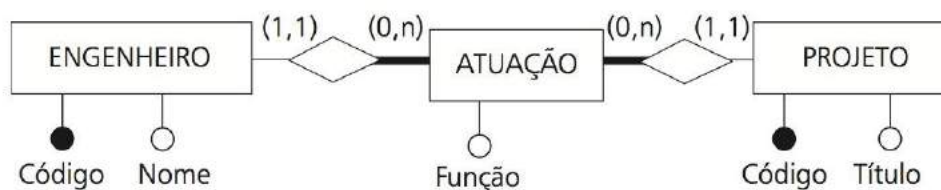


Figura 7.5 Resultado da transformação de um relacionamento em entidade.

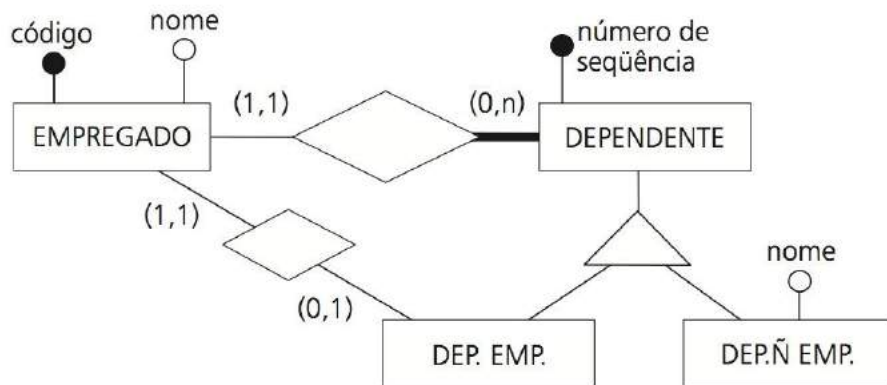


Figura 7.6 Dependente pode ser empregado.

a entidade *DEPENDENTE* em duas, *DEP. Ñ EMP.*, que contém os atributos dos dependentes que não são empregados e *DEP.EMP.*, que não contém atributos mas está relacionada à entidade empregado correspondente.

■ exercício 2.17

A solução encontra-se no modelo ER da Figura 7.7. Caso não sejam usados atributos multivalorados, é necessário criar uma entidade para cada atributo multivalorado. Observe o identificador desta entidade. Um telefone é identificado pelo seu número e pelo cliente correspondente. Isso permite que diferentes clientes tenham o mesmo número de telefone (o que era permitido pelo modelo original).

■ exercício 2.18

A solução para modelar uma especialização compartilhada é usar relacionamentos para ligar as entidades especializadas à entidade genérica (Figura 7.8).

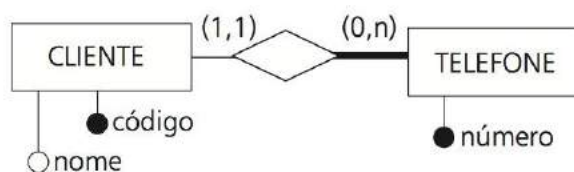


Figura 7.7 Transformação de atributo multivalorado em entidade.

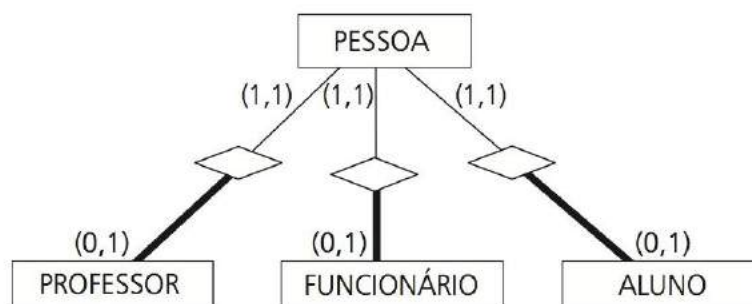


Figura 7.8 Tratando especialização compartilhada através de relacionamentos.

Cada entidade especializada é identificada pela entidade genérica a qual está relacionada. Lembre que na generalização/especialização as entidades especializadas herdam o identificador da entidade genérica.

Observe as cardinalidades dos relacionamentos. Cada ocorrência de uma entidade especializada (PROFESSOR, FUNCIONÁRIO e ALUNO) deve estar relacionada a exatamente uma ocorrência da entidade genérica (PESSOA). Já uma ocorrência da entidade genérica (PESSOA) pode estar relacionada a, no máximo, uma ocorrência de cada uma das entidades especializadas.

Observe que, como trata-se de um relacionamento compartilhado, uma ocorrência da entidade genérica pode, inclusive, estar relacionada a mais de uma ocorrência de entidade especializada, desde que cada ocorrência pertença a uma entidade especializada diferente. Exemplificando, uma ocorrência de PESSOA pode estar relacionada tanto a uma ocorrência de FUNCIONÁRIO quanto a uma ocorrência de ALUNO.

Recorde que, de forma geral, generalizações/especializações podem ser classificadas nos seguintes tipos:

- exclusiva ou compartilhada e
- total ou parcial.

O fato de ser uma generalização/especialização *exclusiva* limita uma ocorrência da entidade genérica a aparecer em somente uma das entidades especializadas. Observe que esta restrição não pode ser transposta ao modelo ER, quando a generalização/especialização é transformada em relacionamentos.

O fato de ser uma generalização/especialização *total* exige que uma ocorrência da entidade genérica apareça em pelo menos uma das entidades especializadas. Também esta restrição não pode ser transposta ao modelo ER, quando a generalização/especialização é transformada em relacionamentos.

■ exercício 2.24

Não é possível expressar esta restrição, pelo fato de o modelo ER não possuir uma notação que mostre que a união de dois relacionamentos (no caso, o de *VENDA* com *MEDICAMENTO* e o de *VENDA* com *PERFUMARIA*) tem cardinalidade mínima 1. Esta restrição teria que ser especificada fora do modelo ER.

■ exercício 2.30

O modelo ER expressa que um processador de textos não pode existir no banco de dados sem que exista uma secretária que o domine (cardinalidade mínima da entidade *PROCESSADOR DE TEXTOS* no relacionamento *DOMÍNIO*). Assim, cada vez que uma secretária for excluída, é necessário verificar cada processador de textos por ela dominado. Caso ela seja a última a dominar determinado processador de textos, a secretária não poderá ser excluída, ou, alternativamente, a exclusão da secretária deverá ser propagada à exclusão do processador de textos em questão.

■ exercício 2.31

A especialização de *EMPREGADO* é *exclusiva*, isto é, uma ocorrência da entidade genérica não pode aparecer em mais de uma de suas especializações. Como as entidades *SECRETÁRIA*, *ENGENHEIRO* e *GERENTE* são especializações de *EMPREGADO* em uma mesma hierarquia de generalização/especialização, um empregado não pode aparecer em mais de uma delas.

Para permitir que uma secretária ou um engenheiro sejam gerentes, é necessário transformar a generalização/especialização em *compartilhada*.

Alternativamente, também é possível retirar a entidade *GERENTE* da hierarquia de generalização/especialização na qual aparece com as entidades *SECRETÁRIA* e *ENGENHEIRO*. Neste caso, *GERENTE* passa a ser um auto-relacionamento de *EMPREGADO*.

■ exercício 3.1

- 1 O relacionamento apresentado não inclui a restrição de que um produto não pode ser composto por ele mesmo.
- 2 A Figura 7.9 apresenta um modelo ER que exclui a possibilidade de um produto ser composto por ele mesmo. Cabe observar que esta solução somente deve ser adotada caso existam atributos específicos das várias especializações de *PRODUTO*. Caso não existam atributos específicos para estas especializações, a melhor alternativa seria manter o modelo original, expressando a restrição de integridade em questão fora do modelo.
- 3 Caso deseje-se armazenar no banco de dados uma hierarquia com número não limitado de níveis de composição, fica impossível registrar no modelo o fato de um produto não poder aparecer em sua composição. Essa restrição exige o uso de recursividade em sua expressão, o que não é possível em modelos ER.

■ exercício 3.3

No modelo, há atributos específicos de pessoa física, outros específicos de pessoa jurídica e alguns comuns a ambos os tipos de clientes. Este é o caso típico para a aplicação do conceito de generalização/especialização. A Figura 7.10 apresenta um modelo para a mesma realidade usando este conceito.

O novo modelo é mais preciso que o original. No original, os atributos das entidades especializadas apareciam como opcionais. O modelo não informava quais atributos pertencem a qual tipo de cliente e se o atributo é ou não obrigatório para um tipo de cliente.

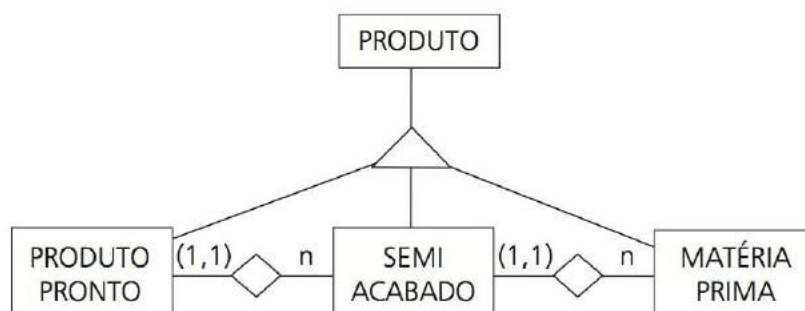


Figura 7.9 Estrutura de produto com número fixo de níveis.

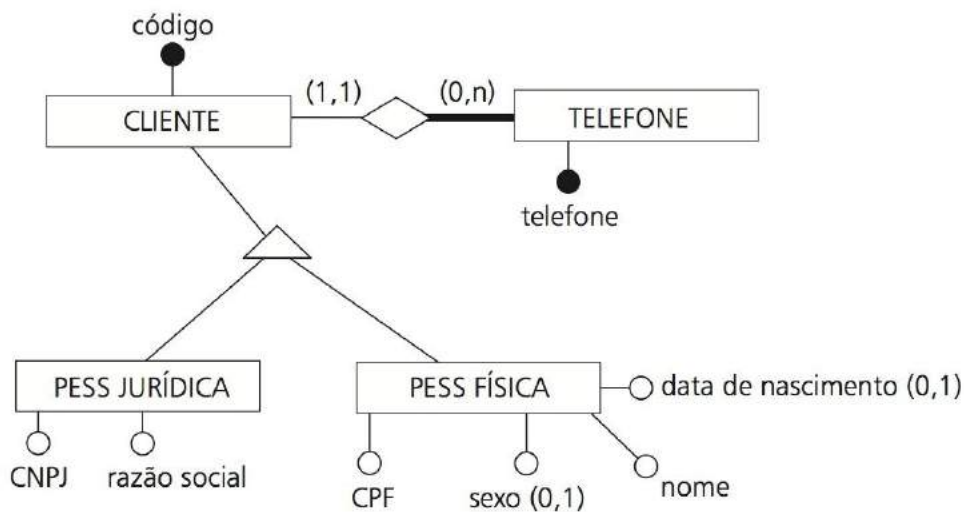


Figura 7.10 Substituindo atributos opcionais por especializações.

No novo modelo, fica claro que atributos pertencem a cada uma das especializações e se eles são ou não obrigatórios.

■ exercício 3.5

O modelo ER apresentado na Figura 2.11 através da notação deste livro é apresentado na Figura 7.11 através da notação européia.

A característica básica desta notação é a interpretação da cardinalidade de relacionamento. Nela, a cardinalidade informa quantas vezes uma ocorrência de uma entidade participa do relacionamento. Assim, a cardinalidade **n** junto

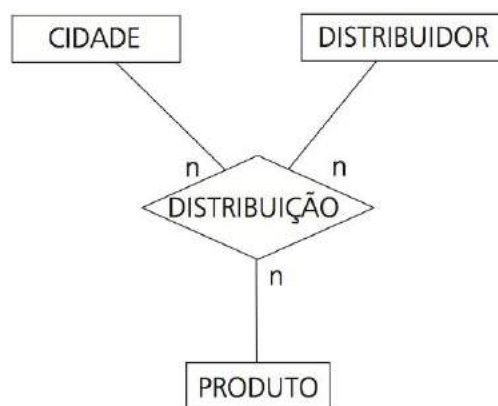


Figura 7.11 Modelo da Figura 2.11 na notação européia.

à entidade *DISTRIBUIDOR* informa que uma ocorrência desta entidade pode participar várias vezes do relacionamento (um distribuidor pode ter várias distribuições).

É importante observar que, neste diagrama, não está expressa a restrição de que um distribuidor tem exclusividade de distribuição para um determinado produto, em uma determinada cidade, conforme especificado no modelo ER original, na Figura 2.11.

Assim, observa-se que, para relacionamentos com grau maior que dois, não há equivalência entre as notações de cardinalidades, a notação européia e a notação usada neste livro.

■ exercício 3.6

A alteração do modelo mostrado na Figura 3.10 para permitir que cada servidor (não cada cargo de servidor) tenha associado seu CPF e seu número de inscrição no PASEP é mostrada na Figura 7.12.

Neste novo modelo, temos informações que são específicas de uma pessoa que é servidor. Estas informações não devem aparecer na entidade *SERVIDOR CARGO*. Como um servidor pode ter vários cargos, as informações ficariam redundantes para cada cargo do servidor. Por outro lado, colocar as informações na própria entidade *PESSOA* resulta em atributos opcionais. Assim, a solução mais adequada é criar uma nova especialização da entidade *PESSOA*, a entidade *SERVIDOR*, que contém as informações específicas daquela pessoa que é servidor. As informações relativas a cada cargo de um servidor permanecem na entidade *SERVIDOR CARGO*.

■ exercício 3.7 – administradora de imóveis

Para o estudo de caso da administradora de imóveis, vamos seguir os passos da estratégia *descendente* de modelagem descritos na Seção Partindo do conhecimento das pessoas (página 103).

Enumeração de entidades. Uma primeira leitura do enunciado pode resultar nas seguintes entidades: *ADMINISTRADORA*, *CONDOMÍNIO*, *UNIDADE*, *PESSOA*

Identificação de relacionamentos. Os relacionamentos que podem ser identificados são os seguintes:

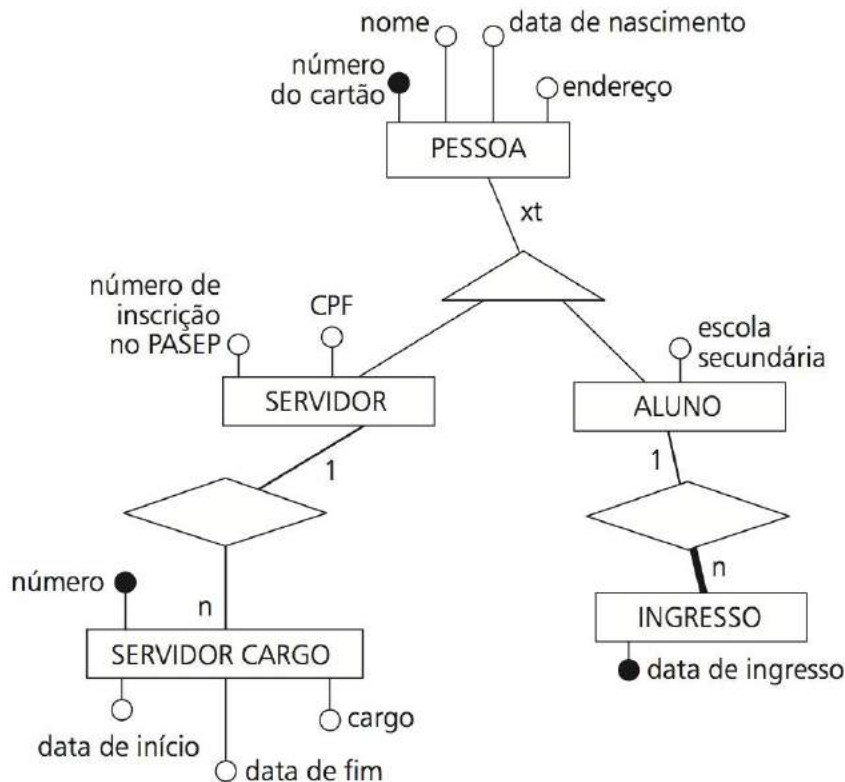


Figura 7.12 Alteração do modelo da Figura 3.10: servidor tem informações associadas.

- *COMPOSIÇÃO* entre *CONDOMÍNIO* e *UNIDADE*
- *PROPRIEDADE* entre *UNIDADE* e *PESSOA*
- *ALUGUEL* entre *UNIDADE* e *PESSOA*

Um relacionamento que poderia parecer necessário é o que liga *ADMINISTRADORA* com *CONDOMÍNIO*. Caso o BD que esteja sendo modelado seja destinado a apenas uma administradora (como no exemplo), este relacionamento não deve aparecer no modelo. Como a entidade *ADMINISTRADORA* possui uma única instância, não é necessário manter no BD a informação de qual administradora administra qual condomínio. Este relacionamento somente deveria aparecer caso várias administradoras pudessem coexistir no mesmo BD (ver discussão na Seção Entidade isolada e entidade sem atributos na página 94.)

Identificação de cardinalidades máximas As cardinalidades identificadas aparecem no DER da Figura 7.13.

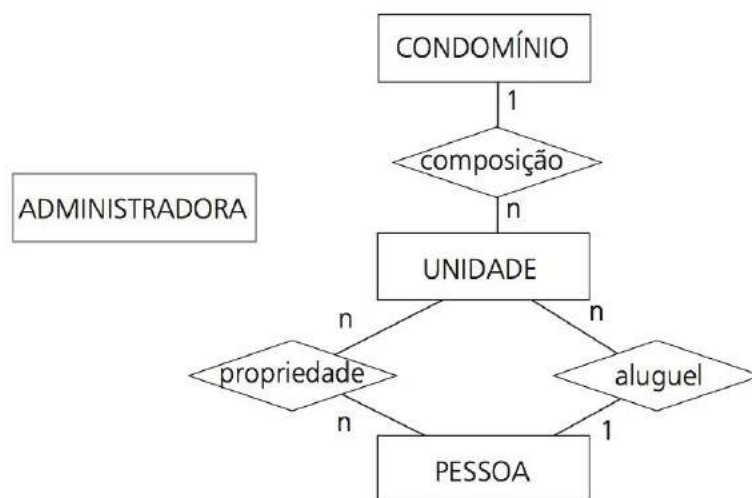


Figura 7.13 Diagrama ER para a administradora de imóveis.

■ exercício 3.8 – locadora de mídias antigas

Seguindo os passos do processo de modelagem, chegamos aos seguintes resultados.

Enumeração de entidades As entidades identificadas são: *LOCADORA*, *FILME*, *FITA*, *CLIENTE*, *CATEGORIA* e *ATOR*.

Uma alternativa de modelagem seria considerar a categoria de um filme como atributo de *FILME*. Como consideramos que o conjunto de categorias de filmes não é imutável e pode variar ao longo da vida do BD, preferimos modelar a categoria como uma entidade (ver discussão na Seção Atributo *versus* entidade relacionada, na página 78).

Outra decisão é a de como modelar o empréstimo, se através de uma entidade ou um relacionamento. Optamos por modelá-lo como relacionamento.

Identificação de relacionamentos

- entre *FILME* e *FITA*
- *EMPRÉSTIMO* entre *FITA* e *CLIENTE*
- entre *FILME* e *CATEGORIA*
- *ESTRELA* entre *ATOR* e *FILME*

Como no estudo de caso precedente, a entidade *LOCADORA* está isolada por possuir uma única instância.

As cardinalidades máximas são simples de obter a partir do enunciado e aparecem no diagrama ER da Figura 7.14.

Determinação de atributos Um primeiro levantamento de atributos, com base na leitura do enunciado, resulta nos atributos que aparecem no DER da Figura 7.14. O único atributo que não aparece explicitamente no enunciado é o número do rolo (atributo de fita). Este atributo é necessário para filmes multirolo, para identificar que rolo do filme está armazenado na fita.

Determinação de identificadores Cada entidade do modelo deve ter seus identificadores (atributos e/ou relacionamentos) definidos. Alguns identificadores aparecem explicitamente no enunciado do problema:

- Cada fita é identificada por seu número.
- Cada filme possui um identificador.
- Cada cliente é identificado por seu número.

Para as demais entidades é necessário criar um identificador. Nomes ou outros atributos que ocupem muito espaço de armazenamento não são recomendados, caso se tenha em vista uma implementação em SGBD relacional, já que eles resultam em estruturas internas de acesso pouco eficientes. Por este motivo, é necessário criar atributos identificadores para as entidades *CATEGORIA* e *ATOR*.

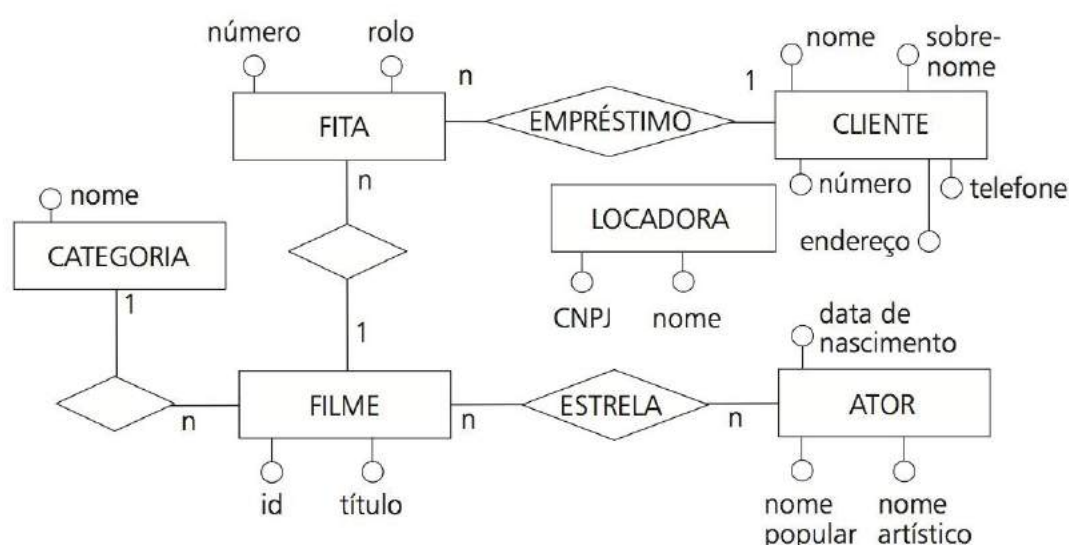


Figura 7.14 Diagrama ER para locadora de mídias antigas (primeira versão).

Verificação de aspectos temporais Não há atributos nem relacionamentos dos quais deva ser mantida história. Não há alterações a fazer no modelo, no que tange a aspectos temporais.

Domínios dos atributos Nesta etapa devem ser definidos os domínios dos atributos. Isso normalmente é feito já com o uso de uma ferramenta CASE. Assim, nos estudos de caso, deixaremos de lado esta etapa.

Definição de cardinalidades mínimas Na Figura 7.15 estão apresentadas as cardinalidades mínimas que podem ser deduzidas do enunciado do problema

■ exercício 3.9 – sistema de reserva de passagens aéreas

Enumeração de entidades A leitura do enunciado nos leva às seguintes entidades: *COMPANHIA*, *RESERVA*, *PASSAGEIRO*, *TRECHO*, *VÔO*, *CIDADE*, *AEROPORTO*, *TIPO AERONAVE*, *HORÁRIO* e *ASSENTO*.

Não foi criada uma entidade correspondente às pessoas que efetivaram a reserva. Não serão armazenadas informações sobre estas pessoas. Decidiu-se não cadastrar pessoas de forma central, pois esta operação demandaria tempo e seriam necessárias informações adicionais sobre a pessoa para resolver o problema de homônimos. Por isso, esta informação será modelada como um atributo da reserva.

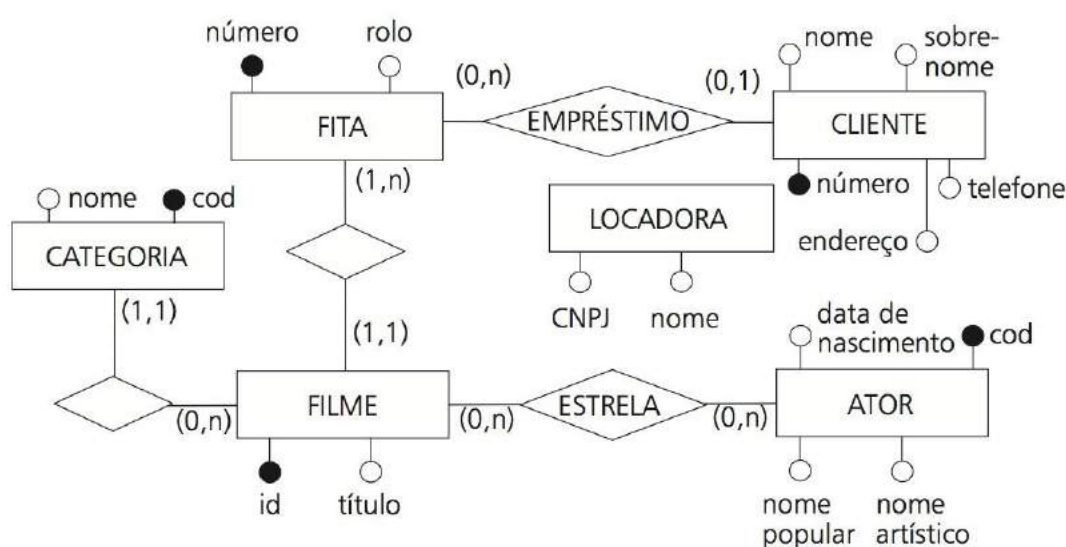


Figura 7.15 Diagrama ER para locadora de mídias antigas (final).

Identificação de relacionamentos Os relacionamentos identificados aparecem no diagrama ER da Figura 7.16. O relacionamento RSRV-TRCH associa a reserva aos vários trechos de vôo que a compõem. Ele representa cada trecho de vôo reservado.

Este relacionamento está sendo tratado como uma entidade associativa, pois, para marcar o assento, é necessário relacionar cada trecho da reserva com o assento reservado.

Determinação de atributos e identificadores Para não sobrecarregar o diagrama, listamos os atributos das entidades. Os atributos identificadores estão sublinhados e os relacionamentos identificadores aparecem no diagrama ER da Figura 7.16.

RESERVA (codigo reserva, passageiro, prazo)

VOO (número)

TRECHO ()

AEROPORTO (código, nome)

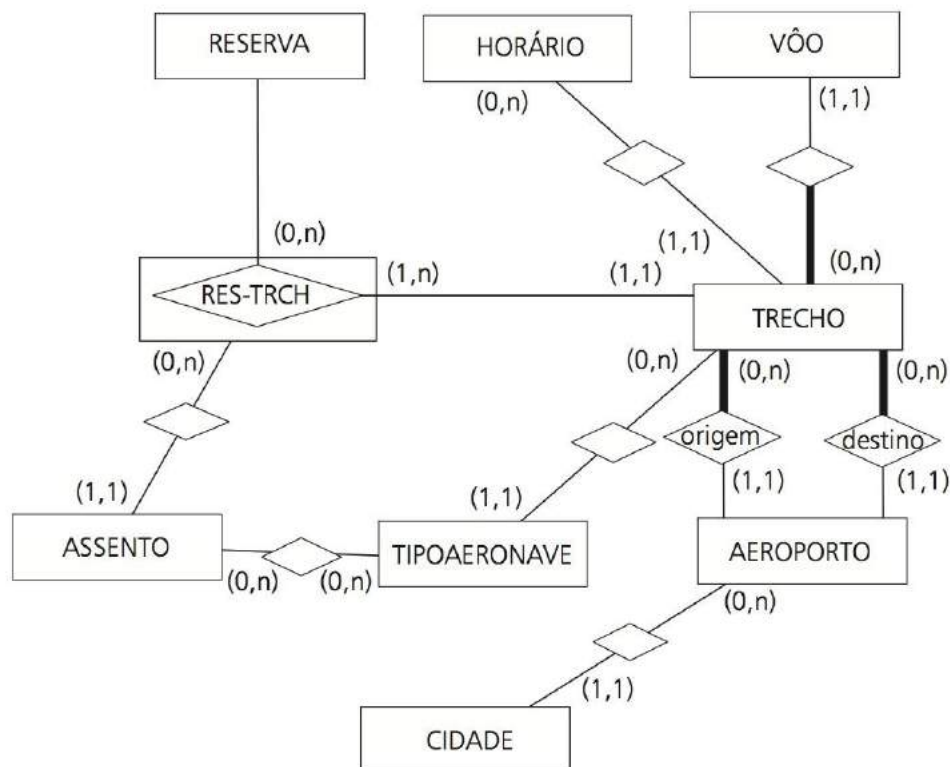


Figura 7.16 Diagrama ER para sistema de reserva de passagens aéreas.

CIDADE (código, nome, país)
 TIPO AERONAVE (código, descrição)
 HORARIO (dia semana, horário partida, horário chegada)
 ASSENTO (número, classe)
 RSRV-TRCH (data)

Restrições que não podem ser representadas no modelo Neste sistema aparecem diversas restrições que não podem ser representadas em modelos ER:

- Uma reserva de trecho somente pode ser realizada caso existam vagas no trecho em questão, na data em questão.
- Uma reserva para um assento somente pode ser feita se o assento em questão existir no tipo de aeronave utilizada no trecho de voo em questão.

Uma observação geral sobre a solução adotada é que ela é conceitual e procura apenas fixar as necessidades de informação do sistema. O modelo não inclui redundâncias de dados que objetivem melhorar a performance de determinadas operações executadas freqüentemente. Estas considerações devem ficar para uma fase posterior do projeto do BD. Neste caso, atributos redundantes, como o número de vagas em um trecho de voo, em uma data, inclusive discriminado por classe, poderia ser necessário. Isso levaria à necessidade de criação de uma entidade *TRECHO-DIA* correspondendo a cada viagem específica de um trecho de voo em uma data. Para uma discussão sobre a inclusão de redundância no banco de dados projetado, ver a Seção Refinamento do modelo relacional na página 162.

■ exercício 3.10 – sistema para locadora de veículos

Enumeração de entidades Uma primeira leitura do enunciado nos leva às seguintes entidades: *LOCADORA*, *TIPO AUTOM OU CAMIONETA PASS*, *TIPO CAMIONETA CARGA*, *VEÍCULO*, *PESSOA FÍSICA*, *PESSOA JURÍDICA* e *FILIAL*. Além destas, são necessárias as entidades *RESERVA* e *LOCAÇÃO* para manter informações sobre as duas transações centrais da locadora.

Há vários atributos e relacionamentos comuns às entidades *TIPO AUTOM OU CAMIONETA PASS* e *TIPO CAMIONETA CARGA*. Por este motivo, é usada uma generalização das três entidades (*TIPO VEÍCULO*).

Um raciocínio análogo pode ser aplicado às entidades *PESSOA FÍSICA* e *PESSOA JURÍDICA*, levando à generalização *CLIENTE*.

A entidade *REVISÃO* é usada para manter as informações sobre as revisões que devem ser feitas em veículos de um tipo. Essa informação é multivalorada (há várias revisões para um tipo de veículo) e, por isso, não pode ser armazenada em um atributo de *TIPO VEÍCULO*. A entidade *MOTORISTA* destina-se a armazenar informações sobre a habilitação do motorista que está dirigindo o veículo. Estas informações não foram colocadas em *CLIENTE*, já que um cliente pessoa jurídica pode ter diferentes motoristas cadastrados.

Identificação de relacionamentos Os relacionamentos identificados aparecem no diagrama ER da Figura 7.17.

Entre as entidades *RESERVA* e *FILIAL*, são usados dois relacionamentos, um para representar a filial onde o veículo será retirado (origem) e outro para representar a filial onde o veículo será devolvido (destino). A *LOCAÇÃO* está opcionalmente ligada a uma filial de destino. Este relacionamento serve para informar em que filial o veículo será devolvido, caso seja devolvido em uma filial diferente daquela em que foi retirado.

O relacionamento entre *MOTORISTA* e *CLIENTE* serve para informar quais são os motoristas cadastrados por um cliente.

O relacionamento entre *MOTORISTA* e *LOCAÇÃO* serve para informar o motorista responsável pelo automóvel. Observe que não há um relacionamento

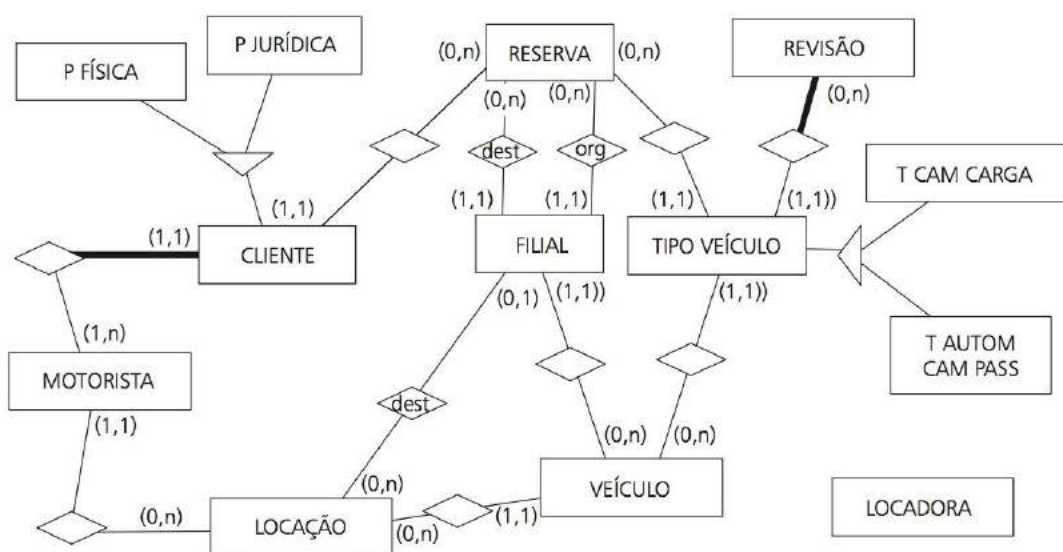


Figura 7.17 Diagrama ER para sistema de locadora de veículos.

direto entre *LOCAÇÃO* e *CLIENTE*, já que este seria redundante (para cada locação, tem-se seu motorista e, para cada motorista, tem-se o cliente correspondente).

Determinação de atributos e identificadores Os atributos das entidades são os seguintes:

CLIENTE (código, nome, endereço)
P FÍSICA (sexo, data nascimento, CPF)
P JURÍDICA (CNPJ, inscrição estadual)
FILIAL (código, localização)
VEÍCULO (placas, número chassi, número motor, cor, quilometragem, data medida quilometragem, quilometragem, última revisão)
TIPO VEÍCULO (código, tipo, horas limpeza, quilometragem, média diária)
T AUTOM CAM PASS (tamanho, número passageiros, ar-condicionado, rádio, MP3, CD, direção hidráulica, câmbio automático)
T CAM CARGA (capacidade carga)
REVISÃO (quilometragem)
MOTORISTA (número habilitação, data vencimento, identidade, nome)
RESERVA (número, data retirada, data devolução)
LOCAÇÃO (número, data retirada, data devolução)
LOCADORA (CNPJ, nome, endereço, telefone)

Os atributos identificadores estão sublinhados. Os relacionamentos identificadores estão representados no diagrama ER da Figura 7.17.

Restrições que não podem ser representadas no modelo As restrições que não estão apresentadas no modelo acima são:

- A habilitação de um motorista não pode vencer durante o período previsto para a locação.
- Um veículo cuja quilometragem exceda a quilometragem de sua próxima revisão não pode ser locado.
- Para um cliente pessoa física, somente deve haver um motorista cadastrado. Neste caso, não deve ser informado o nome do motorista, já que ele é a própria pessoa física.
- Somente pode ser feita uma reserva quando houver previsão de disponibilidade de veículos do tipo, na filial de origem, na data da reserva.
- Uma locação para a qual não tenha sido feita reserva somente pode ocorrer na mesma condição acima.

Assim como na solução do sistema de reservas de passagens aéreas (Exercício 3.9), a presente solução é puramente conceitual, não incluindo dados redundantes que porventura melhorem o desempenho de determinadas operações sobre o banco de dados.

■ exercício 3.11 – sistema de preparação de congressos da IFIP

Uma primeira leitura do exemplo pode nos levar a uma série de entidades modelando papéis de pessoas: autor, moderador, inscrito, membro de GT, etc. Além disso, um dos objetivos do sistema, que aparece implícito no enunciado, é o de criar um cadastro central de pessoas. Isso fica claro quando o enunciado exige que a pessoa não deva aparecer várias vezes em certos relatórios ou que é necessário identificar os vários papéis que uma pessoa cumpriu no passado. Assim, fica evidente que é necessária uma entidade *PESSOA*. Uma solução que poderia ser tentada é a de usar o conceito de generalização/especialização, na forma apresentada na Figura 7.18.

Esta solução não é adequada para o problema. Não há propriedades específicas de uma pessoa em cada um dos papéis (autor, inscrito, membro de GT, etc.). Mesmo que houvessem propriedades (por exemplo, a data de inscrição de uma pessoa em um congresso), estas propriedades seriam do relacionamento entre a pessoa e o congresso.

Por esse motivo, optamos por modelar os papéis que as pessoas cumprem através de relacionamentos, conforme apresentado no diagrama da Figura 7.19.

Os atributos identificados são os seguintes:

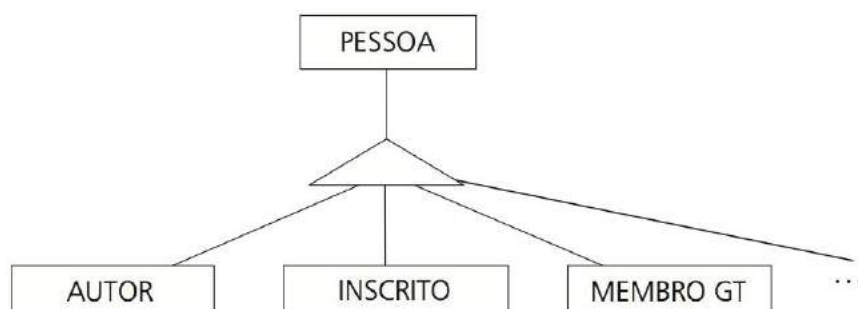


Figura 7.18 Usando generalização/especialização para modelar papéis de pessoas (incorreto).

```

CONGRESSO(código, nome, data realização, local,
           prazo submissão artigos, prazo inscrição)
GT(código, nome)
PESSOA(código, nome, instituição, endereço, telefone, fax, e-mail)
ARTIGO(número, título, aceito/rejeitado)
SESSÃO(código, data, hora, título)
MEMBRO-GT(cargo)
AVALIAÇÃO(parecer, estado avaliação)

```

O atributo estado *avaliação* serve para informar se uma artigo já foi distribuído a um avaliador ou se já retornou do avaliador e recebeu parecer.

Observe que não há um relacionamento *CONVIDADO*. Do ponto de vista conceitual, este relacionamento é desnecessário. As pessoas que são convidadas a um congresso já se encontram no modelo (são os autores de trabalhos submetidos ao congresso, os membros do CO e do CP do congresso, e assim por diante). Assim, este relacionamento é redundante diante das informações já existentes e não deve aparecer no modelo conceitual. Observe que não estamos afirmando que, mais tarde, durante o projeto lógico, este relacionamento não deva ser criado, para atender requisitos de desempenho da aplicação.

Pelas regras de equivalência de modelos vistas na Seção Diferentes modelos podem ser equivalentes, é possível transformar em entidades os diversos relacionamentos **n:n** que aparecem no modelo da Figura 7.19. Nessa variante de solução, os papéis de pessoa, como *MODERADOR*, *AVALIADOR*, *AUTOR* e outros, são transformados em entidades. A Figura 7.20 apresenta, de forma parcial, um esboço de como seria o modelo nesta alternativa.

Observe que as entidades originárias dos relacionamentos **n:n** não são especializações de *PESSOA*, como na Figura 7.18, mas sim entidades relacionadas à entidade *PESSOA*. Além disso, os identificadores das entidades que representam os papéis de pessoa são diferentes do identificador de *PESSOA*. Por exemplo, uma ocorrência de *MEMBRO GT* é identificada pela pessoa que é membro, bem como pelo GT da qual ela participa, enquanto uma ocorrência de *PESSOA* é identificada pelo seu código.

- Finalmente, cabe salientar que há uma série de restrições de integridade que não estão expressas no modelo ER. A seguir, listamos estas restrições:

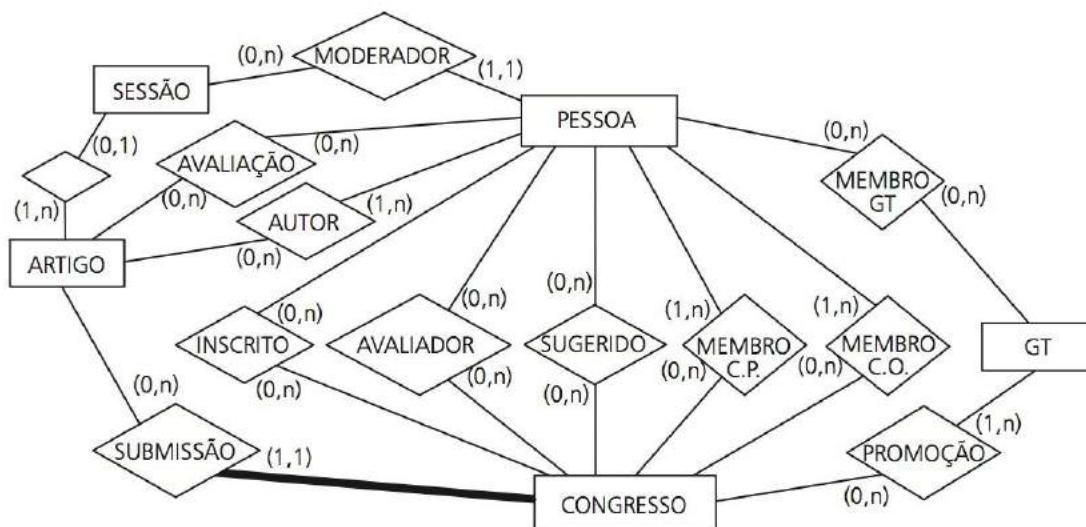


Figura 7.19 Diagrama ER para o sistema de preparação de congressos.

- Um avaliador de um artigo deve ser avaliador cadastrado no congresso.
- Um avaliador de um artigo não deve ser autor deste artigo.
- Um artigo que tenha sido julgado deve ter pelo menos três avaliações.
- Um inscrito deve ter sido convidado para o congresso (ver enunciado para verificar quem é convidado).
- Somente pode aparecer em uma sessão um artigo aceito.

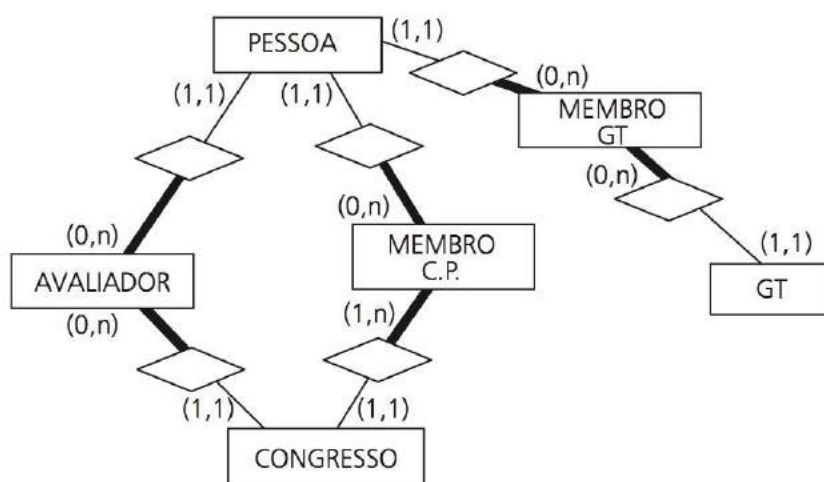


Figura 7.20 Diagrama ER para o sistema de preparação de congressos (parcial – usando entidades em vez de relacionamentos $n:n$).

■ exercício 3.12 – sistema de almoxarifado

A Figura 7.21 apresenta o diagrama ER para o sistema de almoxarifado. A solução apresentada modela os itens de uma ordem de compra, de um pedido, de uma ordem de compra, de uma entrega, etc. como entidade e não como relacionamento **n:n** (entre ordem de compra, pedido, ...e peça). As duas soluções são equivalentes. Aqui, preferimos usar a primeira, apenas para demonstrar a sua utilização, diferentemente dos modelos das questões anteriores, onde usamos relacionamentos **n:n**.

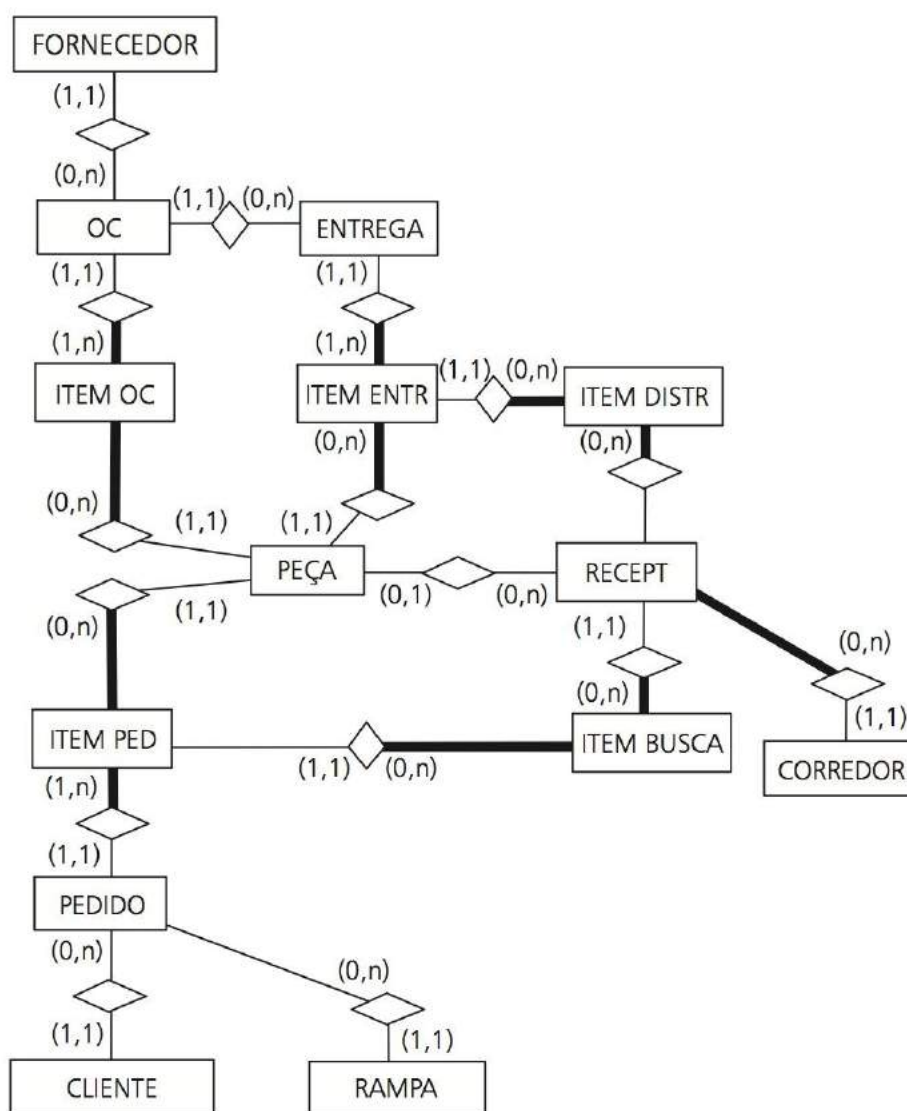


Figura 7.21 Diagrama ER para o sistema de almoxarifado.

■ exercício 4.1

Em um banco de dados relacional, a chave primária de uma tabela é aquela chave única utilizada como chave estrangeira em tabelas que referenciam a chave em questão. No caso do exercício, `CodigoEmpregado` é a chave primária da tabela `Empregado`, já que ela é usada na tabela `Dependente` como chave estrangeira que referencia `Empregado`.

■ exercício 4.2

Em organização de arquivos, o termo *chave secundária* é usado para denotar um campo ou conjunto de campos para os quais existe um índice que admite duplicatas. Conforme mencionado na Seção chave, na abordagem relacional o conceito de chave possui a conotação de *restrição de integridade* e não de caminho de acesso. Por este motivo, o termo chave secundária não é usado na abordagem relacional.

■ exercício 4.3

A Figura 7.22 mostra as chaves primárias e estrangeiras para o modelo do exercício.

■ exercício 4.4

- 1 Uma linha é incluída na tabela `Consulta`.

A tabela `Consulta` contém duas chaves estrangeiras, (`CodigoConvenio`, `NumeroPaciente`) e `CRM`. Quando ocorrer uma in-

```

Aluno (CodigoAluno, Nome, CodigoCurso)
      CodigoCurso referencia Curso
Curso (CodigoCurso, Nome)
Disciplina (CodigoDisciplina, Nome, Creditos, CodigoDepartamento)
          CodigoDepartamento referencia Departamento
Curriculo (CodigoCurso, CodigoDisciplina, Obrigatória-Opcional)
           CodigoCurso referencia Curso
           CodigoDisciplina referencia Disciplina
Conceito (CodigoAluno, CodigoDisciplina, Ano-Semestre, Conceito)
          CodigoAluno referencia Aluno
          CodigoDisciplina referencia Disciplina
Departamento (CodigoDepartamento, Nome)

```

Figura 7.22 Modelo relacional para o Exercício 4.3.

clusão em Consulta, é necessário verificar se estas chaves aparecem nas respectivas tabelas (Paciente e Medico).

2 Uma linha é excluída da tabela Paciente.

A tabela Paciente é referenciada em outra tabela (Consulta) por uma chave estrangeira. Assim, ao realizar a exclusão, é necessário verificar se não mais existem linhas em Consulta que referenciem a linha de Paciente que está sendo excluída.

■ exercício 5.1

A alternativa 1 (Aluno (CodAl, Nome, CodCurso, Endereco)) é preferível, caso considerarmos os princípios nos quais estão baseadas as regras de tradução de modelos ER para modelos relacionais. Os princípios são (página 138):

- Evitar junções – ter os dados necessários a uma consulta em uma única linha
- Diminuir o número de chaves primárias
- Evitar campos opcionais

A alternativa 1 atende aos dois primeiros princípios. O terceiro princípio não é considerado neste caso, já que nenhuma das alternativas implica em criar campos opcionais.

Quanto ao primeiro princípio, a alternativa 1 minimiza a necessidade de junções. Na alternativa 2, cada vez que forem necessários dados do aluno junto com dados de seu endereço será necessário fazer uma junção entre as duas tabelas.

Quanto ao segundo princípio, a alternativa 1 implica em um menor número de chaves primárias, já que implica em uma única tabela.

Conforme mencionado na Seção Transformação ER para relacional, as regras de tradução apresentadas representam um conjunto de heurísticas que, em geral, levam a um banco de dados bem projetado. Podem existir casos em que as regras não devem ser adotadas, principalmente por limitações do SGBD ou considerações de performance. Assim, podem haver situações em que a alternativa 2 é usada, mesmo violando os princípios acima. A seguir consideramos uma situação em que a alternativa 2 poderia ser preferível.

Considere que os dados de aluno (nome e código do curso) e endereço sofrem constantemente alterações de valores. Além disso, considere que o nome e

o código do aluno são alterados através de transações diferentes das que alteram o endereço do aluno e que estas alterações ocorrem de forma concorrente. A maioria dos SGBDs relacionais não permite que duas transações diferentes alterem a mesma linha concorrentemente. Neste caso, a alternativa 1 implica em *seqüencializar* as transações que alteram informações referentes a um mesmo aluno, fazendo com que uma transação tenha que esperar pela outra. Já na alternativa 2, como os dados estão em tabelas diferentes, as transações podem ser executadas de forma concorrente.

■ exercício 5.2

A Figura 7.23 mostra o esquema relacional referente ao modelo ER da Figura 5.30.

Em relação a este modelo, cabem os seguintes comentários.

O relacionamento entre *FABRICANTE* e *PRODUTO* foi implementado através da chave estrangeira *CNPJ* dentro da tabela *Produto*. Como o relacionamento é identificador, a chave estrangeira faz parte da chave primária da tabela.

- A especialização de *PRODUTO* foi implementada através de tabela única (ver Seção Relacionamentos 1:1 para uma discussão de alternativas). Para identificar o tipo de produto (medicamento ou perfumaria) foi criada uma coluna (*TipoProd*) na tabela *Produto*.
- A entidade associativa (relacionamento entre *VENDA* e *MEDICAMENTO*) foi implementada como um relacionamento **n:n** através de tabela própria.

```

Produto (CNPJ, NúmeroProd, NomeComercial, TipoEmbalagem, Quantidade,
         PreçoUnitário, TipoProd, Tarja, Fórmula, Tipo)
         CNPJ referencia Fabricante
Fabricante (CNPJ, Nome, Endereço)
Venda (Data, NúmeroNota, NomeCliente, CidadeCliente)
PerfumariaVenda (CNPJ, NúmeroProd, NúmeroNota, Quantidade, Imposto)
         (CNPJ, NúmeroProd) referencia Produto
         NúmeroNota referencia Venda
MedicamentoVenda (CNPJ, NúmeroProd, NúmeroNota,
                  Quantidade, Imposto, CRM, Número)
         (CNPJ, NúmeroProd) referencia Produto
         NúmeroNota referencia Venda
         (CRM, Número) referencia ReceitaMédica
ReceitaMédica (CRM, Número, Data)

```

Figura 7.23 Modelo relacional para o Exercício 5.2.

- O relacionamento com *RECEITA MÉDICA* foi implementado através de chave estrangeira, dentro da tabela referente à entidade associativa.

■ exercício 5.3

A Figura 7.24 mostra o esquema relacional referente ao modelo ER da Figura 5.31.

A implementação segue as regras apresentadas. A única decisão tomada foi a de implementar a especialização de *TIPO DE VEÍCULO* através de uma tabela para cada entidade especializada (ver discussão na Seção Implementação de generalização/especialização).

■ exercício 5.4

A Figura 7.25 apresenta o modelo ER obtido através da aplicação das regras de engenharia reversa de modelos relacionais.

Os atributos de cada entidade/relacionamento estão listados na Figura 7.26 (os atributos identificadores estão sublinhados).

```

Escritório (NúmeroEscr, Local)
Contrato aluguel (NúmeroEscr, NúmeroContr, Data, Duração, NúmeroVeic,
                 NúmeroCartMotorista, EstadoCartMotorista)
    NúmeroEscr referencia Escritório
    NúmeroVeic referencia Veículo
    (NúmeroCartMotorista, EstadoCartMotorista)
                                     referencia Cliente
Cliente (NúmeroCartMotorista, EstadoCartMotorista,
        Nome, Endereço, Telefone)
Veículo (Número, DataPróximaManutenção, Placa, CódigoTipo)
    CódigoTipo referencia TipoVeiculo
TipoVeículo (CódigoTipo, Nome, ArCondicionado)
Similaridade (CódigoTipo, CódigoTipoSimilar)
    CódigoTipo referencia TipoVeiculo
    CódigoTipoSimilar referencia TipoVeiculo
Automóvel (CódigoTipo, NúmeroPortas, DireçãoHidráulica,
            CâmbioAutomático, Rádio)
    CódigoTipo referencia TipoVeículo
Ônibus (CódigoTipo, NúmeroPassageiros, Leito, Sanitário)
    CódigoTipo referencia TipoVeículo

```

Figura 7.24 Modelo relacional para o Exercício 5.3.

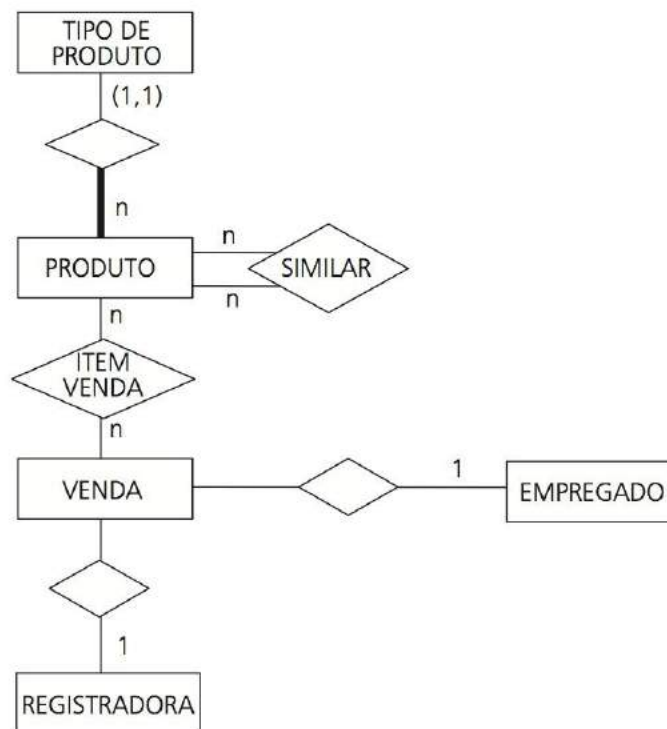


Figura 7.25 Modelo ER obtido através de engenharia reversa para o Exercício 5.4.

A aplicação das regras é direta. No modelo, apenas foram apresentadas as cardinalidades que puderam ser obtidas a partir do modelo ER:

- A cardinalidade do relacionamento *ITEM VENDA* é **n:n**, pois ele representa uma tabela cuja chave primária contém múltiplas chaves estrangeiras. A regra não determina a cardinalidade mínima.
- O mesmo se aplica ao relacionamento *SIMILAR*.
- Cada ocorrência da entidade *PRODUTO* está associada a exatamente uma ocorrência de *TIPO DE PRODUTO*. Esta cardinalidade é obtida pela sequência de raciocínio a seguir:
 - O relacionamento representa a chave estrangeira *CodigoTipoProd* da tabela *Produto*. Como no modelo relacional todos os campos são mo-

```

    Produto (NumeroProd, DescricaoProd, PreçoProd)
    TipoProd (CodigoTipoProd, DescricaoTipoProd)
    Venda (NúmeroNF, DataVenda)
    ItemVenda (QtdeItem, PreçoItem)
    Registradora (CodReg, SaldoReg)
    Empregado (CodEmp, NomeEmp, SenhaEmp)
    
```

Figura 7.26 Atributos do modelo ER do Exercício 5.4.

novalorados, conclui-se que cada produto pode estar associado a, no máximo, um tipo de produto (cardinalidade máxima 1).

- Além disso, a chave estrangeira `CódigoTipoProd` faz parte de uma chave primária. Colunas que fazem parte da chave primária são obrigatórias. Daí conclui-se que cada produto deve estar obrigatoriamente associado a um tipo de produto (cardinalidade mínima 1).
- O relacionamento entre *VENDA* e *EMPREGADO* representa uma chave estrangeira dentro da tabela *Venda* que não faz parte da chave primária. A única restrição de cardinalidade que pode ser derivada do modelo relacional é que cada venda tem a ela associado no máximo um empregado (cardinalidade máxima 1). Como o modelo relacional do exercício é incompleto e não informa quais colunas são obrigatórias e quais colunas são opcionais, não é possível determinar se uma venda está obrigatória ou opcionalmente associada a um empregado. Já na outra direção do relacionamento (quantas vendas estão associadas a um empregado), não é possível derivar alguma informação referente à cardinalidade a partir do modelo relacional fornecido.
- O mesmo aplica-se ao relacionamento entre *VENDA* e *REGISTRADORA*.

■ exercício 5.5

A Figura 7.27 apresenta o diagrama ER obtido por engenharia reversa do modelo relacional do exercício.

Os atributos das entidades e relacionamentos estão listados abaixo (atributos identificadores sublinhados):

Pessoa (PessID, PessNome, DataNasc, DataFalec, Sexo)

Local (LocID, Cidade, País)

Profissão (ProfID, ProfNome)

Casamento (CasamID, DataCasam)

■ exercício 6.1

Considerando apenas o conteúdo atual da tabela apresentada na Figura 6.8, as dependências funcionais nela contidas são:

$(A, B) \rightarrow C$

$A \rightarrow D$

$D \rightarrow A$

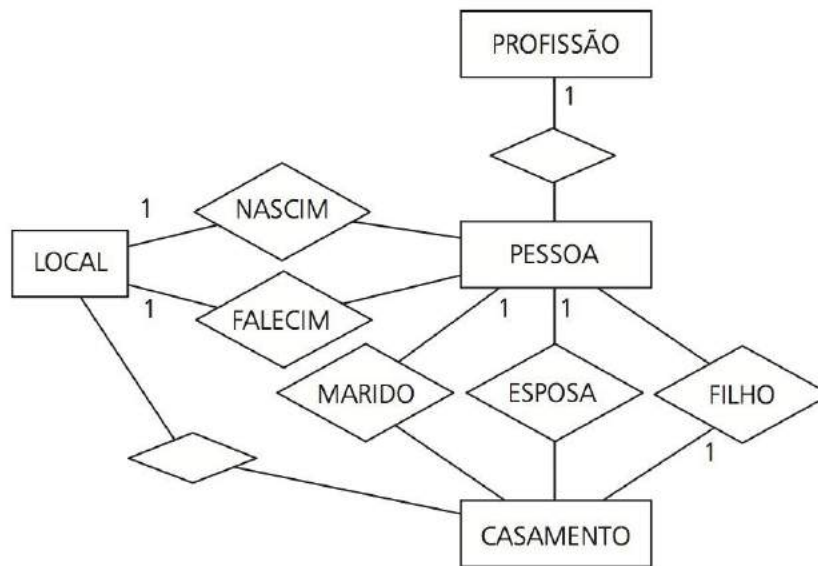


Figura 7.27 Modelo ER obtido por engenharia reversa.

■ exercício 6.2

A passagem à segunda forma normal (2FN) consta da eliminação de *dependências funcionais parciais*, isto é, de colunas não-chave que dependem apenas de uma parte da chave primária e não da chave primária completa. No exercício, as dependências funcionais parciais são:

```

(CodigoTipoProd, NumeroProd) → DescricaoProd
CodigoTipoProd → DescricaoTipoProd
NúmeroNF → DataVenda
NúmeroNF → CodReg
NúmeroNF → CodEmp
NúmeroNF → NomeEmp
    
```

Observe que o identificador de um produto é a chave composta pelo código do tipo do produto e pelo número do produto. A passagem à 2FN resulta nas seguintes tabelas:

2FN

```

ItemVenda (NúmeroNF, CodigoTipoProd, NumeroProd, QtdeItem, PreçoItem)
Produto (CodigoTipoProd, NumeroProd, DescricaoProd)
TipoProd (CodigoTipoProd, DescricaoTipoProd)
Venda (NúmeroNF, DataVenda, CodReg, CodEmp, NomeEmp)
    
```


1FN

```
(CodCongr, NomeCongr)
(CodCongr, CodGT, NomeGT)
(CodCongr, NumeroArt, TitArt, AssPrincArt)
(CodCongr, NumeroArt, CodAutor, NomeAutor)
```

2 FN

```
(CodCongr, NomeCongr)
(CodCongr, CodGT)
(CodGT, NomeGT)
(CodCongr, NumeroArt, TitArt, AssPrincArt)
(CodCongr, NumeroArt, CodAutor)
(CodAutor, NomeAutor)
```

$$3FN = 2FN$$

■ exercício 6.5

A seguir, são apresentadas as formas normais:

 $\tilde{N}N$

Comentário: Os campos NO-DE-AUTORES, NO-DE-REVISORES e NO-DE-TEMAS contêm informações redundantes, servindo para controlar o número de ocorrências dos grupos repetidos AUTOR, TEMA e REVISOR. Por este motivo, de acordo com a regra descrita na Seção Problemas da normalização na página 207, estes campos são eliminados já na passagem à forma ÑÑ.

```
(CodCongr,NumeroArt,NomeCongr,TitArt,  
      (CodAutor,NomeAutor)  
      (CodTema,NomeTema)  
      (CodRevisor,NomeRevisor,StatusRevisao))
```

1FN

```
(CodCongr, NumeroArt, NomeCongr, TitArt)
(CodCongr, NumeroArt, CodAutor, NomeAutor)
(CodCongr, NumeroArt, CodTema, NomeTema)
(CodCongr, NumeroArt, CodRevisor, NomeRevisor, StatusRevisao)
```

2 FN

```
(CodCongr, NumeroArt, TitArt)
(CodCongr, , NomeCongr)
(CodCongr, NumeroArt, CodAutor)
```

```

(CodAutor, NomeAutor)
(CodCongr, NumeroArt, CodTema)
(CodTema, NomeTema)
(CodCongr, NumeroArt, CodRevisor, StatusRevisao)
(CodRevisor, NomeRevisor)

```

3FN=2FN

■ exercício 6.6

O documento é um bom exemplo das eventuais dificuldades da normalização de documentos preparados para leitura por pessoas e não para processamento eletrônico.

NN

```

(CodCongr, NomeCongr,
    (Data,
        (Hora, TituloSessao, CodModerador, NomeModerador,
            (CodArt, TituloArt,
                (CodAutor, NomeAutor,
                    PaisAutor))))))

```

Comentários:

- Algumas sessões de um congresso, como a de registro (*registration*) ou a de intervalo (*break*) não possuem moderador. As sessões técnicas, nas quais são apresentados artigos, possuem um moderador. Para simplificar, considerou-se que cada sessão possui um campo "moderador" e que este campo é opcional.
- O documento contém chaves primárias propositadamente omitidas (ver Seção Problemas da normalização na pág. 205). No caso, foram omitidos o número do artigo, o código do autor e o código do moderador. Conforme explicado naquela seção, estes campos devem ser tratados como se aparecessem no documento.
- O documento informa o país dos autores. Esta informação aparece fatorada para todos os autores de um artigo que são do mesmo país. Para simplificar tratou-se o documento como se a informação aparecesse para cada autor.

1FN

```

(CodCongr, NomeCongr)
(CodCongr, Data)

```

(CodCongr, Data, Hora, TituloSessao, CodModerador, NomeModerador)
 (CodCongr, Data, Hora, CodArt, TituloArt)
 (CodCongr, Data, Hora, CodArt, CodAutor, NomeAutor, PaisAutor)

Comentário: Cada artigo é apresentado uma única vez. Essa informação é necessária para determinar a chave primária da quarta e da quinta tabelas. Nestas tabelas, a chave primária da tabela não é a concatenação das colunas chave primária das tabelas na forma ÑÑ, como foi nos exemplos até aqui apresentados. Para detalhes, ver Seção Passagem à primeira forma normal (1FN) na página 190.

2FN

(CodCongr, NomeCongr)
 (CodCongr, Data
 (CodCongr, Data, Hora, TituloSessao, CodModerador, NomeModerador)
 (CodCongr, CodArt, TituloArt, Data, Hora)
 (CodCongr, CodArt, CodAutor
 (CodAutor, NomeAutor, PaisAutor)

3FN

(CodCongr, NomeCongr)
 (CodCongr, Data
 (CodCongr, Data, Hora, TituloSessao, CodModerador)
 (CodModerador, NomeModerador)
 (CodCongr, CodArt, TituloArt, Data, Hora)
 (CodCongr, CodArt, CodAutor
 (CodAutor, NomeAutor, PaisAutor)

■ exercício 6.7

Comentários:

- O documento é uma carta que é enviada para o autor principal do artigo. Assim, o documento é identificado pelo código do congresso e o número do artigo (identificação de artigo).
- Como no exercício anterior, há uma chave primária omitida, que é o identificador do autor.
- O documento não contém grupos multivalorados de dados. Por isso, não há tabelas aninhadas.

ÑÑ

(CodAutorPrinc, NomeAutor, InstituicaoAutor, RuaNumAutor,
 CidadeAutor, EstadoAutor, PaisAutor, NumArtigo,

TituloArtigo, CodCongr, NomeCongr, DataCongr,
LocalCongr, DataLimCongr)

1FN=ÑÑ

2FN

(CodCongr, NumArtigo, CodAutorPrinc, NomeAutor,
InstituicaoAutor, RuaNumAutor, CidadeAutor, EstadoAutor,
PaisAutor, TituloArtigo)
(CodCongr, NomeCongr, DataCongr, LocalCongr, DataLimCongr)

3FN

(CodCongr, NumArtigo, CodAutorPrinc, TituloArtigo)
(CodAutorPrinc, NomeAutor, InstituicaoAutor, RuaNumAutor, CidadeAutor,
EstadoAutor, PaisAutor)
(CodCongr, NomeCongr, DataCongr, LocalCongr, DataLimCongr)

■ exercício 6.8

ÑÑ

(CodCongr, NomeCongr, DataInscrCongr,
(CodGT, NomeGT) ,
(CodInscr, NomeInscr, PaisInscr))

1FN

(CodCongr, NomeCongr, DataInscrCongr)
(CodCongr, CodGT, NomeGT)
(CodCongr, CodInscr, NomeInscr, PaisInscr)

2FN

(CodCongr, NomeCongr, DataInscrCongr)
(CodCongr, CodGT)
(CodGT, NomeGT)
(CodCongr, CodInscr)
(CodInscr, NomeInscr, PaisInscr)

3FN=2FN

■ exercício 6.9

Os modelos normalizados de cada documento estão listados a seguir. Cada tabela recebeu um nome, facilitando a identificação das tabelas que devem ser juntadas.

■ modelo do exercício 6.4

Congresso (CodCongr, NomeCongr)
 CongrGT (CodCongr, CodGT)
 GT (CodGT, NomeGT)
 Artigo (CodCongr, NumeroArt, TitArt, AssPrincArt)
 ArtigoAutor (CodCongr, NumeroArt, CodAutor)
 Autor (CodAutor, NomeAutor)

■ modelo do exercício 6.5

Artigo (CodCongr, NumeroArt, TitArt)
 Congresso (CodCongr, NomeCongr)
 ArtigoAutor (CodCongr, NumeroArt, CodAutor)
 Autor (CodAutor, NomeAutor)
 CongrTema (CodCongr, NumeroArt, CodTema)
 Tema (CodTema, NomeTema)
 CongrRevisao (CodCongr, NumeroArt, CodRevisor, StatusRevisao)
 Revisor (CodRevisor, NomeRevisor)

■ modelo do exercício 6.6

Congresso (CodCongr, NomeCongr)
 CongrData (CodCongr, Data)
 Sessão (CodCongr, Data, Hora, TituloSessao, CodModerador)
 Moderador (CodModerador, NomeModerador)
 Artigo (CodCongr, CodArt, TituloArt, Data, Hora,)
 ArtigoAutor (CodCongr, CodArt, CodAutor)
 Autor (CodAutor, NomeAutor, PaisAutor)

■ modelo do exercício 6.7

Artigo (CodCongr, NumArtigo, CodAutorPrinc, TituloArtigo)
 Autor (CodAutorPrinc, NomeAutor, InstituicaoAutor,
 RuaNumAutor, CidadeAutor, EstadoAutor, PaisAutor)
 Congresso (CodCongr, NomeCongr, DataCongr, LocalCongr, DataLimCongr)

■ modelo do exercício 6.8

Congresso (CodCongr, NomeCongr, DataInscrCongr)
 CongrGT (CodCongr, CodGT)
 GT (CodGT, NomeGT)
 CongrInscr (CodCongr, CodInscr)
 Inscrito (CodInscr, NomeInscr, PaisInscr)

■ modelo integrado

```

Artigo (CodCongr, NumeroArtigo, TituloArt, AssPrincArt,
        CodAutorPrinc, Data, Hora)
ArtigoAutor (CodCongr, NumeroArt, CodAutor)
Pessoa (CodPess, NomePess, Instituicao, RuaNum, Cidade, Estado, Pais)
Congresso (CodCongr, NomeCongr, DataCongr, LocalCongr,
           DataLimCongr, DataInscrCongr)
CongrGT (CodCongr, CodGT)
CongrInscr (CodCongr, CodInscr)
ArtRevisao (CodCongr, NumeroArt, CodRevisor, StatusRevisao)
ArtTema (CodCongr, NumeroArt, CodTema)
GT (CodGT, NomeGT)
Sessão (CodCongr, Data, Hora, TituloSessao, CodModerador)
Tema (CodTema, NomeTema)

```

Comentários:

- Nos diferentes modelos, a mesma tabela aparece sob diferentes nomes (o problema dos sinônimos descrito na Seção Integração de tabelas com mesma chave na página 210). Trata-se de uma única tabela, que deve constar uma única vez no modelo integrado. Por exemplo, as várias tabelas que contêm dados de pessoas (autor, moderador, revisor,...) foram unidas na tabela Pessoa. O mesmo problema de sinônimos aplica-se a campos. O campo número do artigo aparece uma vez sob a denominação CodArtigo e outra sob a denominação NumArtigo.
- A tabela CongrData foi eliminada pelo fato de seus dados estarem contidos na tabela Sessão. Para detalhes, ver Seção Integração de tabelas com chaves contidas na página 211.

■ exercício 6.10

```

Artigo (CodCongr, NumeroArtigo, TituloArt, AssPrincArt,
        CodAutorPrinc, Data, Hora)
        CodCongr referencia Congresso
        (CodCongr, Data, Hora) referencia Sessão
        CodAutorPrinc referencia Pessoa
ArtigoAutor (CodCongr, NumeroArt, CodAutor)
        (CodCongr, NumeroArt) referencia Artigo
        CodAutor referencia Pessoa
Pessoa (CodPess, NomePess, Instituicao, RuaNum, Cidade, Estado, Pais)
Congresso (CodCongr, NomeCongr, DataCongr, LocalCongr,
           DataLimCongr, DataInscrCongr)

```

```

    CongrGT(CodCongr,CodGT) [CodCongr referencia
    Congresso [CodGT referencia GT
    CongrInscr(CodCongr,CodInscr)
    CodCongr referencia Congresso
    CodInscr referencia Pessoa
    ArtRevisao(CodCongr,NumeroArt,CodRevisor,StatusRevisao)
    (CodCongr,NumeroArt) referencia Artigo
    CodRevisor referencia Pessoa
    ArtTema(CodCongr,NumeroArt,CodTema)
    (CodCongr,NumeroArt) referencia Artigo
    CodTema referencia Tema
    GT(CodGT,NomeGT)
    Sessao(CodCongr,Data,Hora,TituloSessao,CodModerador)
    CodCongr referencia Congresso
    CodModerador referencia Pessoa
    Tema(CodTema,NomeTema)

```

Observações:

- As várias colunas que compõem uma chave estrangeira não necessariamente foram mostradas juntas na tabela. Este é o caso da chave estrangeira composta pelas colunas CodCongr, Data e Hora, que aparece na tabela Artigo e que referencia a tabela Sessao.
- A chave estrangeira não necessariamente tem o mesmo nome da chave primária que referencia. Este é o caso das várias chaves estrangeiras que referenciam a tabela Pessoa no exemplo.

■ exercício 6.12

A Figura 7.28 apresenta o diagrama ER obtido através da engenharia reversa do modelo relacional obtido no exercício precedente. Os atributos das entidades e dos relacionamentos do diagrama ER são os seguintes:

```

Artigo(NumeroArtigo,TituloArt,AssPrincArt)
Pessoa(CodPess,NomePess,Instituicao,RuaNum,Cidade, Estado,Pais)
Congresso(CodCongr,NomeCongr,DataCongr,LocalCongr,
          DataLimCongr,DataInscrCongr)
GT(CodGT,NomeGT)
Sessao(Data,Hora,TituloSessao)
Tema(CodTema,NomeTema)

```

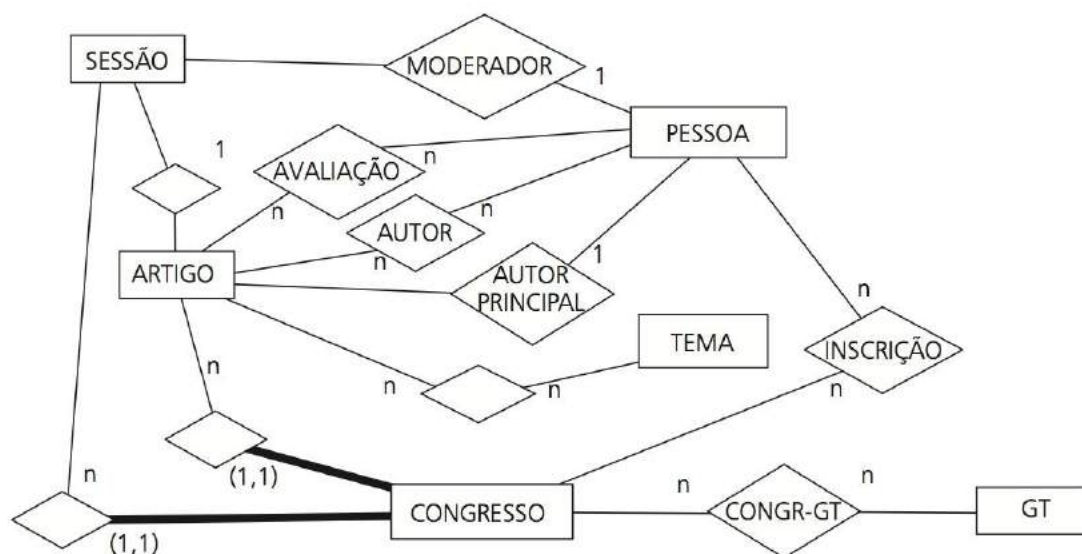


Figura 7.28 Diagrama ER obtido através de engenharia reversa.

Em linhas gerais, os símbolos do diagrama ER têm a mesma disposição que aquela apresentada no modelo do sistema de preparação de congressos que aparece na Figura 7.19. Com isso, é possível comparar o modelo obtido através da engenharia reversa com o modelo obtido através da estratégia descendente, usando como entrada do processo a descrição do problema.

O modelo obtido por engenharia reversa é diferente daquele obtido pela estratégia descendente partindo da descrição do problema (Figura 7.19). Vários elementos do modelo que haviam sido identificados na Figura 7.19 não aparecem no modelo, como os relacionamentos referentes aos membros do GT, aos membros dos comitês de programa e organizador. A engenharia reversa somente identifica os elementos do modelo ER que aparecem nos documentos ou arquivos tratados.

Por outro lado, da engenharia reversa resultaram alguns elementos (como a entidade *TEMA*) que não apareciam no modelo da Figura 7.19, já que eles não haviam sido referenciados na descrição do problema, mas aparecem nos arquivos e documentos processados na engenharia reversa.

O modelo obtido pela engenharia reversa não é necessariamente um modelo perfeito. Ele pode ainda conter anomalias e até mesmo redundâncias de dados, conforme descrito a seguir.

A engenharia reversa não detecta relacionamentos redundantes (ver Seção Um modelo deve ser livre de redundância na página 87). Este é o caso do relacionamento entre a entidade *SESSÃO* e a entidade *CONGRESSO*. O relacionamento é redundante, uma vez que as informações nele contidas podem ser obtidas através dos relacionamentos entre *SESSÃO* e *ARTIGO* e entre *ARTIGO* e *CONGRESSO*. Este relacionamento aparece no modelo, pelo fato de o código do congresso fazer parte da chave primária da entidade *SESSÃO*.

Outra anomalia do modelo obtido pela engenharia reversa são os relacionamentos entre um artigo e a pessoa que é autora do artigo. Foram identificados dois relacionamentos, um que informa o autor principal de cada artigo e outro que informa o conjunto de todos os autores do artigo. Do ponto de vista da modelagem conceitual, estes relacionamentos não estão incorretos. Entretanto, do ponto de vista do desenvolvimento de aplicações sobre o banco de dados em questão, a separação dos autores de um artigo em dois conjuntos complica o tratamento (alterações e consultas) de autores, já que exige um tratamento especial para o autor principal do artigo. O tratamento de autores é simplificado se for utilizado um único relacionamento de autoria, tendo este relacionamento um atributo, que informa se o autor é ou não o autor principal do artigo.

Além dos problemas acima mencionados, o modelo demonstra o resultado de um documento conter uma chave primária implícita e de ela não ter sido tratada durante a normalização (ver Seção Problemas da normalização na página 207). Na entidade *ARTIGO*, aparece uma coluna denominada *Ass-PrincArt*. Esta coluna contém a descrição do assunto ou *tema* principal do artigo. Esta informação não deveria aparecer aqui na forma de um atributo de artigo, e sim, de um relacionamento entre as entidades *ARTIGO* e *TEMA*. O problema ocorreu pelo fato de não termos detectado, quando da normalização do documento referente ao Exercício 6.4, o fato de todo assunto ou tema possuir um código. Deveríamos ter incluído, já na normalização daquele documento, o código do tema. Fica como exercício para o leitor a revisão do resultado dos exercícios incorporando esta correção.

exercício 6.13

ÑÑ

(NoCorr,

(NoRecept, CodPeca, DescrPeca, QtdeEstog))

Observação: O campo data de emissão do documento foi desconsiderado (ver discussão sobre campos irrelevantes na Seção Problemas da normalização na página 207).

1FN

(NoCorr)

(NoCorr, NoRecept, CodPeca, DescrPeca, QtdeEstog)

2FN = 1FN

3FN

(NoCorr)

(NoCorr, NoRecept, CodPeca, QtdeEstog)

(CodPeca, DescrPeca)

■ exercício 6.14

ÑN

(CodOC, DataEntrega, NoEstrado, CodEmpilh, CodOperEmpilh,

(NoCorr,

(NoRecept, CodPeca, DescrPeca,
QtdePeca)))

Observação: Os campos NO-DE-CORREDORES e NO-DE-RECEPTACULOS foram desconsiderados por serem redundantes, servindo apenas para controlar o tamanho dos itens de grupo do COBOL (ver Seção Problemas da normalização na página 207).

1FN

(CodOC, DataEntrega, NoEstrado, CodEmpilh, CodOperEmpilh)

(CodOC, DataEntrega, NoCorr)

(CodOC, DataEntrega, NoCorr, NoRecept, CodPeca, DescrPeca, QtdePeca)

2FN = 1FN

3FN

(CodOC, DataEntrega, NoEstrado, CodEmpilh, CodOperEmpilh)

(CodOC, DataEntrega, NoCorr)

(CodOC, DataEntrega, NoCorr, NoRecept, CodPeca, QtdePeca)

(CodPeca, DescrPeca)

■ exercício 6.15

NN

(NoPed, CodCli, NomeCli, NoRampa)

1FN=NN

2FN=1FN

3FN

(NoPed, CodCli, NoRampa)

(CodCli, NomeCli)

■ exercício 6.16

NN

(NoOC, DataOC, CodFornec, NomeFornec,
(CodPeca, DescrPeca, QuantPed,
(DataEntr, QuantEntr)))

1FN

(NoOC, DataOC, CodFornec, NomeFornec)

(NoOC, CodPeca, DescrPeca, QuantPed)

(NoOC, CodPeca, DataEntr, QuantEntr)

2FN

(NoOC, DataOC, CodFornec, NomeFornec)

(NoOC, CodPeca, QuantPed)

(CodPeca, DescrPeca)

(NoOC, CodPeca, DataEntr, QuantEntr)

3FN

(NoOC, DataOC, CodFornec)

(CodFornec, NomeFornec)

(NoOC, CodPeca, QuantPed)

(CodPeca, DescrPeca)

(NoOC, CodPeca, DataEntr, QuantEntr)

■ exercício 6.17

NN

(NoPed, DataPed, CodCli, NomeCli,
(NoTel),
(CodPeca, DescrPeca, QuantPed))

1FN

(NoPed, DataPed, CodCli, NomeCli)
 (NoPed, NoTel)
 (NoPed, CodPeca, DescrPeca, QuantPed)

2FN

(NoPed, DataPed, CodCli, NomeCli)
 (NoPed, NoTel)
 (NoPed, CodPeca, QuantPed)
 (CodPeca, DescrPeca)

3FN

(NoPed, DataPed, CodCli)
 (CodCli, NomeCli)
 (NoPed, NoTel)
 (NoPed, CodPeca, QuantPed)
 (CodPeca, DescrPeca)

Este exercício exemplifica o problema referido na página 190 (Seção Passagem à primeira forma normal (1FN)), quanto à opção de usar a decomposição de tabelas na passagem à 1FN. No exemplo, os números de telefone que aparecem são os números dos telefones do cliente. Entretanto, ao decompor a tabela ÑÑ, o telefone fica desvinculado do cliente correspondente, na tabela:

(NoPed, NoTel)

Quando o esquema relacional for transformado em um modelo ER, esta tabela corresponderá a uma entidade Telefone vinculada à entidade Pedido e não à entidade Cliente, como seria o correto. Deixamos a correção deste problema para depois da construção do modelo ER.

■ exercício 6.18

ÑÑ

(NoPed, DataPed, CodCli, NomeCli,
 (NoCorr,
 (NoRecept, CodPeca, DescrPeca, QuantBusca)))

1FN

(NoPed, DataPed, CodCli, NomeCli)
 (NoPed, NoCorr)

(NoPed, NoCorr, NoRecept, CodPeca, DescrPeca, QuantBusca)

2FN = 1FN

3FN

(NoPed, DataPed, CodCli)

(CodCli, NomeCli)

(NoPed, NoCorr)

(NoPed, NoCorr, NoRecept, CodPeca, QuantBusca)

(CodPeca, DescrPeca)

■ exercício 6.19

Os modelos normalizados de cada documento estão listados a seguir. Cada tabela recebeu um nome, facilitando a identificação das tabelas que devem ser juntadas.

■ modelo do exercício 6.13

Corredor (NoCorr)

Receptáculo (NoCorr, NoRecept, CodPeca, QtdeEstog)

Peca (CodPeca, DescrPeca)

■ modelo do exercício 6.14

Entrega (CodOC, DataEntrega, NoEstrado, CodOperEmpilh)

DistrCorr (CodOC, DataEntrega, NoCorr)

ItemDistr (CodOC, DataEntrega, NoCorr, NoRecept, CodPeca, QtdePeca)

Peca (CodPeca, DescrPeca)

■ modelo do exercício 6.15

Pedido (NoPed, CodCli, NoRampa)

Cliente (CodCli, NomeCli)

■ modelo do exercício 6.16

OC (NoOC, DataOC, CodFornec)

Fornec (CodFornec, NomeFornec)

ItemOC (NoOC, CodPeca, QuantPed)

Peca (CodPeca, DescrPeca)

ItemEntr (NoOC, CodPeca, DataEntr, QuantEntr)

■ modelo do exercício 6.17

Pedido (NoPed, DataPed, CodCli)
 Cliente (CodCli, NomeCli)
 Telefone (NoPed, NoTel)
 ItemPedido (NoPed, CodPeca, QuantPed)
 Peca (CodPeca, DescrPeca)

■ modelo do exercício 6.18

Pedido (NoPed, DataPed, CodCli)
 Cliente (CodCli, NomeCli)
 BuscaCorredor (NoPed, NoCorr)
 ItemBusca (NoPed, NoCorr, NoRecept, CodPeca, QuantBusca)
 Peca (CodPeca, DescrPeca)

■ modelo integrado

Corredor (NoCorr)
 Receptáculo (NoCorr, NoRecept, CodPeca, QtdeEstoq)
 Peca (CodPeca, DescrPeca)
 Entrega (NoOC, DataEntrega, NoEstrado, CodOperEmpilh)
 DistrCorr (NoOC, DataEntrega, NoCorr)
 ItemDistr (NoOC, DataEntrega, NoCorr, NoRecept, CodPeca, QtdePeca)
 Pedido (NoPed, CodCli, NoRampa, DataPed)
 Cliente (CodCli, NomeCli)
 OC (NoOC, DataOC, CodFornec)
 Fornec (CodFornec, NomeFornec)
 ItemOC (NoOC, CodPeca, QuantPed)
 ItemEntr (NoOC, CodPeca, DataEntr, QuantEntr)
 Telefone (NoPed, NoTel)
 ItemPedido (NoPed, CodPeca, QuantPed)
 BuscaCorredor (NoPed, NoCorr)
 ItemBusca (NoPed, NoCorr, NoRecept, CodPeca, QuantBusca)

Observações:

- Na transcrição dos esquemas das tabelas ao modelo integrado, foi feito o tratamento de sinônimos (ver Seção Integração de tabelas com mesma chave na página 210). A coluna referente ao código da ordem de compra aparecia sob os nomes NoOC e CodOC.
- Foi feito o tratamento de tabelas com chaves contidas (ver Seção Integração de tabelas com chaves contidas na página 211). A tabela DistrCorr

(modelo do Exercício 6.14) foi eliminada por estar contida na tabela *ItemDistr* do mesmo exercício. Pela mesma razão, foi eliminada a tabela *BuscaCorredor* (modelo do Exercício 6.18). Deve ser observado que a tabela *Corredor* (modelo do Exercício 6.13) não foi integrada à tabela *Receptáculo* do mesmo modelo. Ocorre que podem haver estados do banco de dados nos quais um corredor já foi incluído na tabela de *Corredor*, mas não aparece ainda na tabela *Receptáculo* (estamos considerando que corredores e receptáculos são incluídos no banco de dados em transações diferentes). Este é o mesmo caso do terceiro exemplo da Seção Integração de tabelas com chaves contidas (página 211).

■ exercício 6.20

Corredor (NoCorr)
Receptáculo (NoCorr, NoRecept, *CodPeca*, *QtdeEstoque*)
 NoCorr referencia *Corredor*
 CodPeca referencia *Peca*
Peca (CodPeca, *DescrPeca*)
Entrega (NoOC, DataEntrega, NoEstrado, *CodOperEmpilh*)
 NoOC referencia *OC*
ItemDistr (NoOC, DataEntrega, NoCorr, NoRecept, *CodPeca*, *QtdePeca*)
 (NoOC, DataEntrega, *CodPeca*) referencia *ItemEntr*
 (NoCorr, NoRecept) referencia *Receptáculo*

Observação: NoOC não foi considerado como chave estrangeira da tabela OC, nem (NoOC, DataEntrega) foi considerado chave estrangeira da tabela Entr.

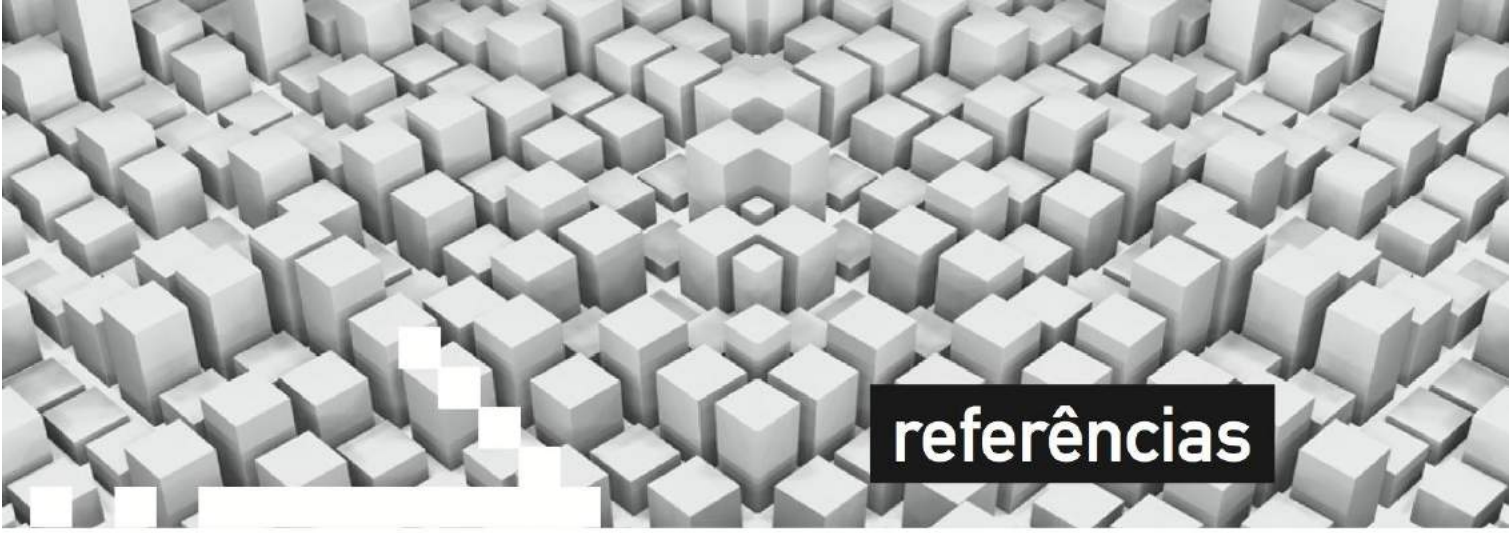
Estas chaves estrangeiras seriam redundantes com as demais chaves estrangeiras que aparecem no modelo. Fica como exercício para o leitor identificar as chaves estrangeiras com as quais estas seriam redundantes.

Pedido (NoPed, *CodCli*, *NoRampa*, *DataPed*)
 CodCli referencia *Cliente*
Cliente (CodCli, *NomeCli*)
OC (NoOC, *DataOC*, *CodFornec*)
 CodFornec referencia *Fornec*
Fornec (CodFornec, *NomeFornec*)
ItemOC (NoOC, CodPeca, *QuantPed*)
 NoOC referencia *OC*
 CodPeca referencia *Peca*
ItemEntr (NoOC, CodPeca, DataEntr, *QuantEntr*)
 (NoOC, DataEntr) referencia *Entrega*

```

        (NoOC,CodPeca) referencia ItemOC
Telefone (NoPed, NoTel)
        NoPed referencia Pedido
ItemPedido (NoPed, CodPeca, QuantPed)
        NoPed referencia Pedido
        CodPeca referencia Peca
ItemBusca (NoPed, NoCorr, NoRecept, CodPeca, QuantBusca)
        (NoPed, CodPeca) referencia Peca
        (NoCorr, NoRecept) referencia Receptáculo

```



referências

ABRIAL, J. Data semantics. In: DATA BASE MANAGEMENT, 1974, Amsterdam. *Anais...* Amsterdam: North-Holland, 1974. p.1–59.

BARKER, R. *Case*method: entity relationship modelling*. Wokingham: Addison-Wesley, 1990.

BATINI, C.; CERI, S.; NAVATHE, S. *Conceptual database design: an entity relationship approach*. Redwood City: Benjamin-Cummings, 1992.

CHEN, P. P. The entity-relationship model-toward a unified view of data. *ACM Trans. on Database Syst.*, Salt Lake City, v. 1, n. 1, p. 9–36, Mar. 1976.

CODD, E. F. A relational model of data for large shared data banks. *Commun. of the ACM*, New York, v. 13, n.6, p. 377–387, June 1970.

CODD, E. F. Further normalization of the data base relational model. In: RUSTIN, R. (Ed.). *Database systems*. Englewood Cliffs: Prentice Hall, 1972. p.33–64. (Courant Computer Symposia Series, v.6).

CODD, E. F. Normalized data structure: a brief tutorial. In: ACM-SIGFIDET WORKSHOP ON DATA DESCRIPTION, ACCESS AND CONTROL, 1971, San Diego. *Proceedings...* New York: ACM, 1971, p. 321-350.

DUMPALA, S.; ARORA, S. Schema translation using the entity-relationship approach. In: INTERNATIONAL CONFERENCE ON ENTITY RELATIONSHIP APPROACH, 3. th, 1983, Amsterdam. *Proceedings...* Amsterdam: North-Holland, 1983, p. 337-356

ELMASRI, R.; NAVATHE, S. B. *Sistemas de Banco de Dados - Fundamentos e Aplicações*. 3.ed. Rio de Janeiro: LTC, 2002.

FOWLER, M.; SCOTT, K. *UML essencial*. 2.ed. Porto Alegre: Bookman, 2000.

GROFF, J. R.; WEINBERG, P. N. *SQL: the complete reference*. Berkeley: McGrawHill, 1999.

JOHANNESSON, P.; KALMAN, K. A method for translating relational schemas into conceptual schemas. In: ENTITY-RELATIONSHIP APPROACH TO DATABASE DESIGN AND QUERYING, PROCEEDINGS OF THE EIGHT INTERNATIONAL CONFERENCE ON ENTITY-RELATIONSHIP APPROACH, 1989, TORONTO. *Anais...* Amsterdam: North-Holland, 1989. p.271–285.

KENT, W. *Data and reality*. Amsterdam: North-Holland, 1978.

KENT, W. Limitations of record-based information models. *ACM Trans. on Database Syst.*, Salt Lake City, v. 4, n. 1, p. 107–131, 1979.

KIPPER, E. F. et al. *Engenharia de informações: conceitos técnicas e métodos*. Porto Alegre: Sagra, 1993.

MANINILA, H.; RÄIHÄ, K. *The design of relational databases*. Wokingham: Addison-Wesley, 1992.

MARTIN, J. *Information engineering*. Upper Saddle River: Prentice-Hall, 1990.

NAVATHE, S. B.; AWONG, A. M. Abstracting relational and hierarchical data with a semantic data model. In: ENTITY-RELATIONSHIP APPROACH, PROCEEDINGS OF THE SIXTH INTERNATIONAL CONFERENCE ON ENTITY-RELATIONSHIP APPROACH, 1987, NEW YORK. *Anais...* Amsterdam: North-Holland, 1987. p. 305–333.

NIJSSSEN, G.; HALPIN, T. *Conceptual schema and relational database design: a fact oriented approach*. Upper Saddle River: Prentice-Hall, 1989.

PRESSMAN, R. S. *Engenharia de software*. 5. ed. Rio de Janeiro: Mc-Graw Hill, 2002.

SANTOS, C. S. dos; NEUHOLD, E. J.; FURTADO, A. L. A data type approach to the entity-relationship approach. In: ENTITY-RELATIONSHIP APPROACH TO SYSTEMS ANALYSIS AND DESIGN. PROCEEDINGS OF THE 1ST INTERNATIONAL

CONFERENCE ON THE ENTITY-RELATIONSHIP APPROACH, 1980, Los Angeles. *Anais...* Amsterdam: North-Holland, 1980. p. 103–119.

SCHEUERMANN, P.; SCHIFFNER, G.; WEBER, H. Abstraction capabilities and invariant properties modelling within the entity-relationship approach. In: ENTITY-RELATIONSHIP APPROACH TO SYSTEMS ANALYSIS AND DESIGN. PROCEEDINGS OF THE 1ST INTERNATIONAL CONFERENCE ON THE ENTITY-RELATIONSHIP APPROACH, 1980, Los Angeles. *Anais...* Amsterdam: North-Holland, 1980. p. 121–140.

SETZER, V. W. *Banco de dados*. 2.ed. São Paulo: Edgard Blücher, 1987.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistemas de Banco de Dados*. 3.ed. São Paulo: Makron Books, 1999.

SMITH, J. M.; SMITH, D. C. P. Database abstractions: aggregation and generalization. *ACM Trans. on Database Syst.*, Salt Lake City, v. 2, n. 2, p. 105–133, June 1977.

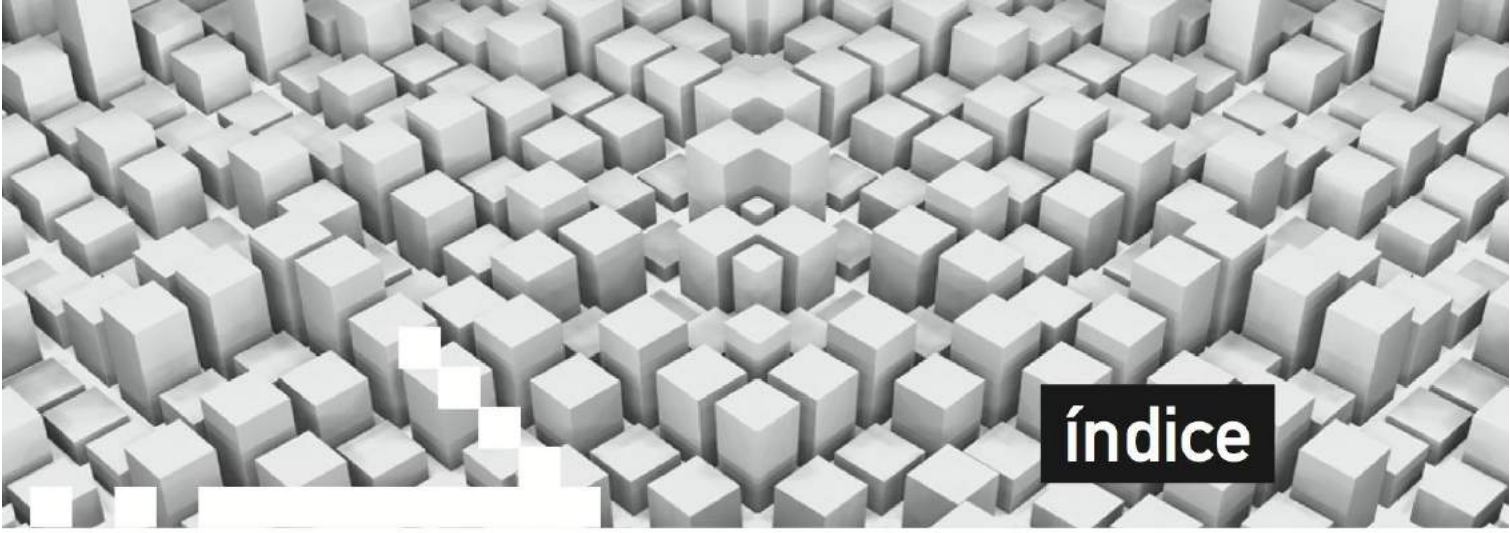
TARDIEU, H.; ROCHFELD, A.; COLLETI, R. *Le methode Merise: principes et outils*. Paris: Les Editions d'Organization, 1983.

TEOREY, T. *Database modeling & design: the fundamental principles*. 2nd. ed. San Francisco: Morgan Kaufmann, 1994.

TEOREY, T. J.; YANG, D.; FRY, J. P. A logical design methodology for relational databases using the extended entity-relationship model. *ACM Computing Surveys*, New York, v. 18, n. 2, p. 197–222, June 1986.

VERHEIJEN, G.; BEKKUM, J. van. NIAM: an information analysis method. In: IFIP TC-8 CONFERENCE ON COMPARATIVE REVIEW OF INFORMATION SYSTEMS METHODOLOGIES (CRIS-1), 1982. *Proceedings...* Amsterdam: North-Holland, 1982, p. 537–590.

WONG, E.; KATZ, R. H. Logical design and schema conversion for relational and DBTG databases. In: ENTITY-RELATIONSHIP APPROACH TO SYSTEMS ANALYSIS AND DESIGN. PROC. INTERNATIONAL CONFERENCE ON THE ENTITY-RELATIONSHIP APPROACH, 1., 1979, Los Angeles. *Anais...* Amsterdam: North-Holland, 1979. p. 311–322.



Índice

Os números em *itálico* referem-se a páginas de exercícios corrigidos.

1:1 ver Relacionamento, **1:1**

1:n ver Relacionamento, **1:n**

n:n ver Relacionamento, **n:n**

1FN ver Forma normal, primeira

2FN ver Forma normal, segunda

3FN ver Forma normal, terceira

4FN ver Forma normal, quarta

Abordagem de modelagem de dados, 25

Abordagem entidade-relacionamento
ver Modelo entidade-relacionamento

Abordagem relacional, 120-131

Abordagem UML, 97-99

Associação ver Relacionamento

Atributo, 48, 48-54, 140, 241

cardinalidade de, 49

critérios de escolha, 77-85

de relacionamento, 49

domínio de, 49, 79, 104, 242

e entidade relacionada, 78-79

e especialização, 79-81

identificador, 50-53, 243, 246

implícito na normalização, 208-209

irrelevante ou derivado, 209

modificado com o tempo, 89-90

monovalorado, 49

multivalorado, 49, 84-85, 165-167,
233

nome de, 121, 141-142

obrigatório, 49

opcional, 49, 83-84, 236-237

redundante, 87-89, 165, 209

temporal, 89-90

Auto-relacionamento, 38-39

Banco de dados, 22

compartilhamento de dados, 20-22

consulta ao, 130-131

engenharia reversa de, 29

sintonia de, 29

Campo, 121-122

domínio de, 126-127

monovalorado, 121

multivalorado, 122, 154

nome de, 121

Cardinalidade, 39, 39-48, 104

de atributo, 49

máxima, 40-43, 239-240

máxima **1**, 40, 146

máxima **n**, 40

mínima, 45, 45-46, 242

mínima **0**, 45

mínima **1**, 45, 235

CASE ver Ferramentas de modelagem

Chave, 122-128

alternativa, 125-126

engenharia reversa, 171-172

- estrangeira, 123-125, 130, 145-146, 251, 265, 273
- primária, 122-123, 143, 156, 197-198, 251
 - nome, 141-142
 - omitida ou incorreta, 207-208
 - restrição de integridade, 123
- secundária, 251
- Coluna, 121, 140, 144-145
 - domínio de, 126-127, 129-130, 210-211
 - engenharia reversa, 174-175
 - monovalorada, 121, 154
 - multivalorada, 121, 154, 187
 - nome de, 141-142
 - obrigatória, 127
 - opcional, 127, 140, 150
 - redundante, 164
- Dependência funcional, 195-196
 - indireta, 258
 - multivalorada, 205
 - parcial, 197, 257
 - transitiva, 258
- DER *ver* Diagrama entidade-relacionamento
- Diagrama de ocorrências, 37, 37-38, 230-231
- Diagrama entidade-relacionamento, 25-26, 34
 - símbolos utilizados no, 62
 - variantes de notação, 94-100
- Dicionário de dados, 49, 101
- Domínio de atributo, 49, 79, 104, 242
- Domínio de campo, 126-127
- Domínio de coluna, 126-127, 129-130, 210-211
- Engenharia de informações, notação, 95-97
- Engenharia reversa, 102-103, 169-175
 - de arquivo, 184-214, 258-262, 267-271
 - de modelos relacionais, 136, 169-175, 254-256, 265-267
 - processo, 170
- Entidade, 34, 34-36, 140, 238, 240, 242, 244
 - associativa, 60-62
 - critérios de escolha, 77-85
 - e especialização, 81-83
 - fraca, 52
 - identificador de, 50-53, 241, 242, 246
 - identificador mínimo, 52
 - implementação na abordagem relacional, 140-144
 - isolada, 94
 - ocorrência de, 35
 - papel de, 38
 - sem atributos, 94
- Entidade-relacionamento
 - abordagem, 25, 34-64, 136-137
 - diagrama, 25-26
- ER *ver* Entidade-relacionamento
- Erro semântico, 86
- Erro sintático, 86
- Especialização *ver* Generalização e especialização
- Esquema de banco de dados, 62-64
 - esquema gráfico *ver* Diagrama entidade-relacionamento
 - esquema textual, 62-64
 - relacional, 128-130
 - notação diagramática, 129-130
 - notação textual, 128-129
- Ferramentas de modelagem, 100-102
 - ferramenta CASE, 72, 94, 101, 242
 - programas de propósito geral, 101-102
- Forma normal, 190 *ver também* Normalização
 - não-normalizada, 187-190
 - primeira, 190
 - quarta, 206
 - segunda, 197, 257, 258
 - terceira, 200, 258

- Generalização e especialização, 54, 54-60, 236-237, 238, 246
 compartilhada, 57-58, 233-235, 235
 exclusiva, 57, 235
 formas de, 79-83
 herança múltipla, 58
 implementação na abordagem relacional, 156-162
 níveis de, 58-60
 parcial, 56
 total, 56, 233-235
- Gerência de banco de dados *ver* Sistema de gerência de banco de dados
- Herança de propriedades, 55
- Identificador de entidade, 50-53, 241-242, 246
- Identificador de relacionamento, 53-54, 143, 163
- Instância, 37
- Integração de modelos, 209-213, 262-264, 271-273
 tabelas com a mesma chave, 210-211
 tabelas com chaves contidas, 211-213
- Integração de visões *ver* Integração de modelos
- Integridade referencial, 127
- Junção, 131, 138-139
- Linguagem de modelagem de dados, 24
- Linguagem de terceira geração, 184
- Linha, 120
- Modelo de banco de dados relacional, 128-130
 modelo diagramático, 129-130
 modelo textual, 128-129
- Modelo de dados, 24, 24-29
 conceitual, 25, 25-26, 27-29, 169, 184
 físico, 27, 29
 lógico, 26, 26-27, 29, 136-137, 184
- Modelo entidade-relacionamento, 25, 34, 34-64, 136-137
 construção, 77-85, 102-105, 214, 240-242
 engenharia reversa, 102-103, 169-175
 estratégia ascendente, 103
 estratégia descendente, 103-104, 238-240
 estratégia *inside-out*, 104-105
 especialização, 81-83
 propriedades, 72-77
 aspecto temporal, 89-93, 242
 capacidade de expressão limitada, 73-75, 87, 235-236
 correção, 85-87
 equivalência, 75-77
 precisão, 72-73, 87-89
 redundância, 87-89, 165, 209
 representação gráfica, 25-26, 34, 62-100
 representação textual, 62-64
 variantes, 94-102
 abordagem UML, 97-99
 notação engenharia de informações, 95-97
 notação européia para cardinalidades, 99-100
 verificação, 85-94, 214
- Multiplicidade, 99
- Normalização, 190-209
 limitações, 214
 passagem à primeira forma normal (1FN), 190-195
 passagem à quarta forma normal (4FN), 203-207
 passagem à segunda forma normal (2FN), 196-200
 passagem à terceira forma normal (3FN), 200-203
 problemas de, 207-209

- retorno à segunda forma normal (2FN), 213
- Notação européia para cardinalidade, 99-100, 237-238
- NULL* ver Vazio (Valor de campo)
- Primeira forma normal, ver Forma normal, primeira (1FN)
- Projeto de banco de dados, 29
 - fase de projeto físico, 29
 - fase de projeto lógico, 29, 170, 252-253
 - objetivos, 138
 - passos, 140
 - princípios, 138-140
 - visão geral, 136-137
- Quarta forma normal, 206
- Redundância de dados, 20, 20-22
 - controlada, 21
 - não-controlada, 21, 21-22
- Redundância em modelos ER ver Modelo entidade-relacionamento
- Relacionamento, 36, 36-46, 144, 238-239, 240-241, 243, 245-246
 - 1:1** (um-para-um), 40-43, 147-152, 151
 - 1:n** (um-para-muitos), 40-43, 152-154
 - n:n** (muitos-para-muitos), 40-43, 154
 - transformação em entidade, 77, 155, 231-232
 - atributo, 49
 - auto-relacionamento, 38-39
 - binário, 40-41, 40-43
 - de grau maior que dois, 43-45, 154-156
 - identificador de, 53-54, 143, 163
 - implementação na abordagem relacional, 142-143
 - implementação, 144-156
 - adição de colunas, 146
 - fusão de tabelas, 146-147, 151
 - tabela própria, 144-145
 - modificado com o tempo, 90-93
 - mutuamente exclusivo, 167-169
 - ocorrência, 37
 - recursivo, 75, 236
 - redundante, 87
 - temporal, 90-93
 - ternário, 43-45, 155, 231
 - Restrição de integridade, 74, 123, 127-128, 244, 246, 248
 - semântica, 128
- Segunda forma normal, 197, 257-258
- SGBD ver Sistema de gerência de banco de dados
- Sistema de gerência de banco de dados, 23
 - pré-relacional, 184
 - relacional, 26, 120, 135
 - linguagem, 130, 141
- Sistema legado, 184
- SQL, 128, 130-131
- Tabela, 120-122
 - aninhada, 187, 190
 - e engenharia reversa, 170-172
 - implementação inicial, 140-147
 - não-normalizada, 187-190
 - não-primeira-forma-normal, 187-190
- Tempo de armazenagem de dados, 93
- Terceira forma normal, 200, 258
- Tupla ver Linha
- Valor de atributo ver Campo
- Vazio (Valor de campo), 126-127, 140